

A decorative background graphic on the left side of the slide, consisting of several overlapping, curved, light blue shapes that resemble stylized waves or a large, abstract letter 'C'.

第5章 SDN南向接口协议

目录

01

OpenFlow总述

02

OpenFlow协议

03

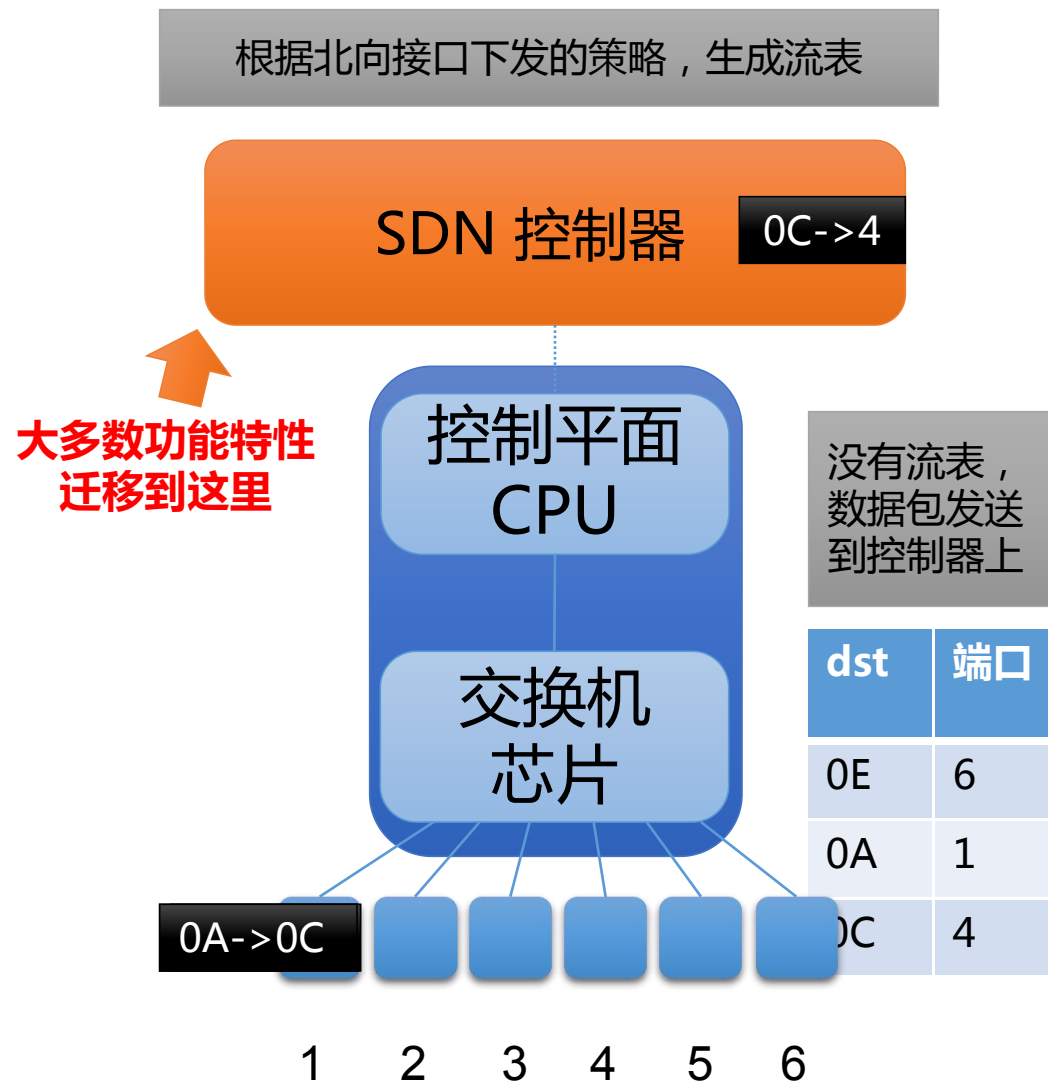
OpenFlow流表演进

04

OpenFlow1.3 规范

交换机的体系结构：控制平面与数据平面的交互

- 当今的交换机
 - 控制/数据平面均在交换机上
 - 数据平面：速度快，读取转发表项
 - 控制平面：缓慢，写入转发表项
- SDN
 - 控制/数据平面隔离开来
 - 数据平面在交换机上
 - 控制平面在别处，如 x86 服务器上，可以完成更重要的工作



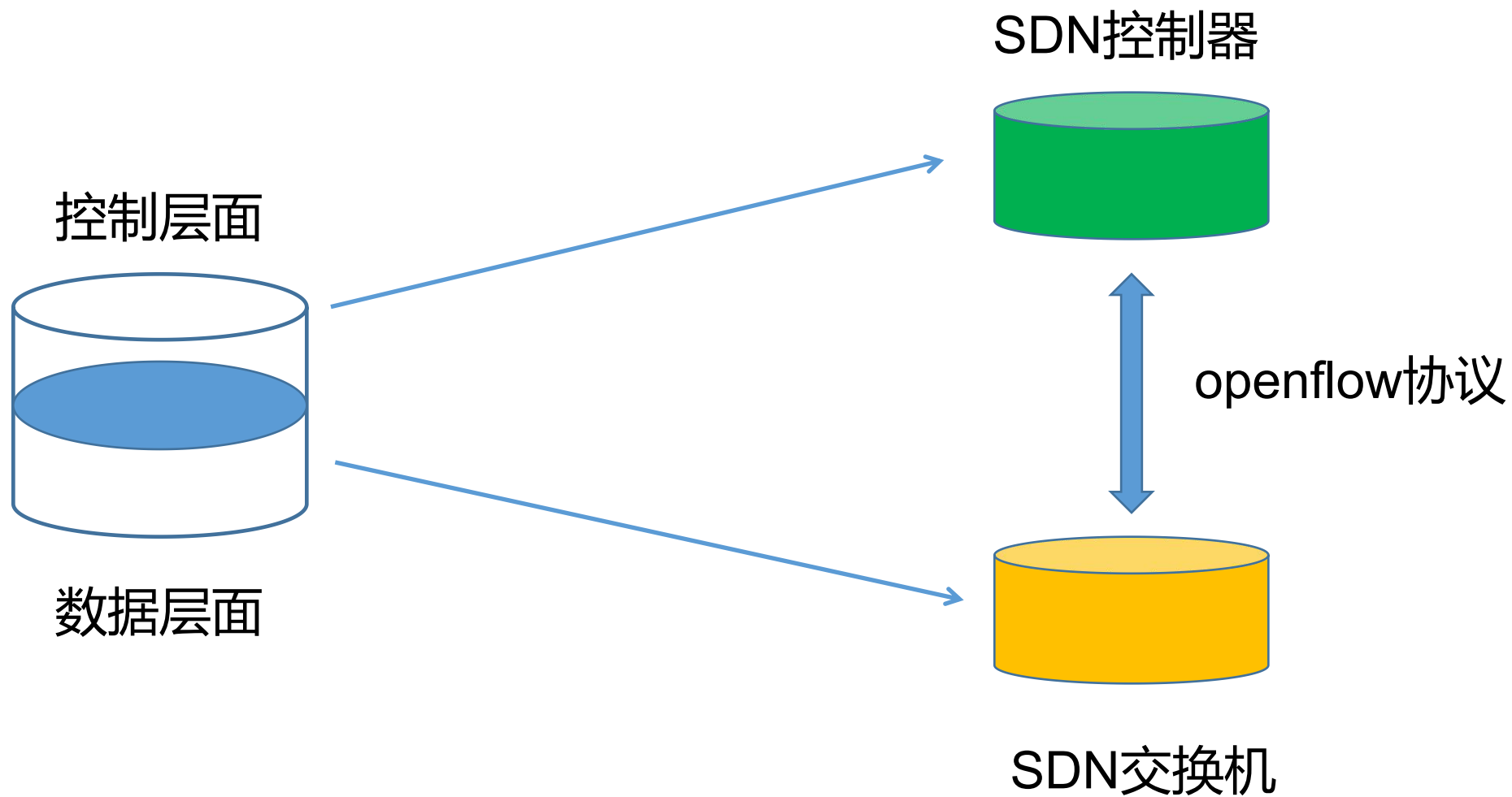
OpenFlow协议在SDN当中是一个灵魂性的知识。2006年Nick教授提出了OpenFlow协议，然后2008年发表了OpenFlow协议的论文：

OpenFlow:enable innovation in campus networks

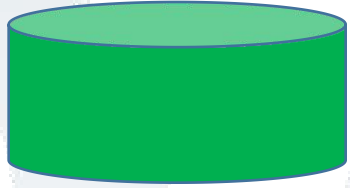
在其基础上Nick教师进一步提出了SDN的最早概念。

openflow这么重要的课程需要更多的时间

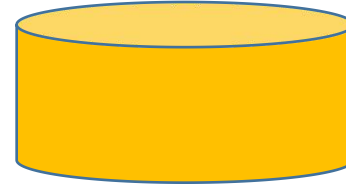
为什么需要OpenFlow协议



SDN控制器

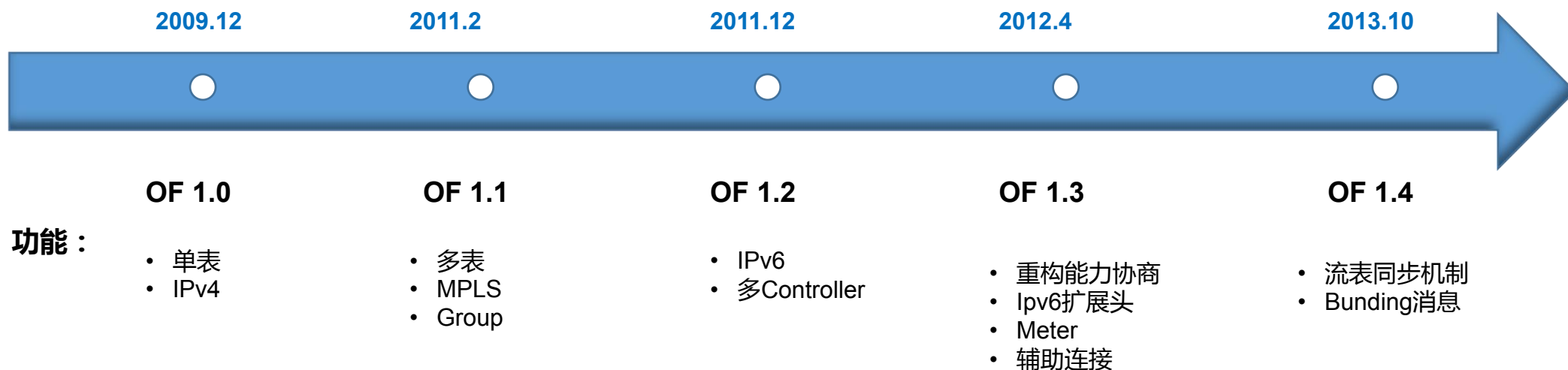


SDN交换机



南向接口协议：OpenFlow

- ◆ 典型的 SDN 南向协议有 **OpenFlow** 协议等。OpenFlow 协议可以通过下发流表项来对数据平面设备的网络数据处理逻辑进行编程，从而实现可编程定义的网络。所以狭义 SDN 南向协议的关键在于是否具有确切的数据平面可编程能力。



- ◆ Openflow 目前最高版本为1.5 但是业界使用最多的是1.0和1.3，其中1.3.X是LTS(long term support)版本。

简书

1.0 版本：1.0版本中只有单流表

1.1版本：为了避免单流表膨胀，1.1版本采用多级流表，以流水线的形式提高数据包匹配效率。

1.2 版本：由于OpenFlow将所有的控制协议都集中到控制器中，导致控制器成为众矢之的，为了提高安全性和健壮性，1.2版本引进了多控制器的概念。

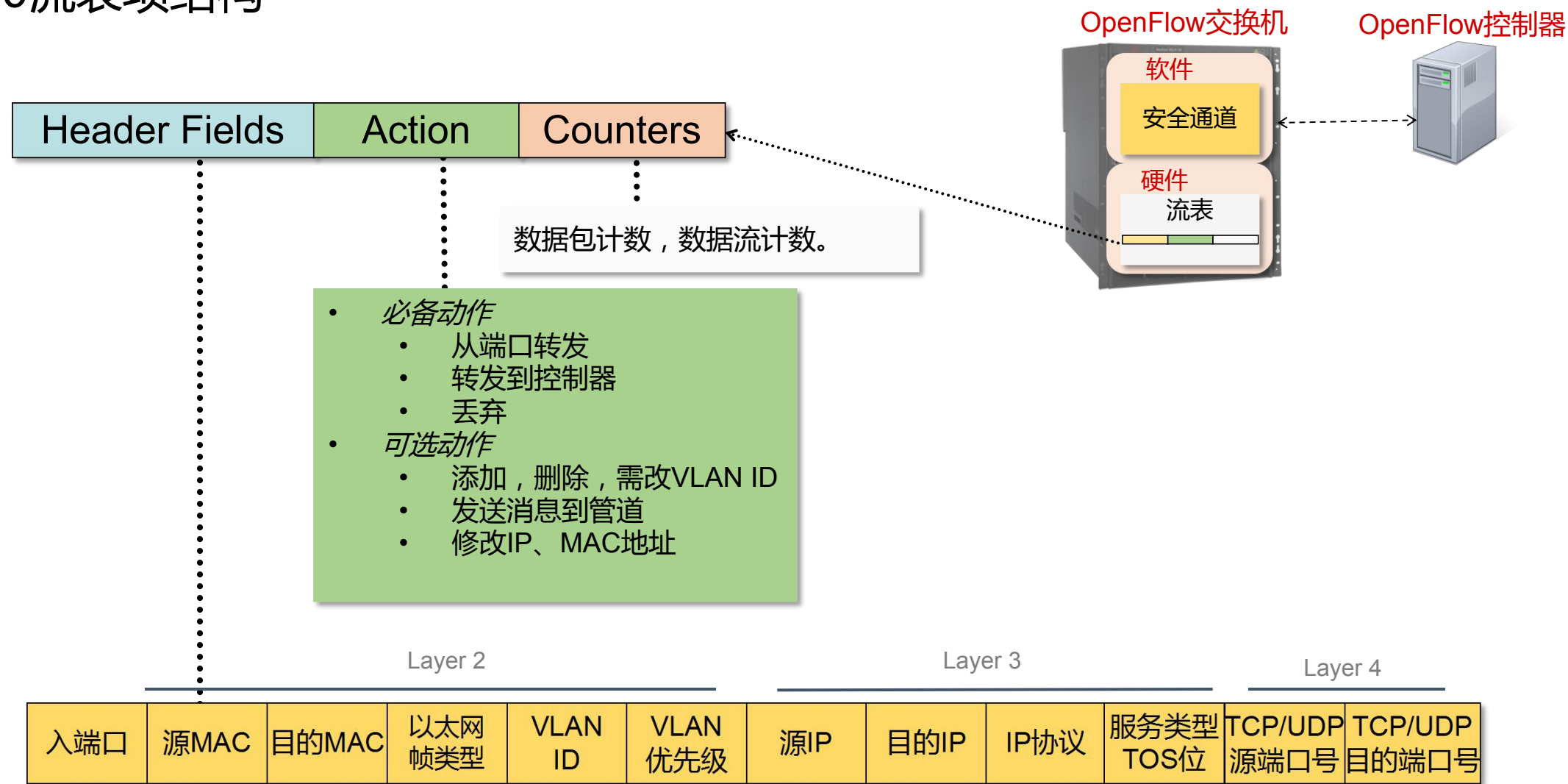
1.3 版本：增加了meter表，匹配项更加丰富。

1.5 版本：增加数据包类型识别流程，提升数据包的处理效率

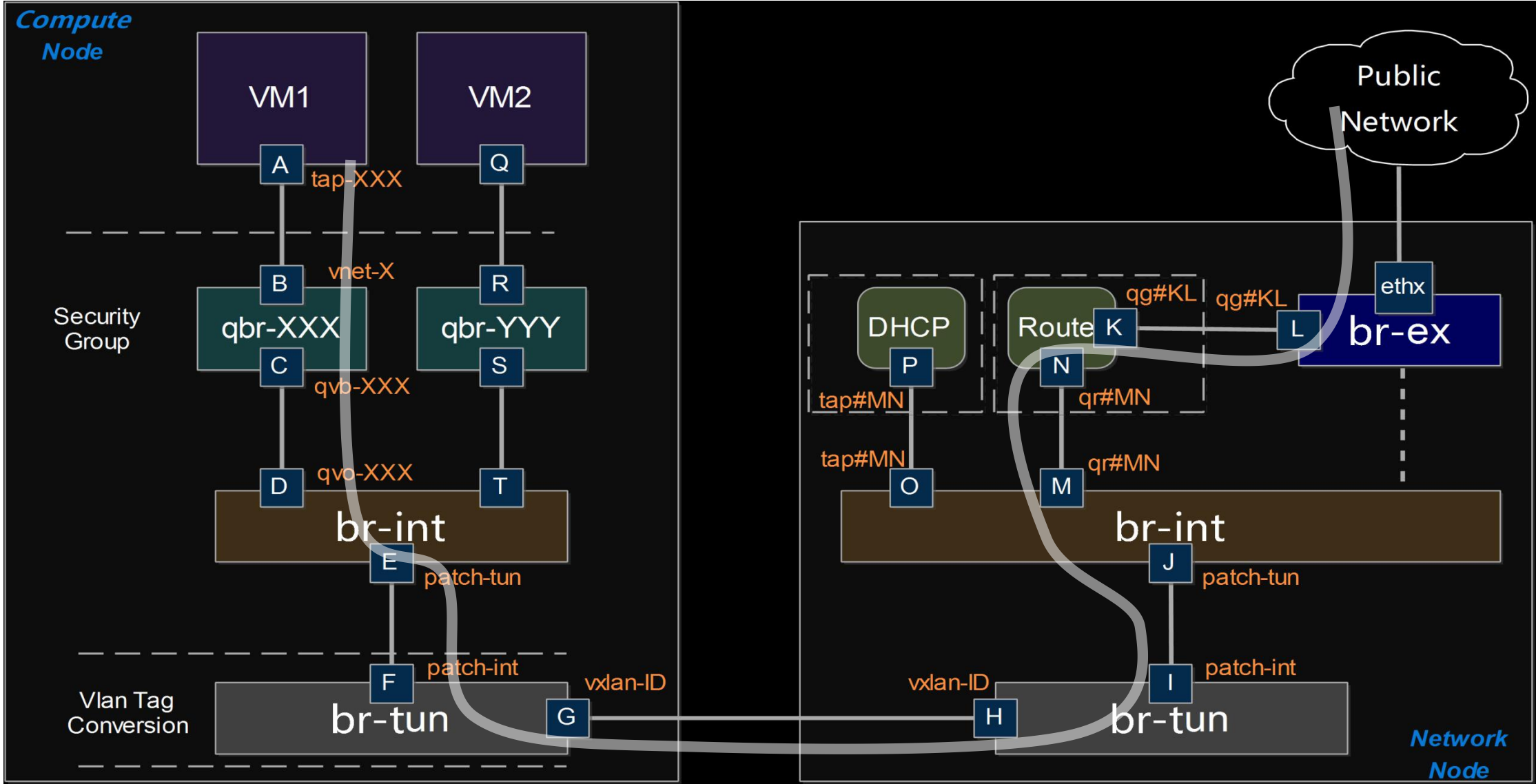
OpenFlow正在一点一点摸索一个平衡点，既能保证控制器充分的控制器主权又让交换机有一定的自主功能。或许，最初的OpenFlow十分理想化，但是在其不断演进的过程中我们可以看出OpenFlow在不断向实际应用场景靠拢。

南向接口协议：OpenFlow1.0流表

1.0流表项结构



Open vSwitch在OpenStack中的应用



控制器流表

```
root@controller:~# ovs-ofctl dump-flows br-int
```

```
NXST_FLOW reply (xid=0x4):
```

```
  cookie=0xbfcbbcc1f0281737c, duration=3728783.452s, table=0,  
  n_packets=88592050, n_bytes=8732527148, idle_age=0, hard_age=65534,  
  priority=2,in_port=1 actions=drop
```

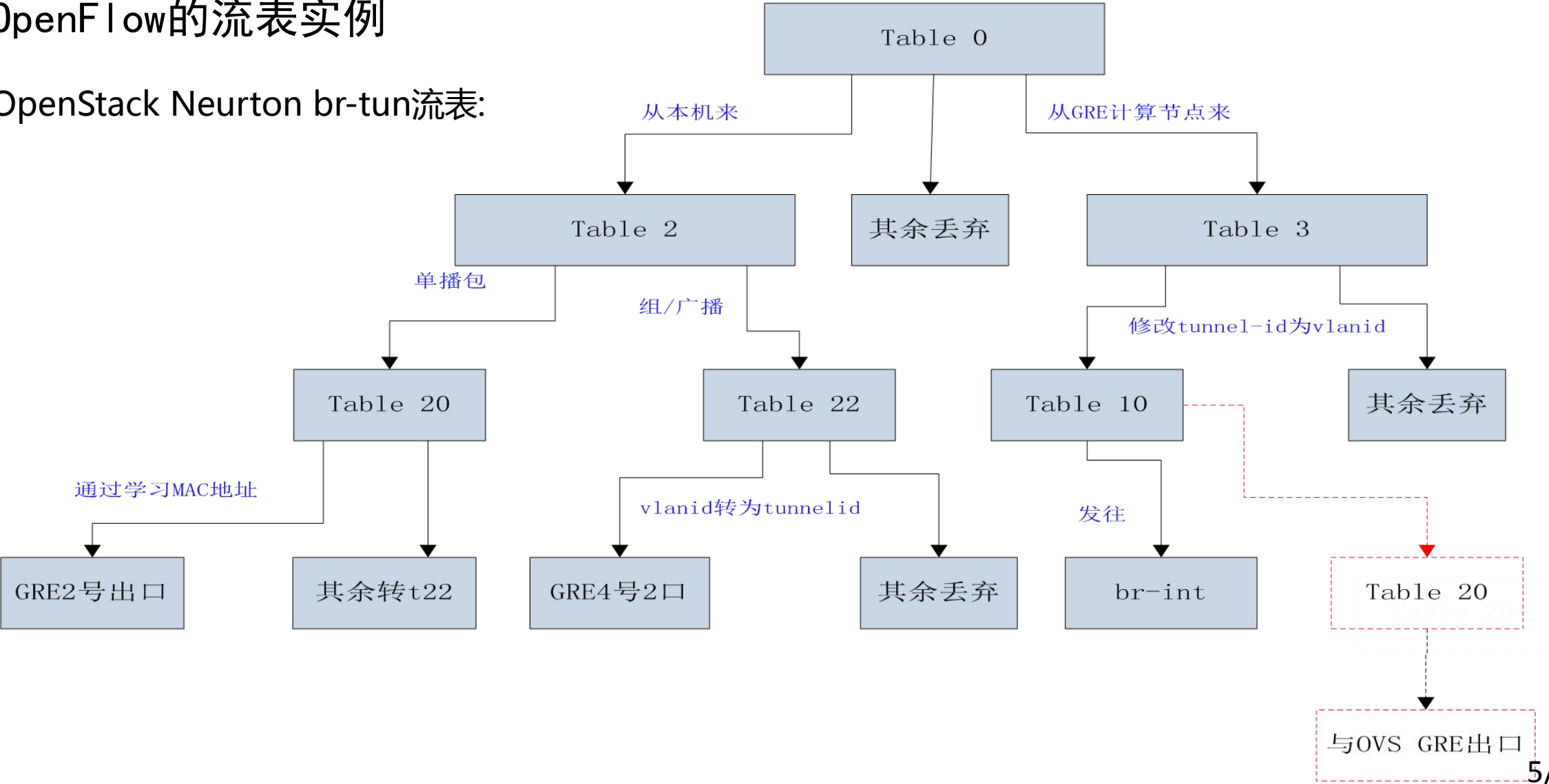
```
  cookie=0xbfcbbcc1f0281737c, duration=852955.679s, table=0, n_packets=0,  
  n_bytes=0, idle_age=65534, hard_age=65534, priority=2,in_port=22111  
  actions=drop
```

```
  cookie=0xbfcbbcc1f0281737c, duration=3728783.744s, table=0,  
  n_packets=333702223, n_bytes=331075888809, idle_age=299,  
  hard_age=65534, priority=0 actions=NORMAL
```

南向接口协议：OpenFlow

OpenFlow的流表实例

OpenStack Neutron br-tun流表:



目录

01

OpenFlow总述

02

OpenFlow 协议

03

OpenFlow流表演进

04

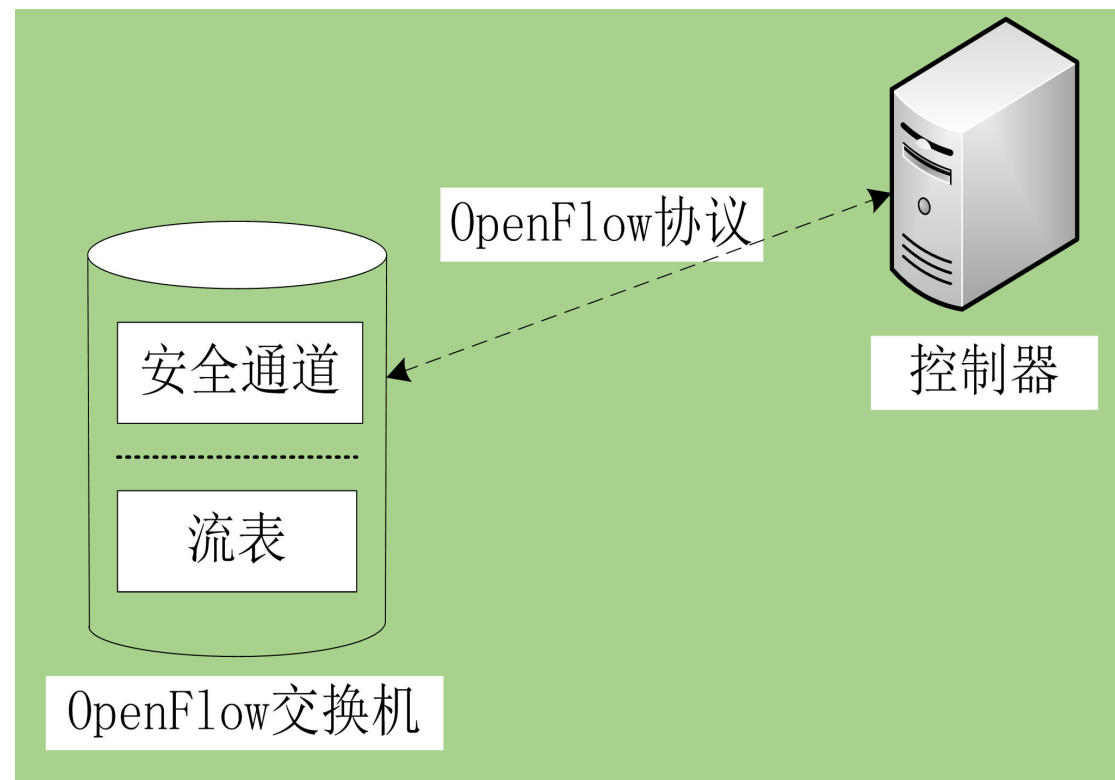
OpenFlow1.3 规范

OpenFlow—OpenFlow 1.0 交换机组成

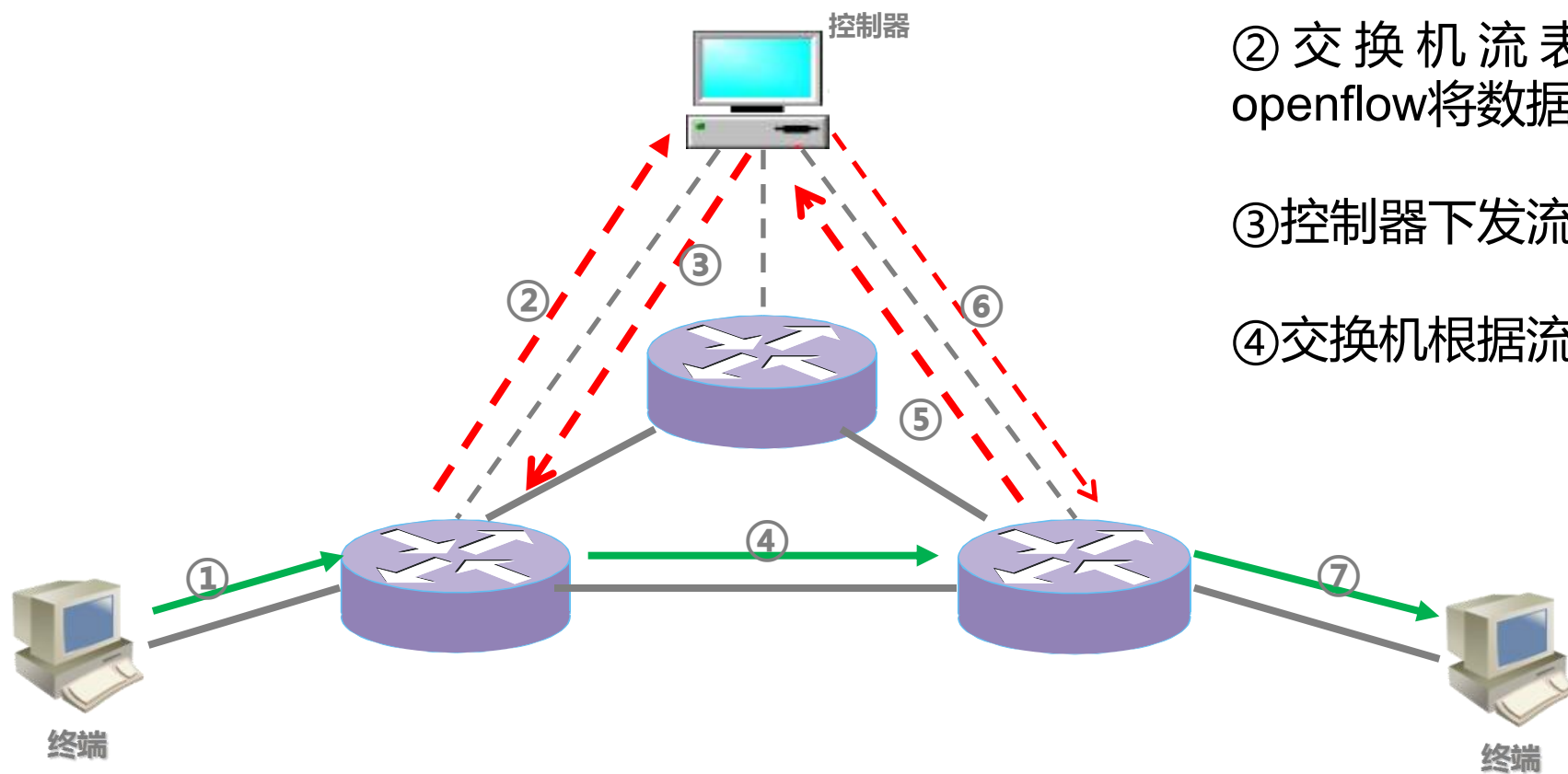
◆ OpenFlow协议的基本架构原理如右图，OF交换机利用基于安全连接的OpenFlow协议与控制器互相通信

◆ 核心概念

- ① 流表
- ② 安全通道
- ③ OpenFlow协议



OpenFlow1.0 — OpenFlow在SDN中职责



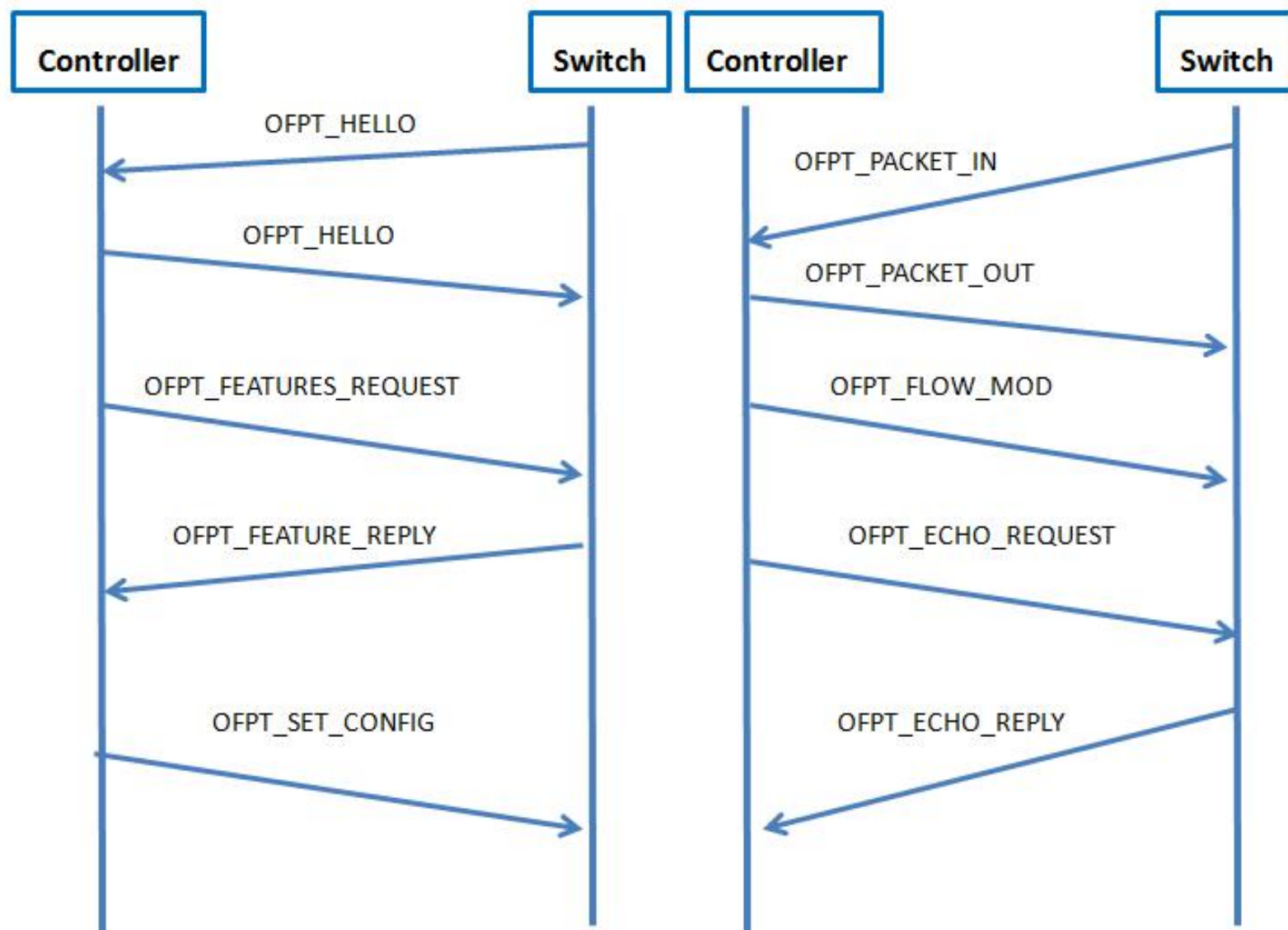
①主机向网络发送数据包

②交换机流表无匹配项，通过openflow将数据包上报给控制器

③控制器下发流表到交换机

④交换机根据流表转发数据包

OpenFlow协议1.0版本消息交互过程





HELLO

HELLO

FEATURE_REQUES
T

FEATURE_REPLY

```
root@openlab:~# ovs-ofctl show br0
OFPT_FEATURES_REPLY (xid=0x2): dpid:00007ed8df73414e
n_tables:254, n_buffers:256
capabilities: FLOW_STATS TABLE_STATS PORT_STATS QUEUE_STATS ARP_MATCH_IP
actions: OUTPUT SET_VLAN_VID SET_VLAN_PCP STRIP_VLAN SET_DL_SRC SET_DL_DST SET_NW_SRC SET_NW_DST
T ENQUEUE
LOCAL(br0): addr:7e:d8:df:73:41:4e
    config:    PORT_DOWN
    state:     LINK_DOWN
    speed: 0 Mbps now, 0 Mbps max
OFPT_GET_CONFIG_REPLY (xid=0x4): frags=normal miss_send_len=0
```

OpenFlow1.0 —主要协议交互过程之版本协商

◆ 当OF安全通道建立起来后，双方必须首先发送HELLO消息

◆ Hello消息中只包含OpenFlow Header

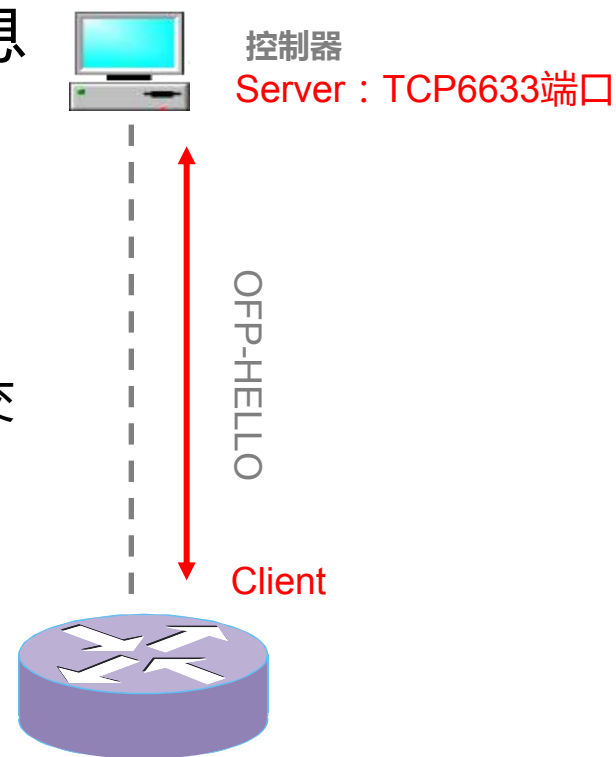
◆ OpenFlow Header中的version字段为发送方所支持的最高版本

OpenFlow协议（OpenFlow 1.0中版本信息为0x01）

◆ 通常情况下，安全通道中使用的版本号为：控制器发送版本号与交换机版本号中较小的一个

◆ 如果双方协商版本不一致时，应发送Error消息后断开连接

◆ 如果双方OpenFlow版本可以兼容，则OpenFlow连接建立成功。



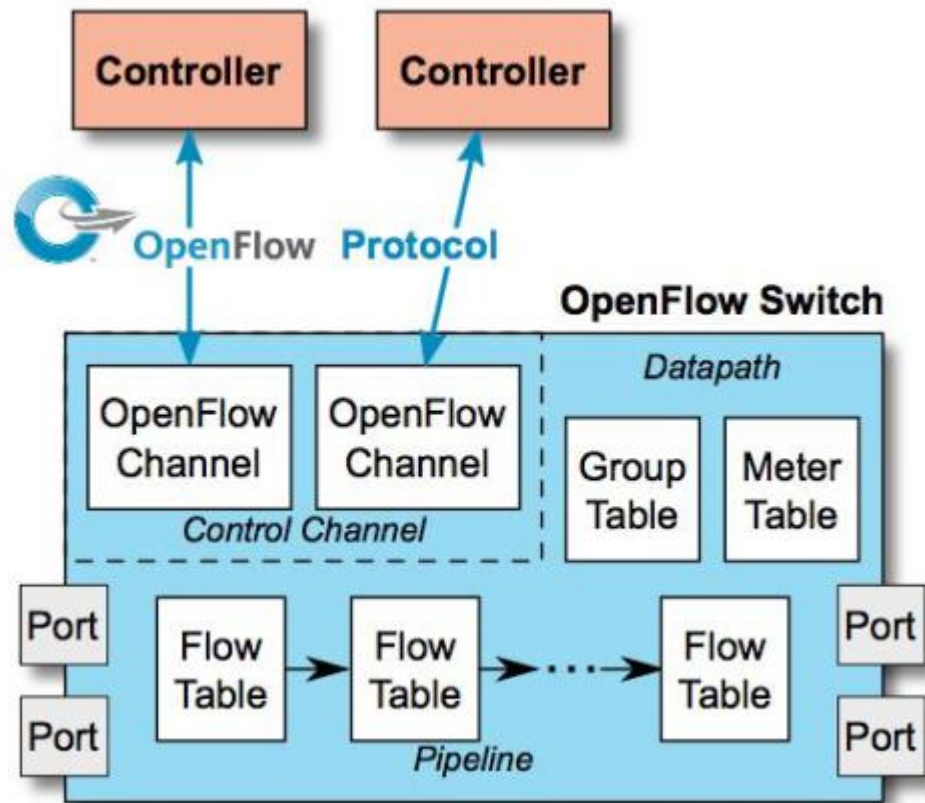
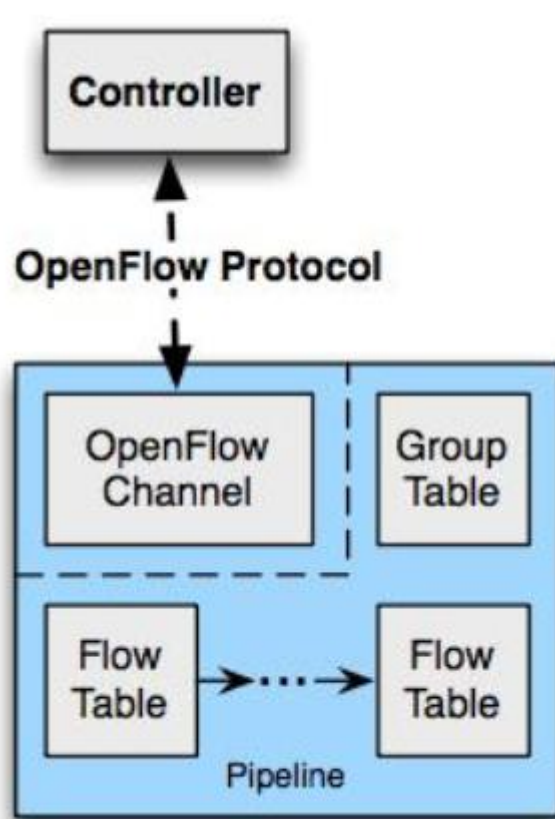
OpenFlow 中的术语

术语	描述
Byte	一个8bit的序列，网络处理基本单元
Packet	以太帧，包括 header 和payload
Port	packets 进出OpenFlow pipeline 的地方，由物理端口， Switch 定义的logic端口， OpenFlow 协议定义的 reserved端口
Pipeline	OFS 中实现matching、转发和 packet 修改的flow table流水线
Flow table	pipeline 的一个stage，包含 flow entries
Flow entry	flow table 中的元素，用于 packets查找和处理，包括用于包匹配的一组 match fields，描述匹配顺序的 priority，跟踪记录 packets 的一组counters，一组待用的 instructions
Match Field	packet 匹配查找时使用的域，包括 packet header、输入端口和 metadata值，一个 match field 可以是通配符（匹配任意值），也可以是 bitmasked 型
Metadata	一个 Maskable register value 用于table 之间的信息传递
Instruction	当一个 packet 匹配某个flow entry 时，对应的 instruction 描述特定的 OpenFlow处理。一个 instruction 可以修改pipeline processing 过程，比如直接把 packets 发给另一个 flow table，也可以包含一组 actions 用来添加到 action set，或包含一个 action list立即应用给 packet

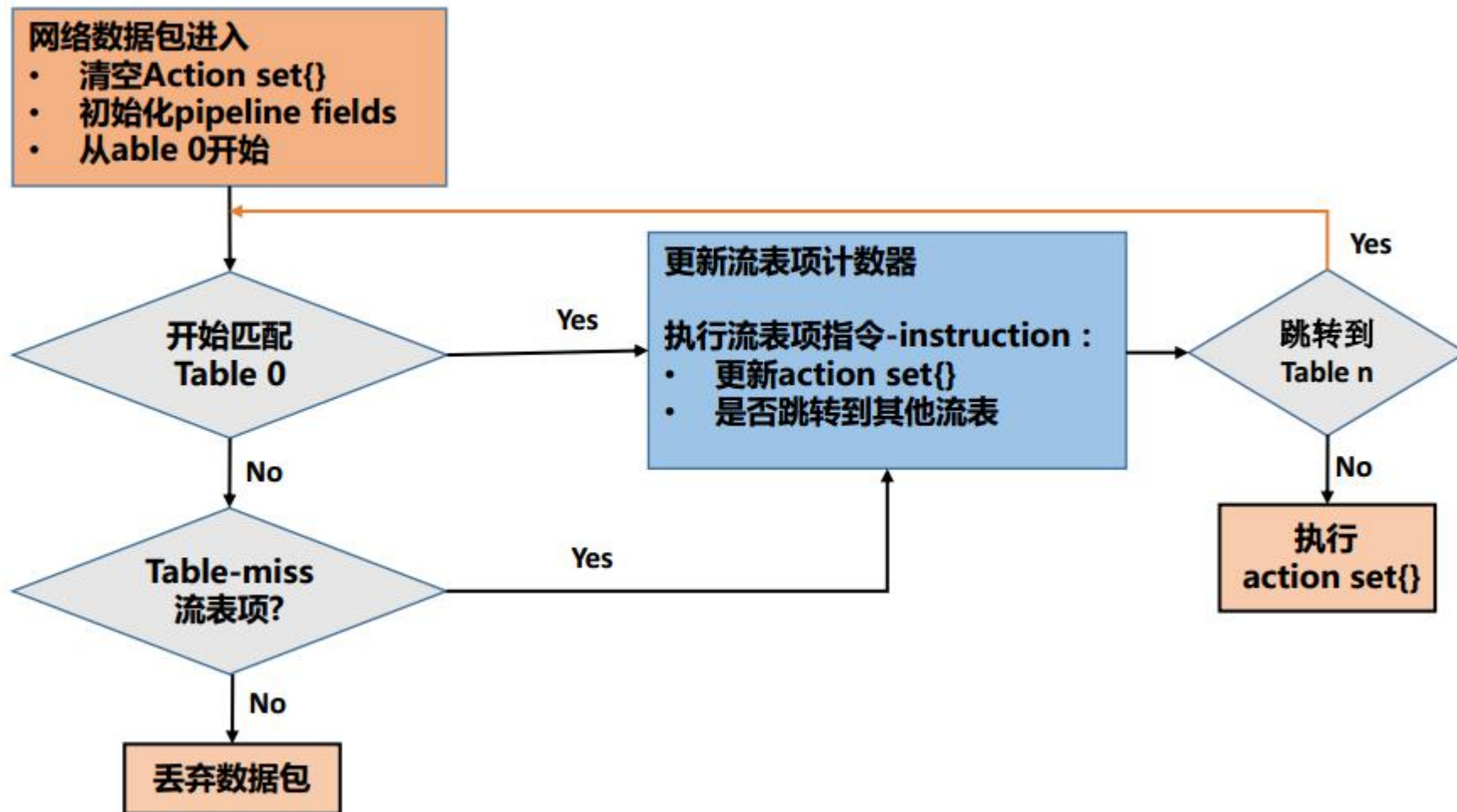
OpenFlow 中的术语

术语	描述
Action	转发 packet 到某个端口或修改 packet 的一个操作，比如减小 TTL 值。Actions 也可作为 flow entry 对应的instruction set 的一部分，或者在 group entry 对应的一个 action bucket 中。Actions 可以被添加到 packet 的 action set 中，也可以立即应用给匹配的 packet
Action set	与 packet 相关的一组 actions，当每个 flow table 处理 packet 时会添加 action set，当 instruction set 表示 packet 会送出 processing pipeline 时，action set 会被执行
Group	一个 action buckets 列表，从其中选择一个或多个 buckets 用在每个 packet 上
Action bucket	一组 actions 以及相关参数，在 group 中定义
Tag	通过 push 和 pop actions 插入 packet 或从 packet 删除的一个 header
Outermost tag	离 packet 起始位置最近的一个 tag
Controller	通过 Openflow 协议与 OVS 交互的设备
Meter	一个 switch 基本单位，能测量和控制 packets 速率，如果 packet 速率或 byte 速率经过 meter 超出预定义的范围，meter 会激活一个 meter band，如果 meter band drop packet，就是所谓的 Rate Limiter

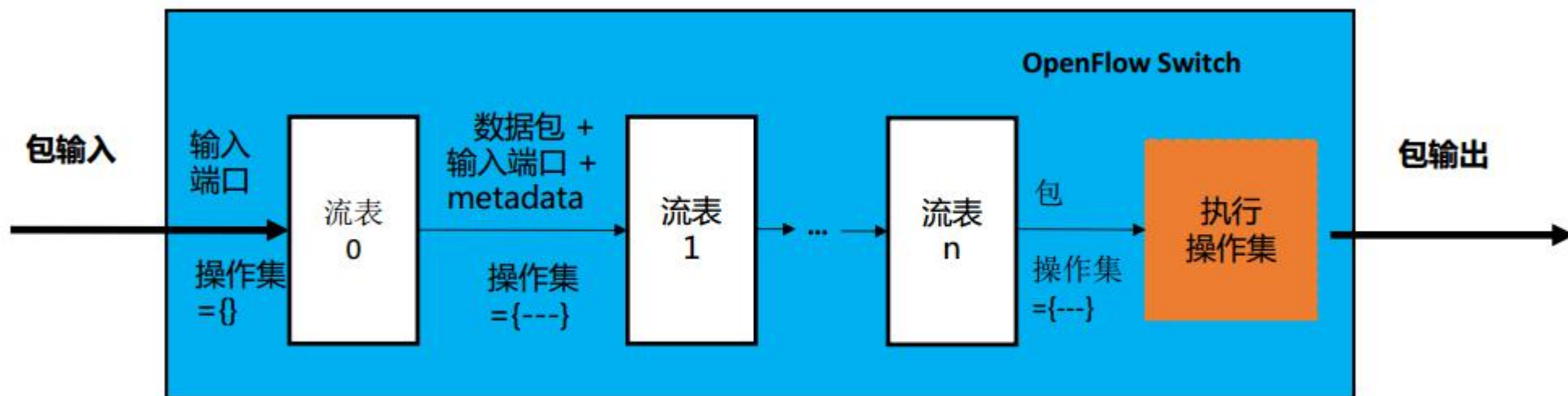
OpenFlow switch-通用转发抽象模型



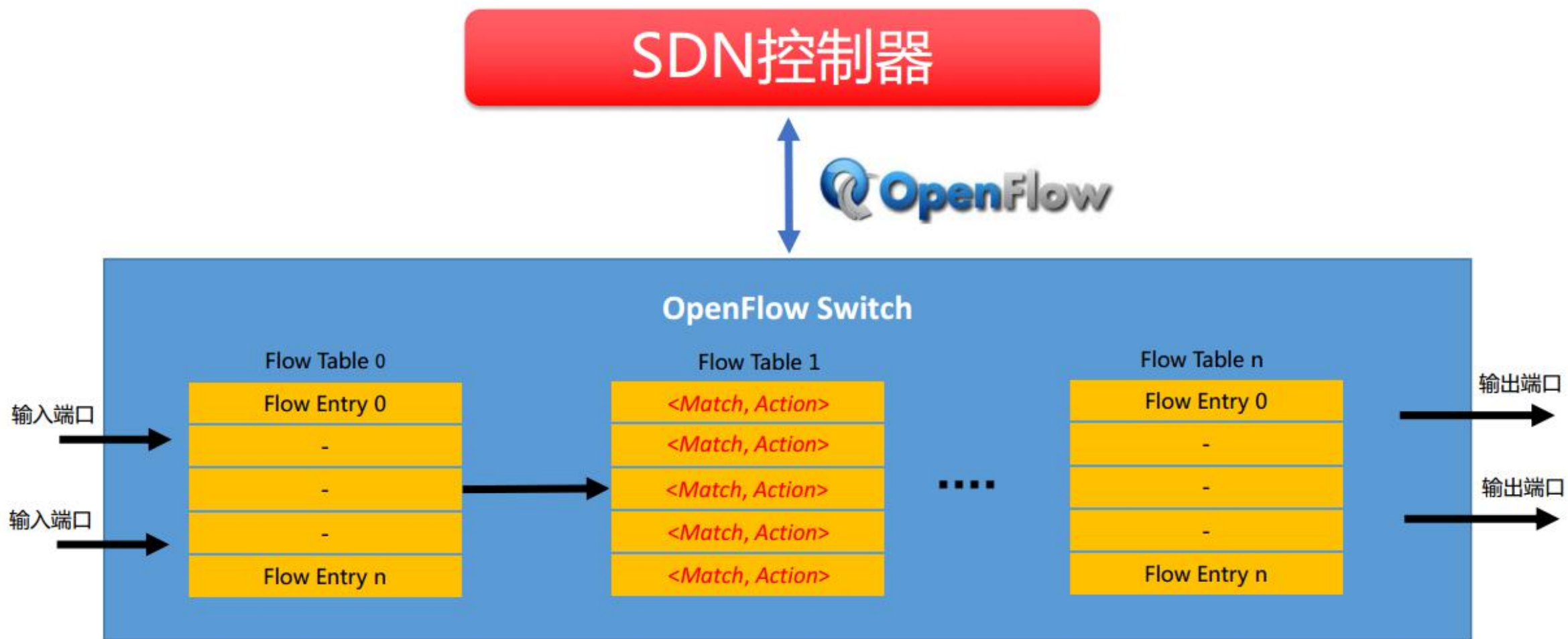
Flow Table处理流程



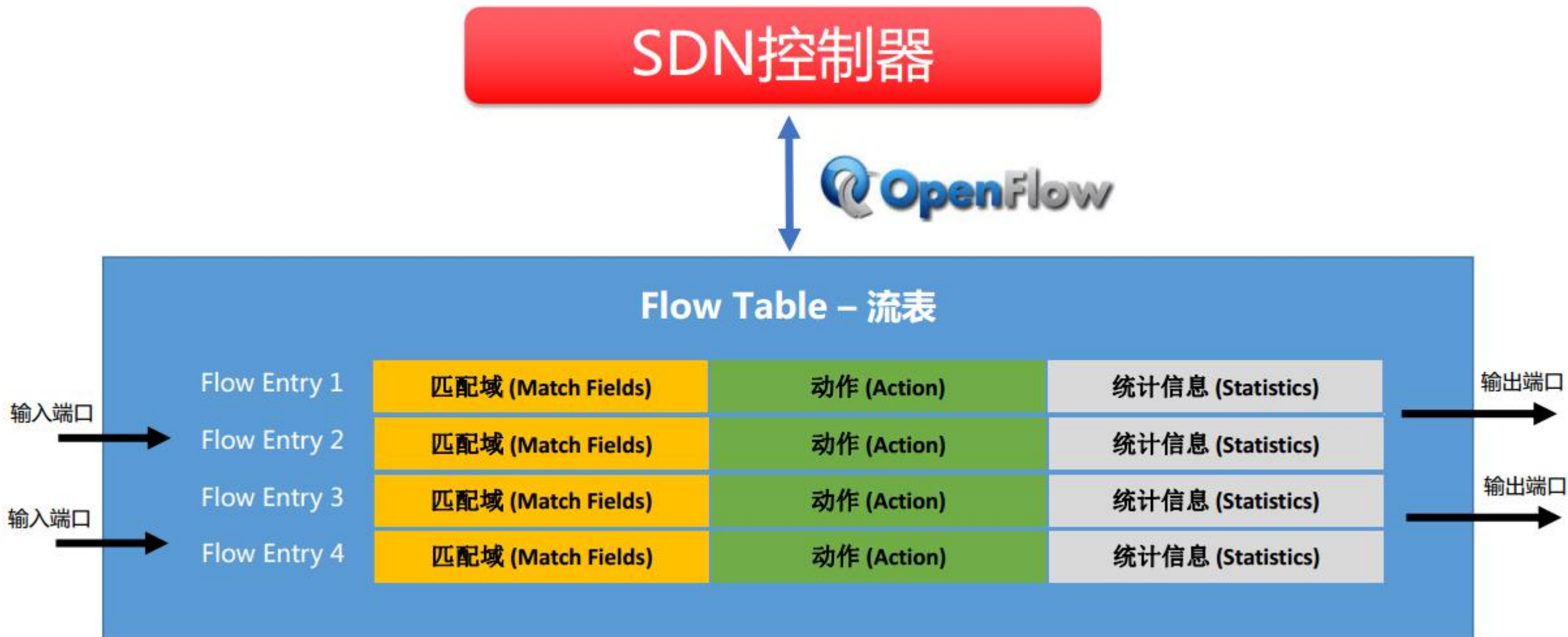
多级流表处理一览



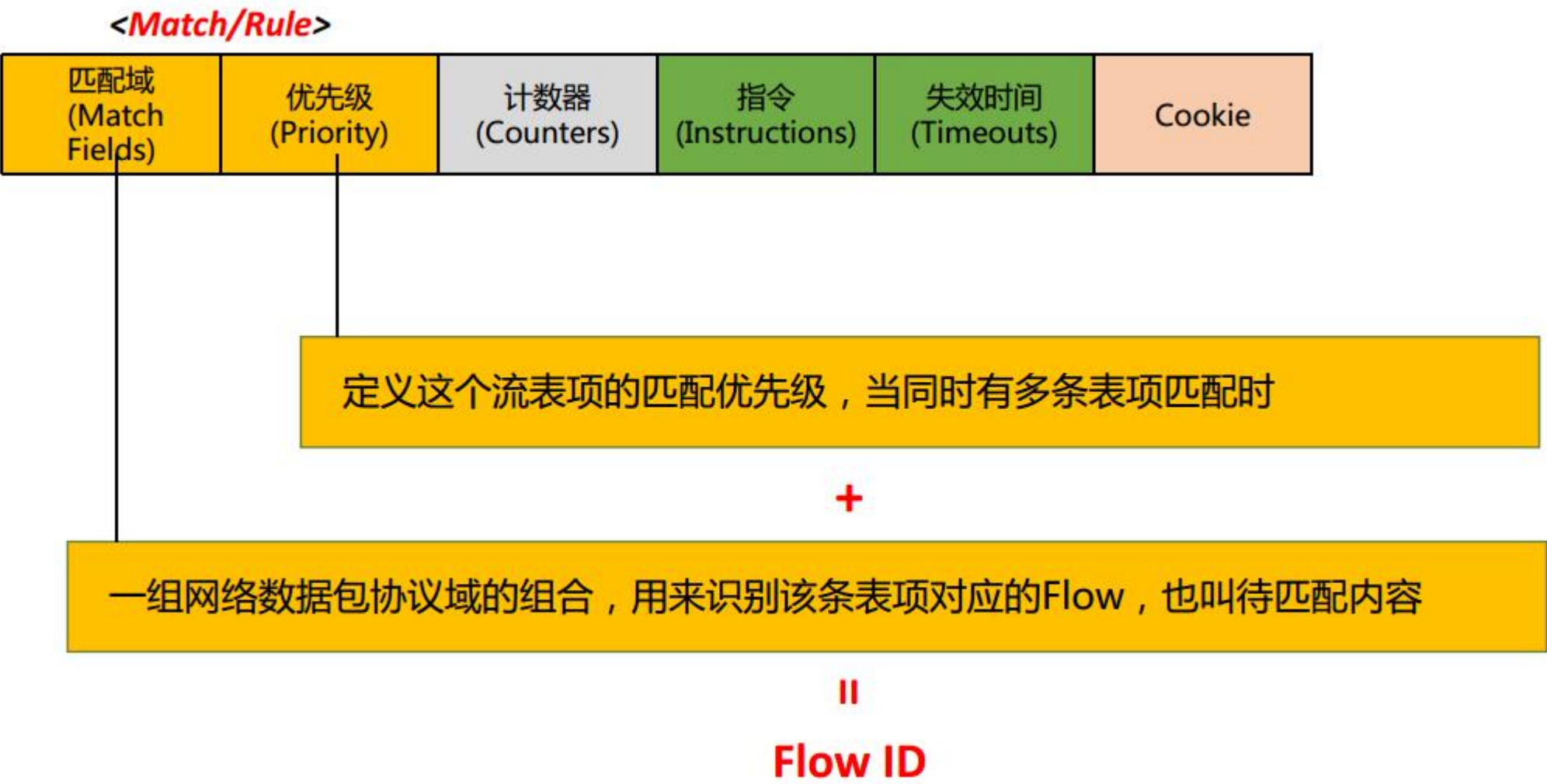
Flow Table--流表子模型



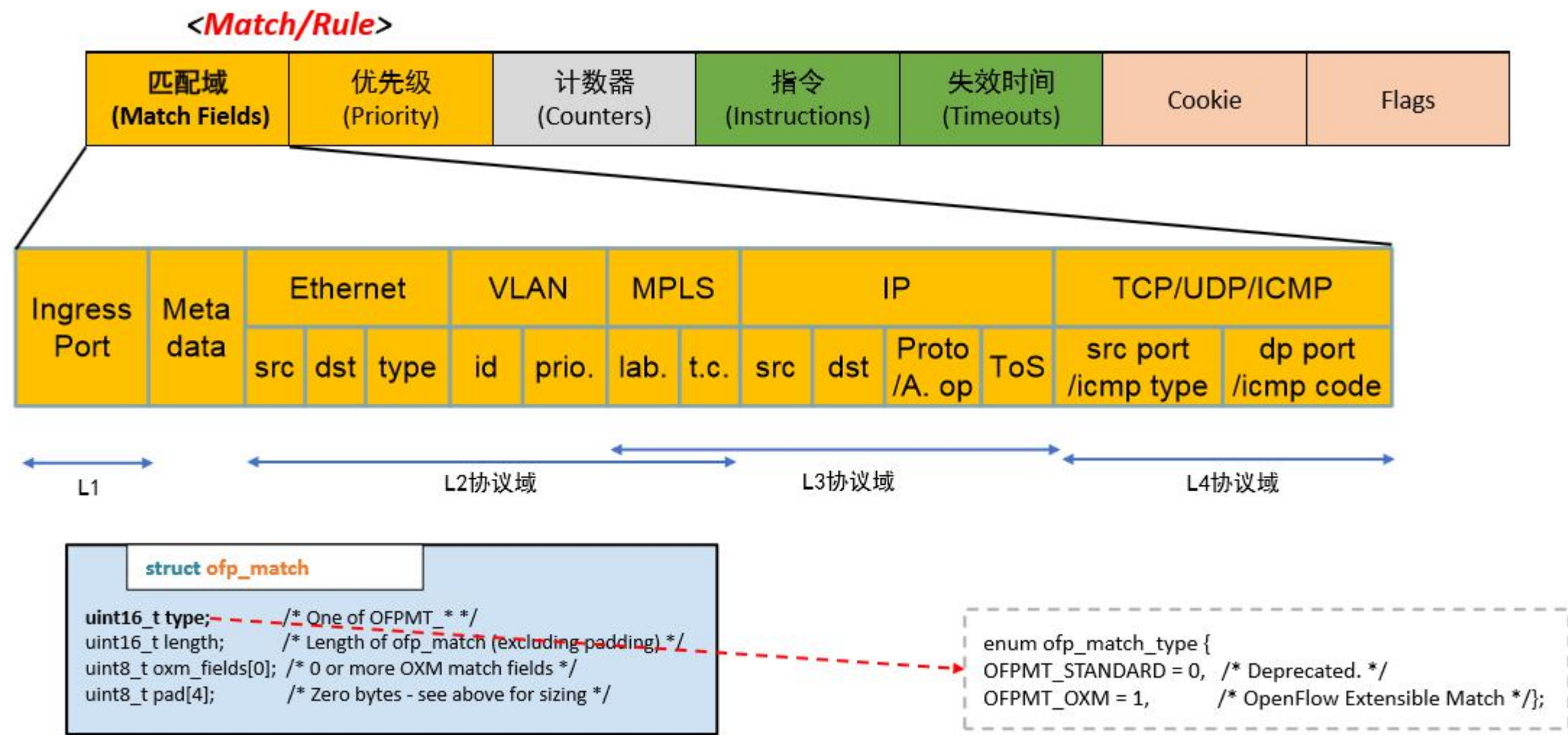
Flow Table--流表子模型



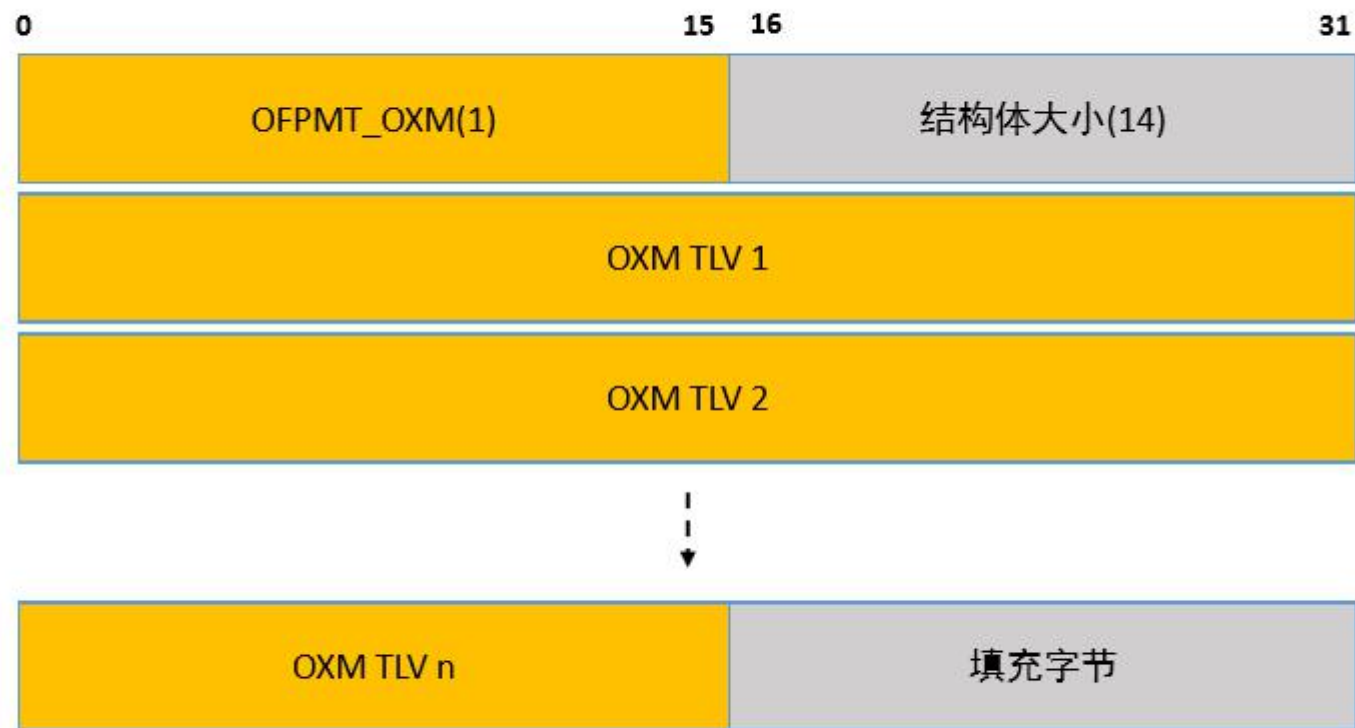
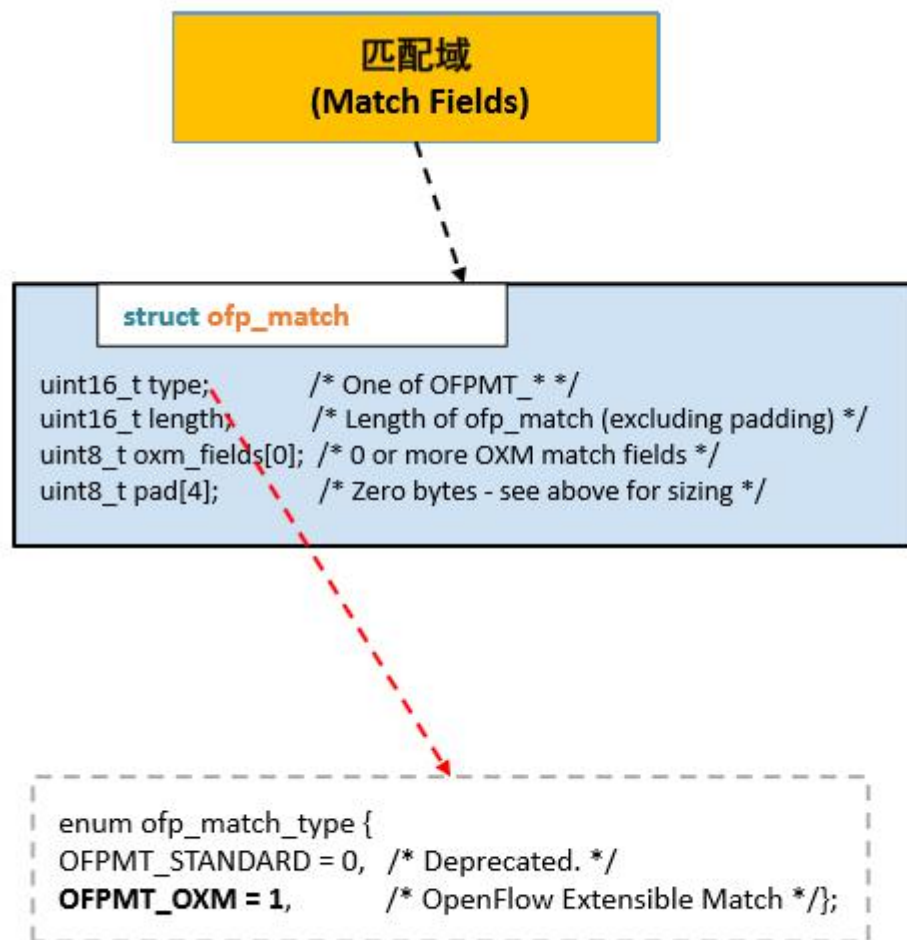
Flow Entry—流表项



Match Fields —— 流表项匹配域



OXM的TLV匹配域描述



Instructions —— 流表项指令

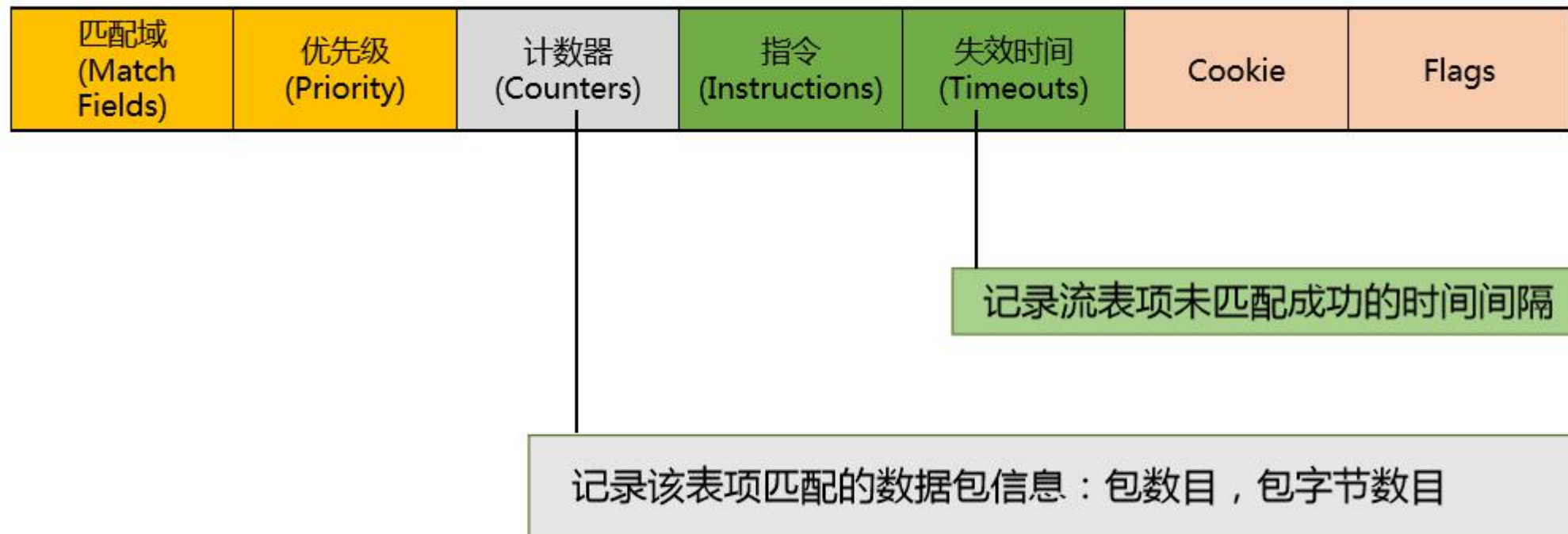
匹配域 (Match Fields)	优先级 (Priority)	计数器 (Counters)	指令 (Instructions)	失效时间 (Timeouts)	Cookie	Flags
--------------------------	-------------------	-------------------	----------------------	--------------------	--------	-------

当数据包匹配该条流表项的匹配域时

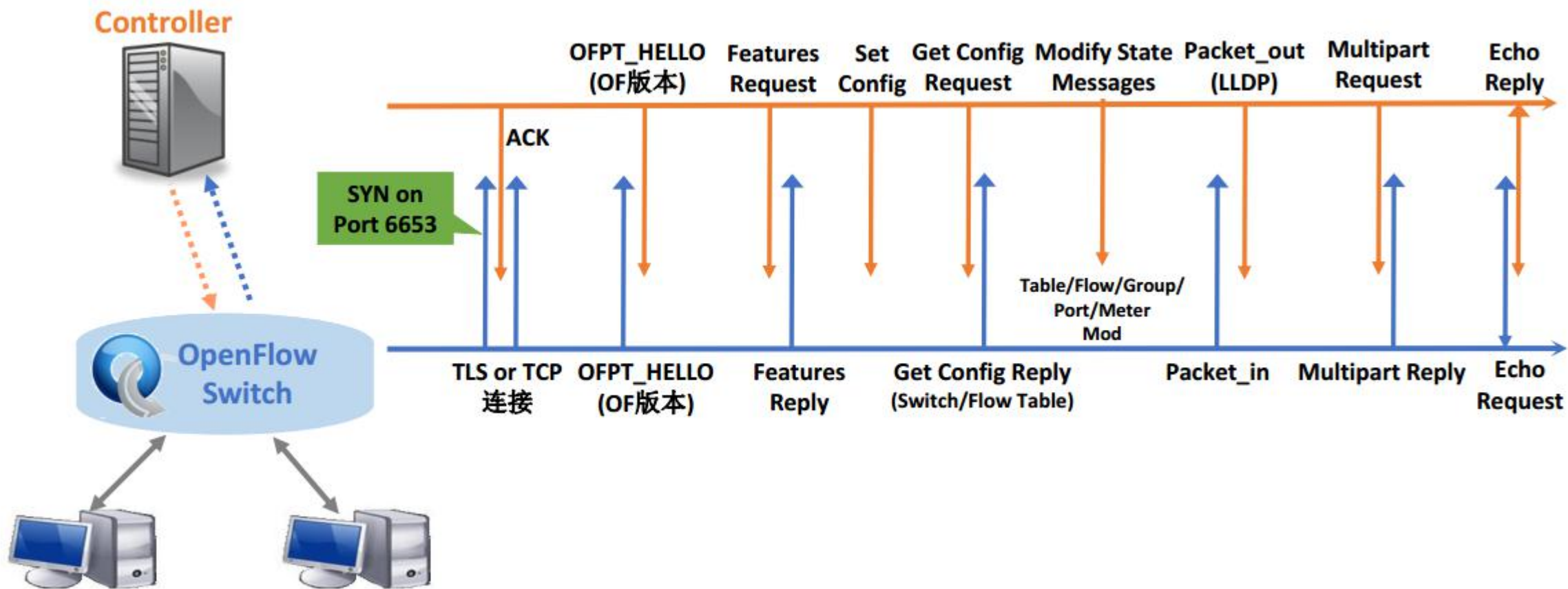
1.修改或操作指令集(Action Set)

2.修改多级流表处理

流表项计数器与失效时间



OpenFlow 协议



OpenFlow 消息

消息类型	消息例子	描述
Controller-to-Switch	• Modify State • Packet out • Switch Configuration • Barrier • Switch Features • Multipart • Role Request	• 控制器使用这些消息可以添加、修改或删除流表项 • 查询交换机的功能和统计， • 配置交换机， • 设置交换机端口属性 • 将数据包发送出指定交换机端口。
Asynchronous (异步)	• Error • Packet In • Flow Removed • Port Status • Table Status • Controller Role Status	• 由交换机发起，发送消息给控制器 • 这些消息可以是没有匹配交换机中任意流表项的数据包或数据包头，因此需要发送给控制器进行处理， • 在流状态、端口状态改变，或者产生一个错误消息时，进行通知
Symmetric (对称)	• Hello • Echo • Experimenter	• Hello消息在用户建立连接时控制器和交换机之间使用； • Echo消息用来确定Controller-to-Switch连接的延时，验证Controller-to-Switch连接是否保持活跃的状态； • Experimenter消息用来未来的消息扩展。

Controller-to-Switch消息

子类型	消息名称	功能描述
Switch features	OFPT_FEATURES_REQUEST OFPT_FEATURES_REPLY	控制器获取交换机的功能信息：Datapath ID (64bits), n_buffers等
Switch Configuration	OFPT_GET_CONFIG_REQUEST OFPT_GET_CONFIG_REPLY OFPT_SET_CONFIG	控制器会设置和查询交换机配置参数
Modify-State	OFPT_TABLE/FLOW/GROUP/PORT/ METER_MOD	控制器配置或更新Flow Table的动态信息： 添加、修改或删除flow/group/meter表项 新增或删除组表的action buckets 设置switch端口参数
Multipart	OFPMP_FLOW/AGGREGATE/GROU P/GROUP_DESC/GROUP_FEATURES	查询特定Flow的statistics/description/features信息
Packet-out	OFPT_PACKET_OUT	控制器指定从交换机某个端口发送数据包 转发通过Packet-in消息接收的数据包
Barrier	OFPT_BUNDLE_CONTROL	控制器用来确保request/reply 消息之间的独立性
Role-Request	OFPT_ROLE_REQUEST OFPT_GET_ASYNC_REQUEST OFPT_GET_ASYNC_REPLY OFPT_SET_ASYNC	设置控制器与交换机连接通道的一些信息 设置一个异步消息过滤器

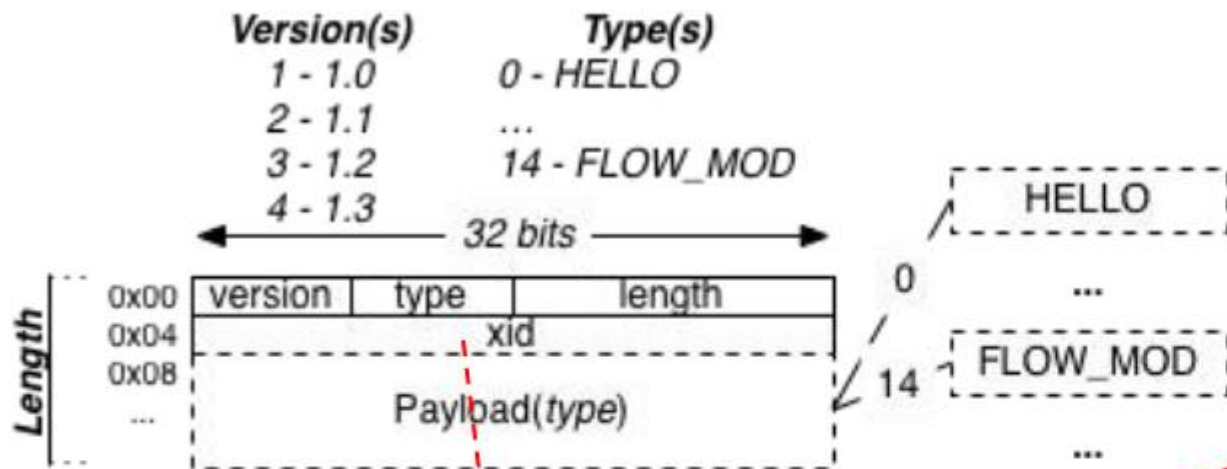
异步消息

子类型	消息名称	功能描述
Packet-in	OFPT_PACKET_IN	交换机将数据包可以发送给控制器
Flow-Removed	OFPT_FLOW_REMOVED	流表项失效时间满足时，OFS会向控制器发送该消息
Port-status	OFPT_PORT_STATUS	OFS会向控制器报告端口状态更新消息，添加，删除或更改端口
Controller Role status	OFPT_ROLE_STATUS	OFS会向控制器报告一个控制器角色更新消息
Table Status	OFPT_TABLE_STATUS	OFS会向控制器报告一个表状态更新消息
Request Forward	OFPT_REQUESTFORWARD	某个控制器成功修改表状态，OFS需要向其他控制器发送此消息
Controller Status	OFPT_CONTROLLER_STATUS	OFS需要向所有控制器报告某个控制器链接变化信息

对称消息

子类型	消息名称	功能描述
Hello	OFPT_HELLO	Hello消息在用户建立连接时控制器和交换机之间使用
Echo Request	OFPT_ECHO_REQUEST	Echo消息用来确定Controller-to-Switch连接的延时，验证Controller-to-Switch连接是否保持活跃的状态
Echo Reply	OFPT_ECHO_REPLY	Echo消息用来确定Controller-to-Switch连接的延时，验证Controller-to-Switch连接是否保持活跃的状态
Error	OFPT_ERROR_MSG	Error消息用来通知某种非正常情况
Experimenter	OFPT_EXPERIMENTER	Experimenter消息用于厂商或用户的消息扩展。

OpenFlow 消息格式



struct ofp_header

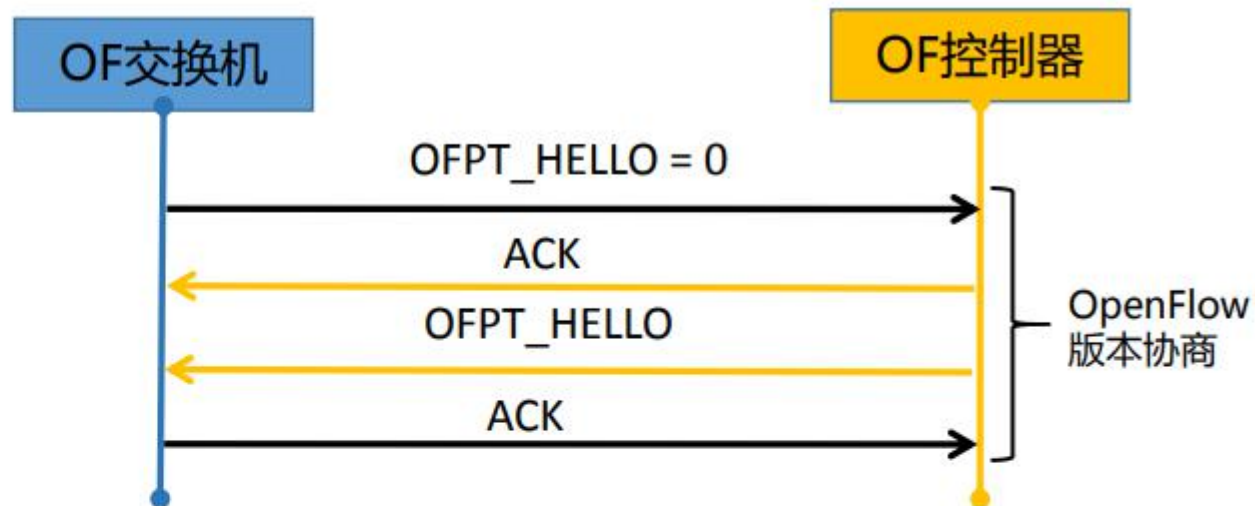
```
uint8_t version; /* OFP_VERSION. */
uint8_t type; /* One of the OFPT_constants. */
uint16_t length; /* Length including this ofp_header. */
uint32_t xid; /* Transaction id associated with this packet.
Replies use the same id as was in the request
to facilitate pairing. */
};OFP_ASSERT(sizeof(struct ofp_header) == 8);
```

```
enum ofp_type {
/* Immutable messages. */
OFPT_HELLO = 0, /* Symmetric message */
/* Switch configuration messages. */
OFPT_FEATURES_REQUEST = 5, /* Controller2switch message */
/* Asynchronous messages. */
OFPT_PACKET_IN = 10, /* Async message */
/* Controller command messages. */
OFPT_PACKET_OUT = 13, /* Controller/switch message */
/* Multipart messages. */
OFPT_MULTIPART_REQUEST = 18, /* Controller/switch message */
/* Barrier messages. */
OFPT_BARRIER_REQUEST = 20, /* Controller/switch message */
/* Controller role change request messages. */
OFPT_ROLE_REQUEST = 24, /* Controller/switch message */
/* Asynchronous message configuration. */
OFPT_GET_ASYNC_REQUEST = 26, /* Controller/switch
message */
/* Meters and rate limiters configuration messages. */
OFPT_METER_MOD = 29, /* Controller/switch message
*/
/* Controller role change event messages. */
OFPT_ROLE_STATUS = 30, /* Async message */
/* Asynchronous messages. */
OFPT_TABLE_STATUS = 31, /* Async message */
/* Request forwarding by the switch. */
OFPT_REQUESTFORWARD = 32, /* Async message */
/* Bundle operations (multiple messages as a single
operation). */
OFPT_BUNDLE_CONTROL = 33, /* Controller/switch
message */
/* Controller Status async message. */
OFPT_CONTROLLER_STATUS = 35, /* Async message
*/
}
```

Hello 消息

```
struct ofp_hello
```

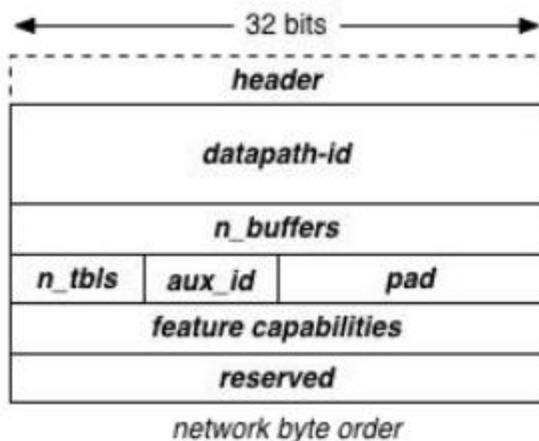
```
struct ofp_header header;  
/* Hello element list */  
struct ofp_hello_elem_header elements[0]; /* List of elements - 0 or more */  
};OFP_ASSERT(sizeof(struct ofp_hello) == 8);
```



Switch Features 消息

struct ofp_switch_features

```
struct ofp_header header;  
uint64_t datapath_id; /* Datapath unique ID. The lower 48-bits are for a MAC  
                        address, while the upper 16-bits are implementer-defined. */  
uint32_t n_buffers; /* Max packets buffered at once. */  
uint8_t n_tables; /* Number of tables supported by datapath. */  
uint8_t auxiliary_id; /* Identify auxiliary connections */  
uint8_t pad[2]; /* Align to 64-bits. */  
/* Features. */  
uint32_t capabilities; /* Bitmap of support "ofp_capabilities". */  
uint32_t reserved;  
}; OFP_ASSERT(sizeof(struct ofp_switch_features) == 32);
```



OF交换机

OF控制器

OFPT_FEATURES_REQUEST

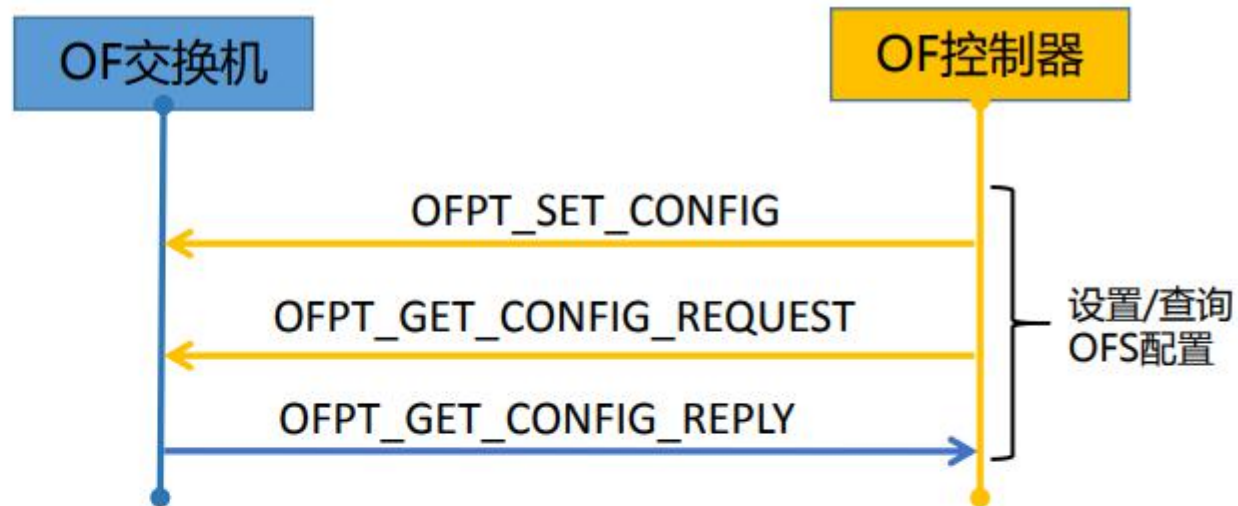
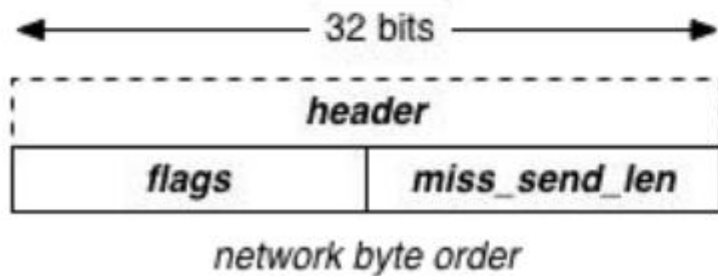
OFPT_FEATURES_REPLY

获取OFS
功能信息

```
enum ofp_capabilities {  
    OFPC_FLOW_STATS = 1 << 0, /* Flow statistics. */  
    OFPC_TABLE_STATS = 1 << 1, /* Table statistics. */  
    OFPC_PORT_STATS = 1 << 2, /* Port statistics. */  
    OFPC_GROUP_STATS = 1 << 3, /* Group statistics. */  
    OFPC_IP_REASM = 1 << 5, /* Can reassemble IP fragments. */  
    OFPC_QUEUE_STATS = 1 << 6, /* Queue statistics. */  
    OFPC_PORT_BLOCKED = 1 << 8, /* Switch will block looping ports. */  
    OFPC_BUNDLES = 1 << 9, /* Switch supports bundles. */  
    OFPC_FLOW_MONITORING = 1 << 10, /* Switch supports flow monitoring. */  
};
```


Switch 配置消息

```
struct ofp_switch_config
{
    struct ofp_header header;
    uint16_t flags; /* Bitmap of OFPC_* flags. */
    uint16_t miss_send_len; /* Max bytes of packet that datapath
                             should send to the controller. See
                             ofp_controller_max_len for valid values. */
}; OFP_ASSERT(sizeof(struct ofp_switch_config) == 12);
```

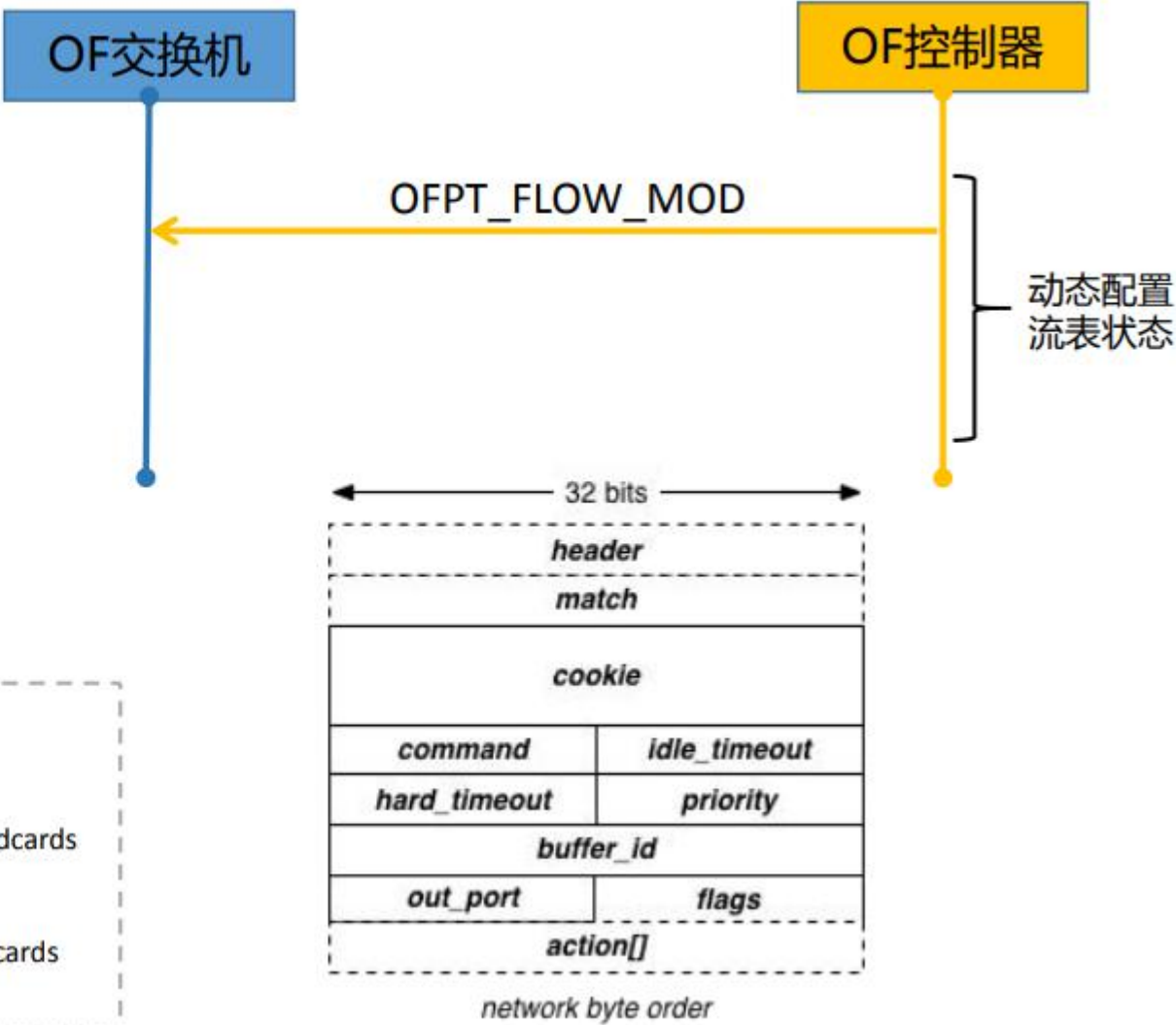


```
enum ofp_config_flags {
    /* Handling of IP fragments. */
    OFPC_FRAG_NORMAL = 0, /* No special handling for fragments. */
    OFPC_FRAG_DROP = 1 << 0, /* Drop fragments. */
    OFPC_FRAG_REASM = 1 << 1, /* Reassemble (only if OFPC_IP_REASM set). */
    OFPC_FRAG_MASK = 3, /* Bitmask of flags dealing with frag. */
};
```

Flow Entry Mod 消息

```
struct ofp_flow_mod
{
    struct ofp_header header;
    uint64_t cookie; /* Opaque controller-issued identifier. */
    uint64_t cookie_mask;
    uint8_t table_id;
    uint8_t command; /* One of OFPFC_*. */
    uint16_t idle_timeout;
    uint16_t hard_timeout;
    uint16_t priority;
    uint32_t buffer_id;
    uint32_t out_port;
    uint32_t out_group;
    uint16_t flags;
    uint16_t importance;
    struct ofp_match match;
}; OFP_ASSERT(sizeof(struct ofp_flow_mod) == 56);
```

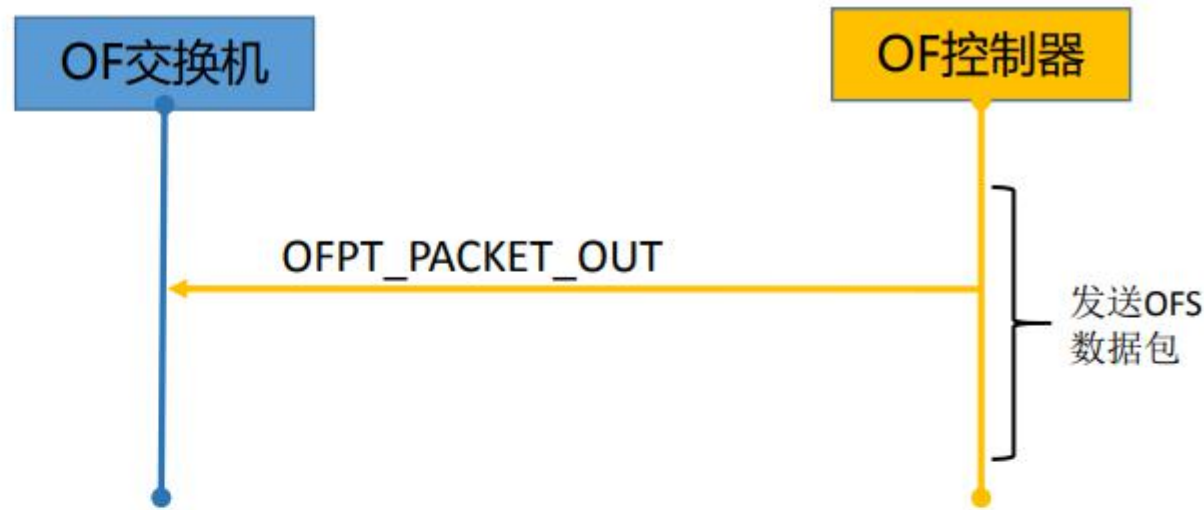
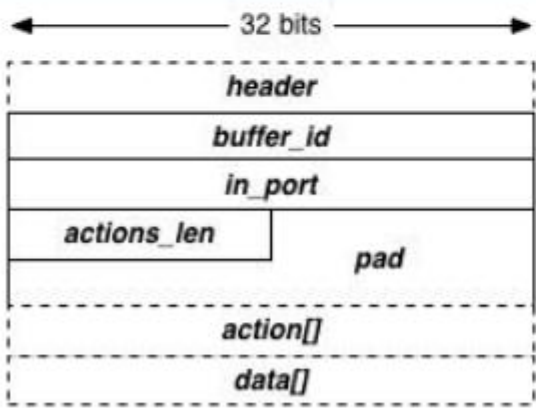
```
enum ofp_flow_mod_command {
    OFPFC_ADD = 0, /* New flow. */
    OFPFC_MODIFY = 1, /* Modify all matching flows. */
    OFPFC_MODIFY_STRICT = 2, /* Modify entry strictly matching wildcards
                             and priority. */
    OFPFC_DELETE = 3, /* Delete all matching flows. */
    OFPFC_DELETE_STRICT = 4, /* Delete entry strictly matching wildcards
                             and priority. */
};
```



Packet-out 消息

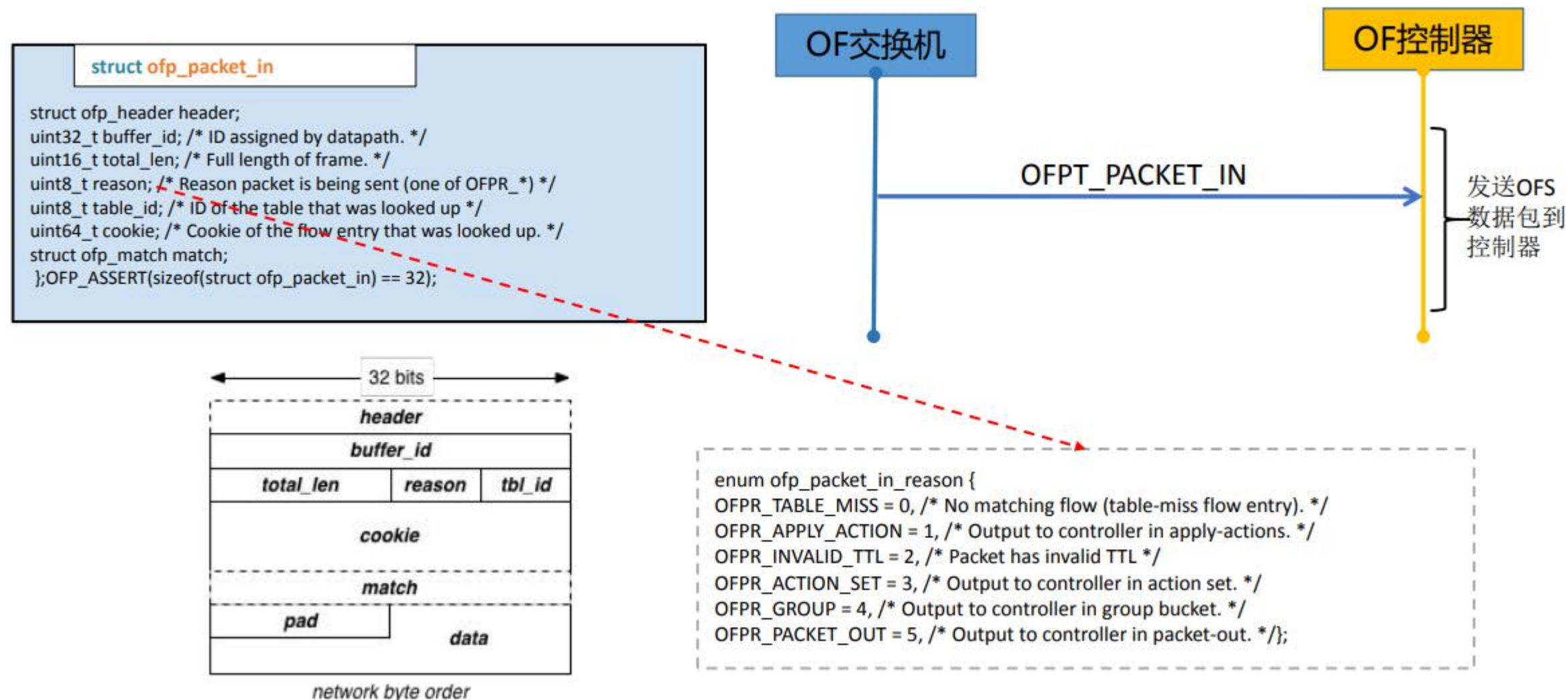
struct ofp_packet_out

```
struct ofp_header header;
uint32_t buffer_id; /* ID assigned by datapath (OFP_NO_BUFFER if none). */
uint16_t actions_len; /* Size of action array in bytes. */
uint8_t pad[2]; /* Align to 64 bits. */
struct ofp_match match; /* Packet pipeline fields. Variable size. */
/* The variable size and padded match is followed by the list of actions. */
/* struct ofp_action_header actions[0]; */ /* Action list - 0 or more. */
/* The variable size action list is optionally followed by packet data.
 * This data is only present and meaningful if buffer_id == -1. */
/* uint8_t data[0]; */ /* Packet data. The length is inferred
from the length field in the header. */
};OFP_ASSERT(sizeof(struct ofp_packet_out) == 24);
```

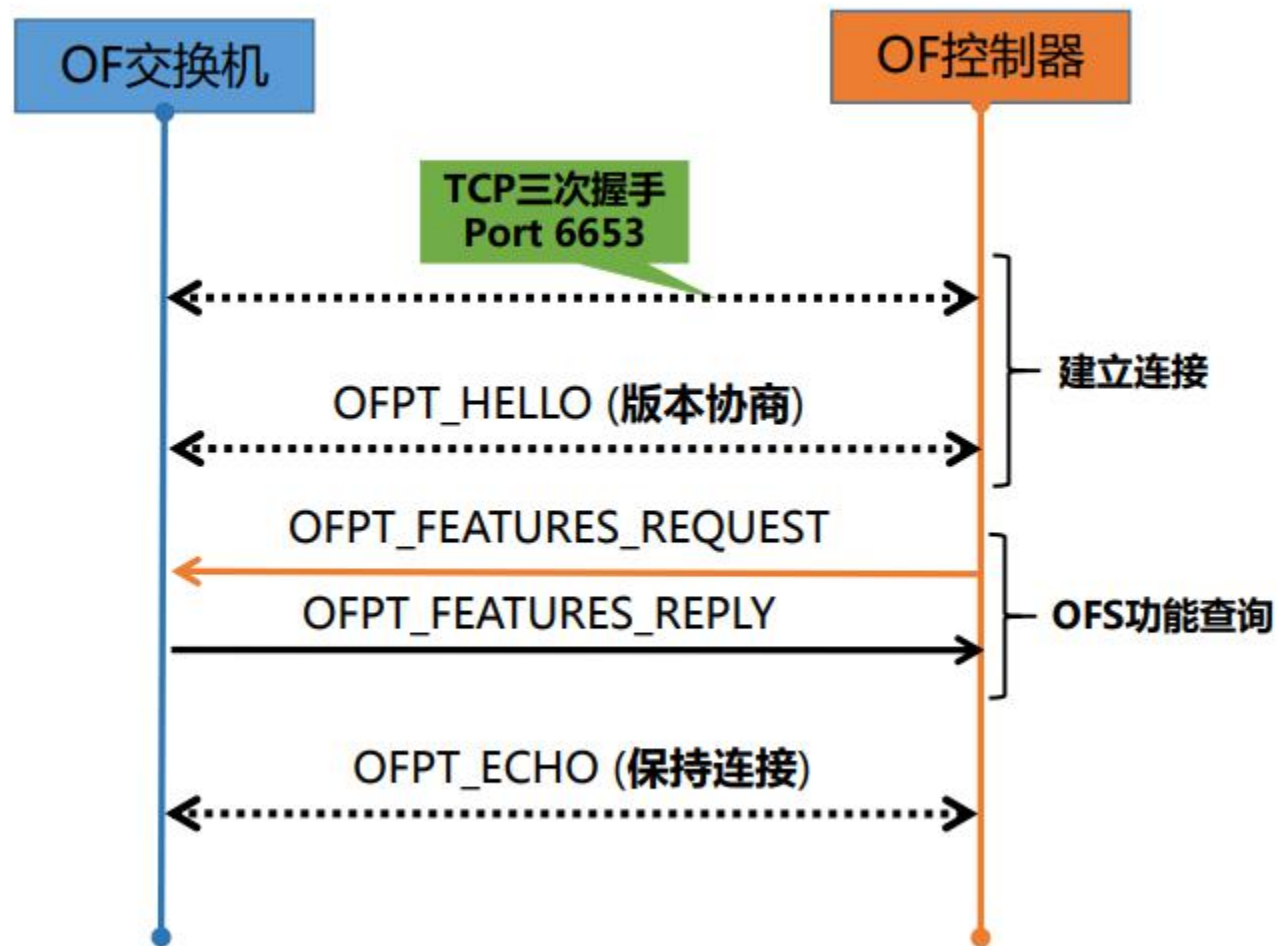
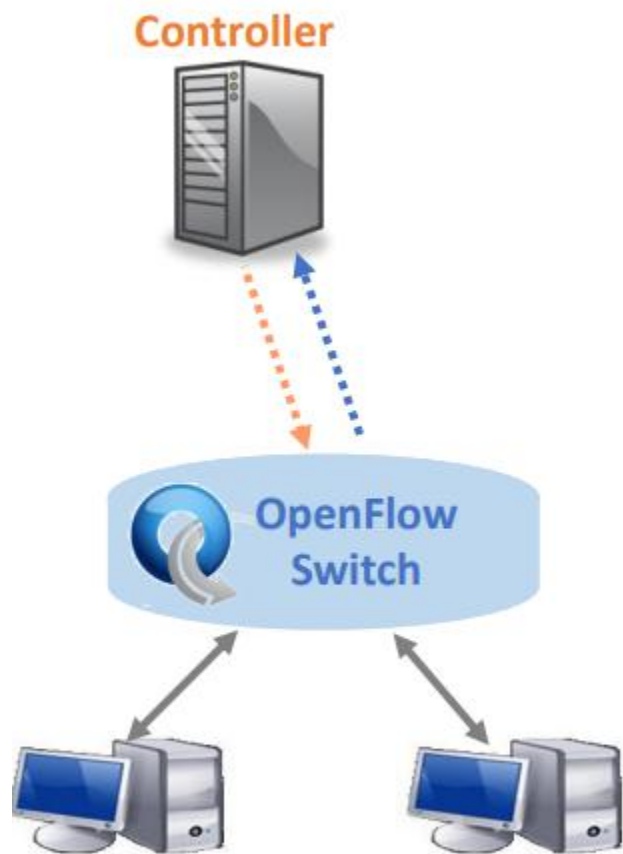


Name	Bits	Byte Ordering	Constraints
buffer_id	32	MSBF	none
in_port	32	MSBF	in 0..0xffffffff00
actions_len	16	MSBF	none
padding	48	-	none
action[]	32	MSBF	see action
data[]	-	-	none

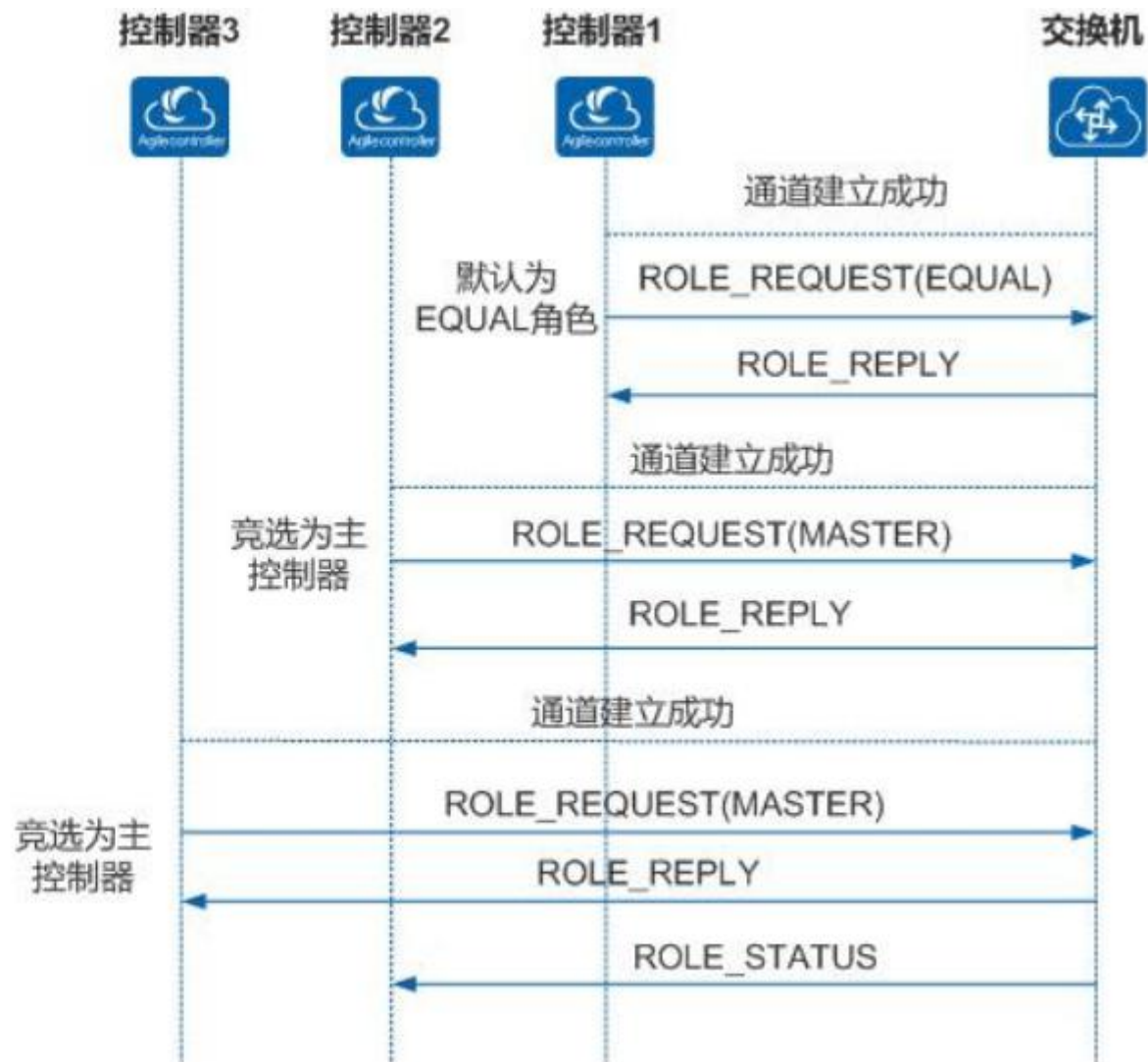
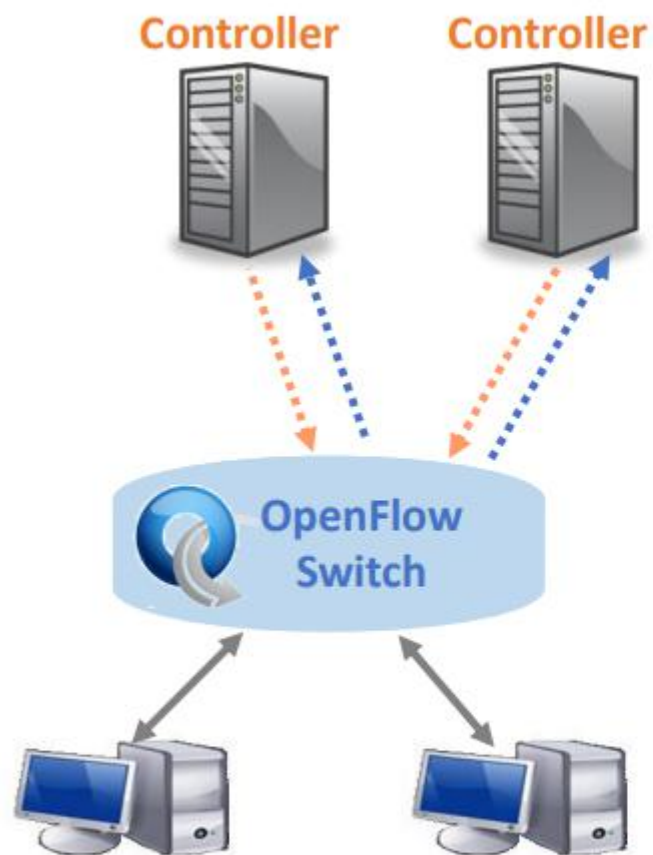
Packet-in 消息



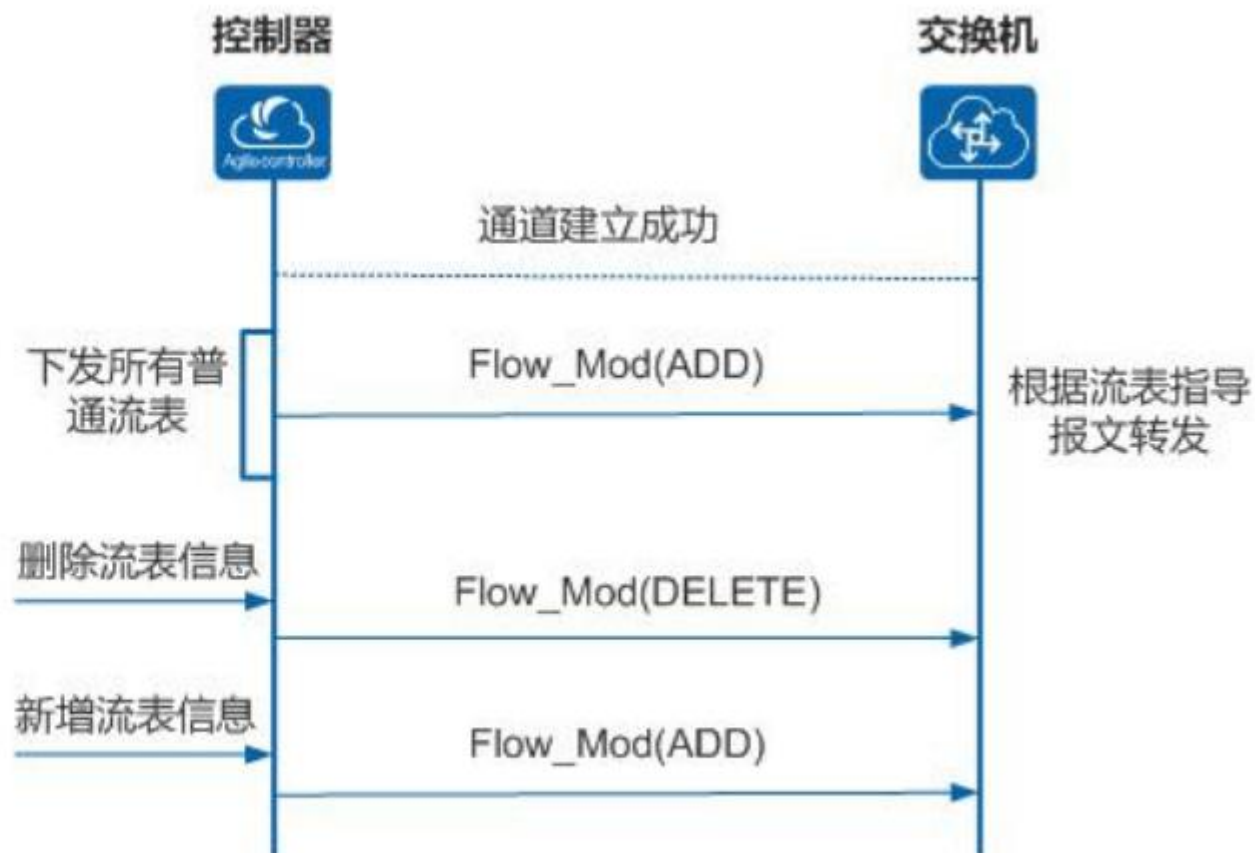
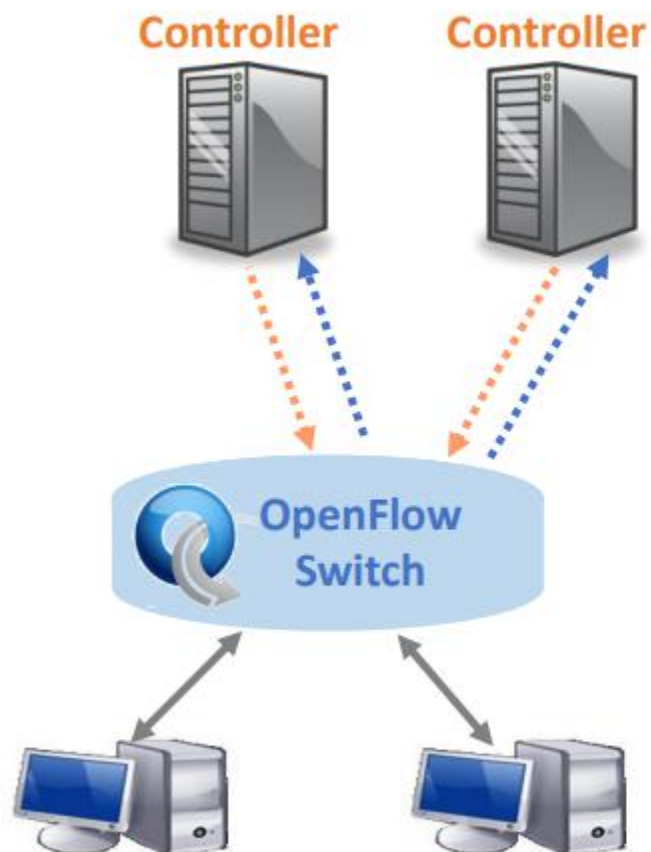
OF通道建立与维护



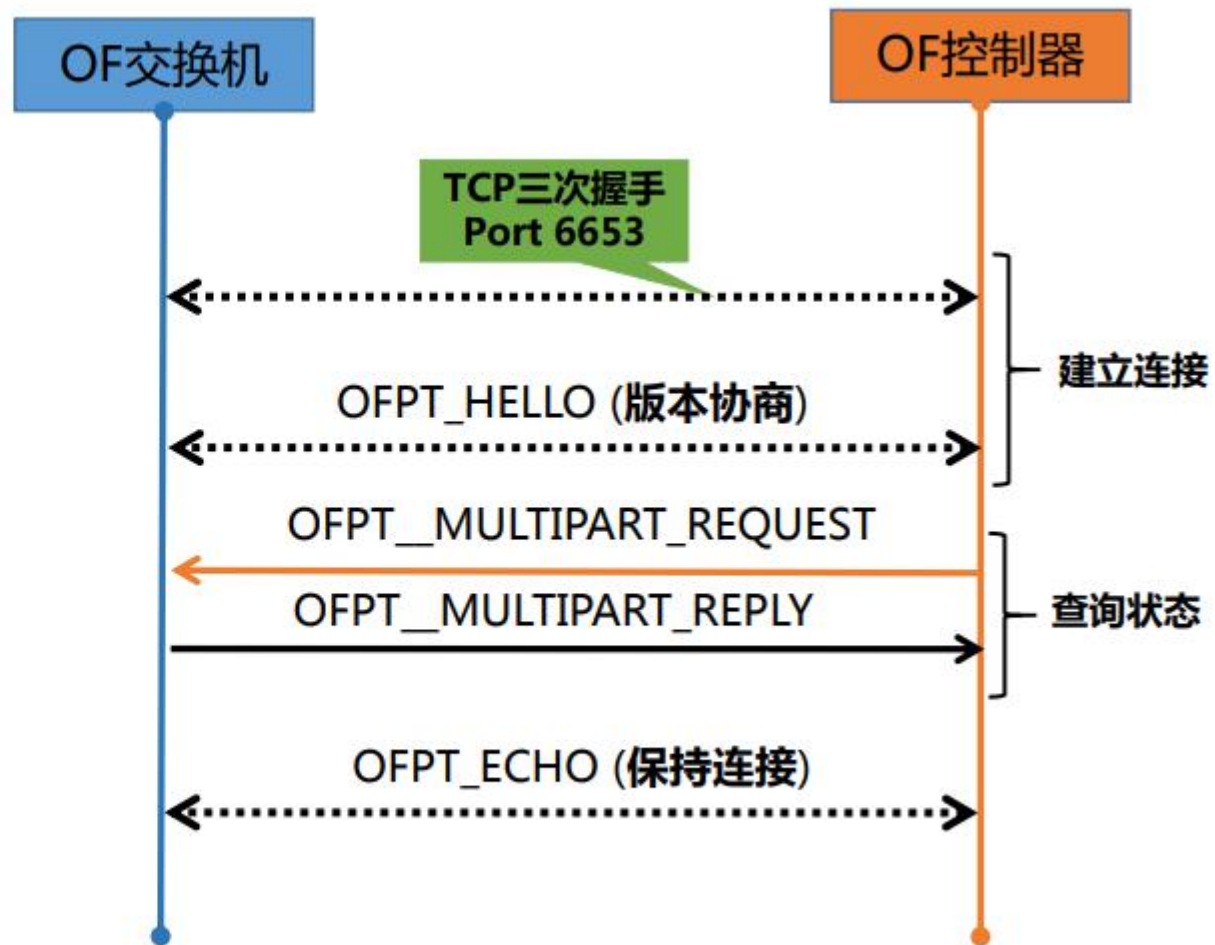
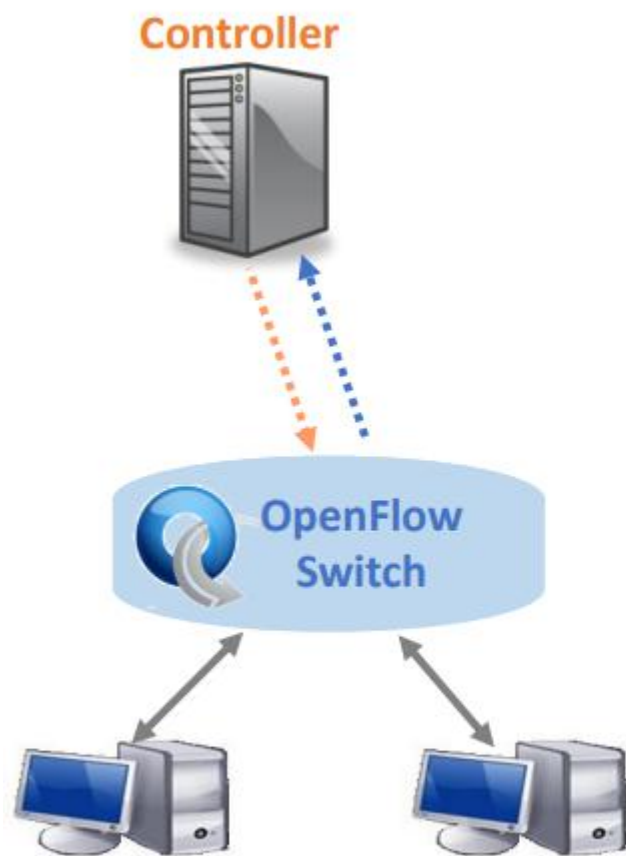
控制器角色通知



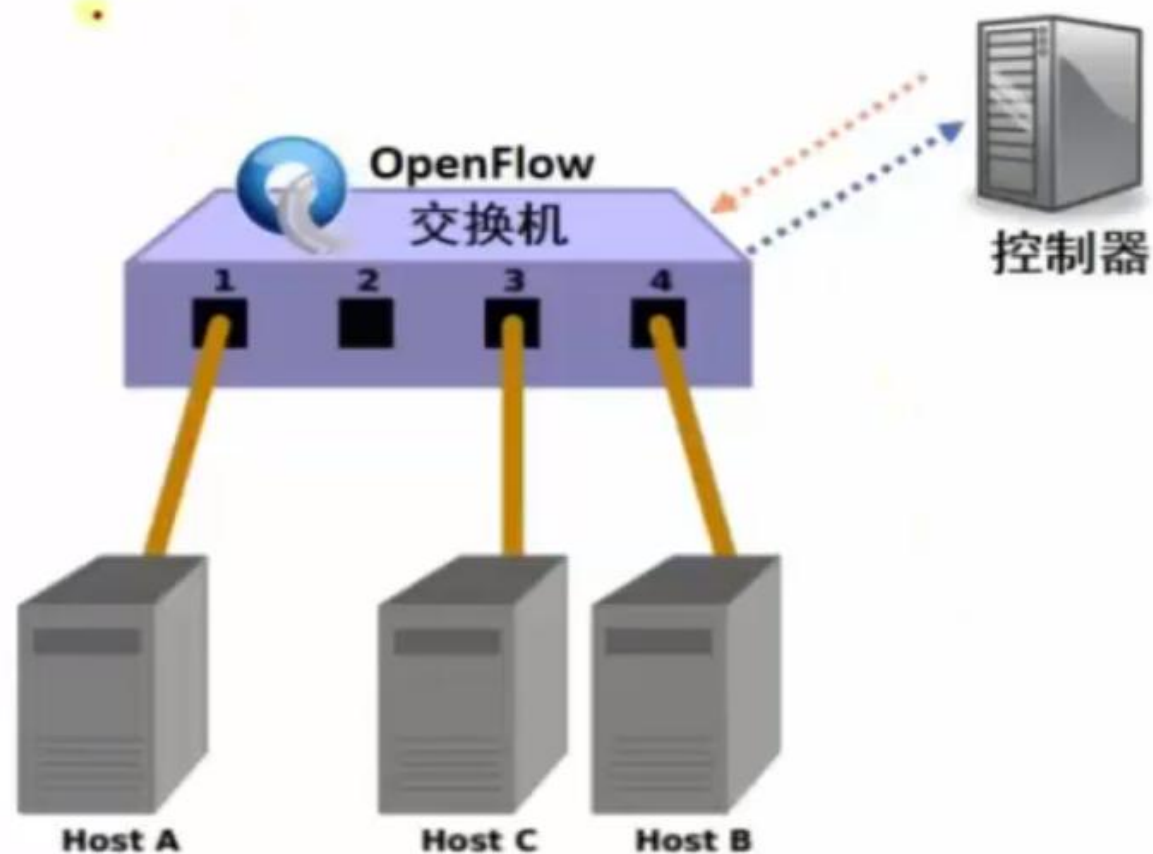
流表项下发



状态查询与收集



简单学习型交换机功能开发

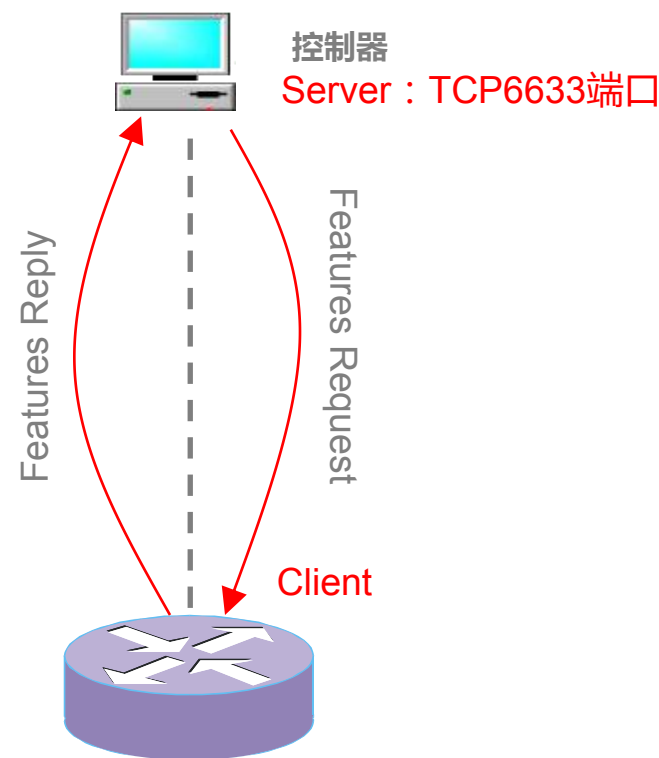


OpenFlow交换机 + 控制器 + 交换机应用

如何在基于OpenFlow的SDN架构下实现？

OpenFlow1.0—主要协议交互过程之获取交换机特性

- ◆ OpenFlow连接建立后，控制器最需要获得交换机的特性信息，交换机的特性信息包括交换机的ID(DPID)，交换机缓冲区数量，交换机端口及端口属性等等
 - ◆ 控制器向交换机发送Features Request消息查询交换机特性，Features Request消息只包含OpenFlow Header。
 - ◆ 交换机在收到Features Request消息后返回Features Reply消息，Features Reply消息包括OpenFlow Header 和Features Reply Message



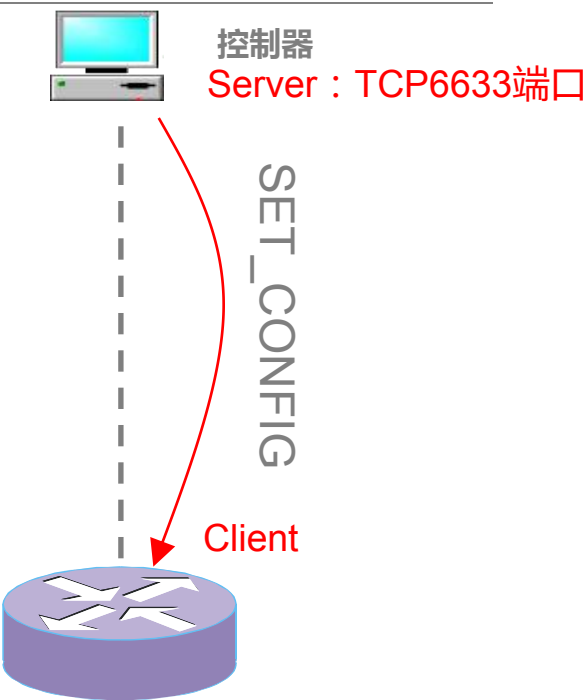
(实验演示)

OpenFlow1.0—主要交互过程之配置交换机

◆ SET_CONFIG消息结构

```
struct ofp_switch_config {
    struct ofp_header header;
    uint16_t flags;          /* OFPC_* flags. */
    uint16_t miss_send_len; /* Max bytes of new flow that datapath should
                             send to the controller. */
};

enum ofp_config_flags {
    /* Handling of IP fragments. */
    OFPC_FRAG_NORMAL    = 0, /* No special handling for fragments. */
    OFPC_FRAG_DROP      = 1, /* Drop fragments. */
    OFPC_FRAG_REASM     = 2, /* Reassemble (only if OFPC_IP_REASM set). */
    OFPC_FRAG_MASK      = 3
};
```



- ◆ 第一个属性为flags，用来指示交换机如何处理IP分片数据包，正常，丢弃，重组
- ◆ 第二个属性为miss_send_len，用来指示当一个交换机无法处理的数据包到达时，将数据包的发给控制器的最大字节数。

标志	数值	内容
OFPC_FRAG_NORMAL	0	不对碎片进行处理，依据流表项进行处理
OFPC_FRAG_DROP	1	丢弃IP碎片
OFPC_FRAG_REASM	2	对IP碎片进行重组
OFPC_FRAG_MASK	3	(可能是隐藏了flags中的ip碎片处理部分的项目)

◆ GET_CONFIG消息和响应

OpenFlow1.0— 主要协议交互之Packet-in事件

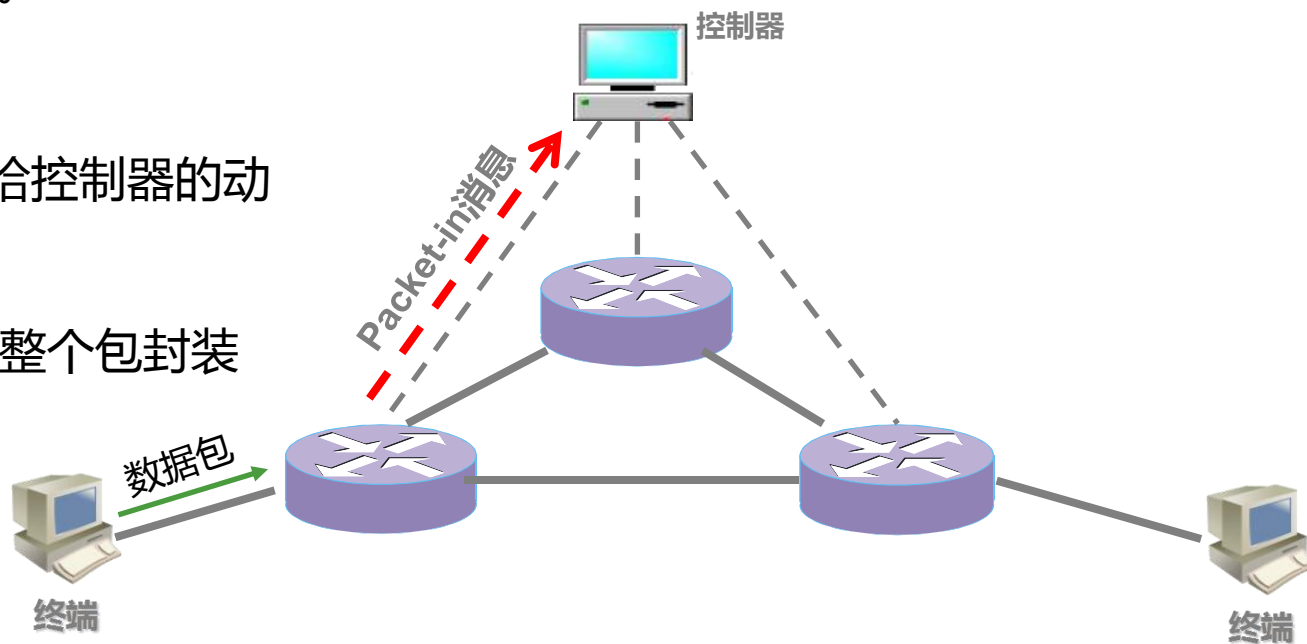
◆ Packet-in消息触发情况1：

◆ 流表中没有相应的匹配流表项

- 1.交换机会根据SET_CONFIG消息中miss_send_len设置的值为最大的数据包数据封装 packet-in 发送至OpenFlow控制器，
- 2.同时数据包缓存到交换机中等待处理。

◆ Packet-in消息触发情况2：

- ◆ 交换机流表所指示的action列表中包含转发给控制器的动作（Output=CONTROLLER）。
- ◆ 此时数据包不会被缓存在交换机中，而是将整个包封装 packet-in 消息发送给控制器处理。



OpenFlow1.0—主要协议交互过程之 Flow-Mod消息

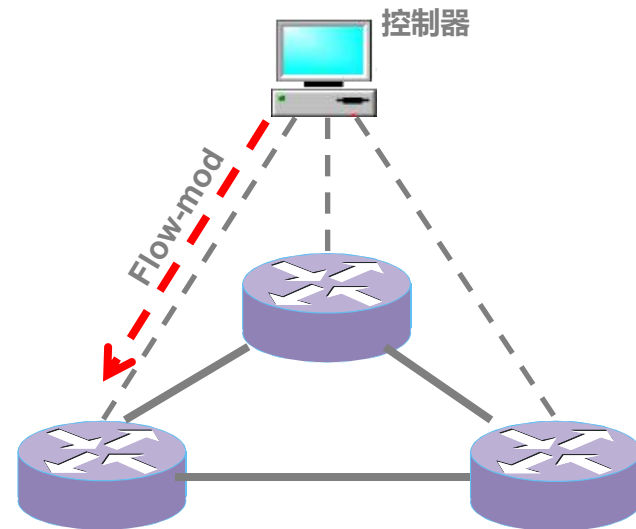
◆ Flow-Mod消息（重要）

- ◆ 控制器通过对OpenFlow交换机进行流表项的设置的消息

◆ Flow-Mod消息类型

- ◆ 通过Flow-Mod消息，可对流表项进行添加、删除、变更设置等操作

名称	说明
ADD	增加一个新的流表项
MODIFY	修改所有匹配的流表项
MODIFY_STRICT	修改严格匹配的流表项
DELETE	删除所有匹配流表项
DELETE_STRICT	删除严格匹配的流表项



OpenFlow1.0—主要协议交互过程之 Flow-Mod消息

- ◆ 流表项移除
 - ◆ 定时器计时结束
 - ◆ Idle_timeout(单位s)：计算的是没有流匹配发生的最大空闲时间
 - ◆ Hard_timeout(单位s)：计算的是流表项在流表中的总的生存时间
 - ◆ 一旦到达时间期限，交换机将自动删除该流表项，同时发送一条流删除消息
 - ◆ 控制器主动删除流表项：控制器通过下发DELETE_STRICT、DELETE等指令相关的协议消息主动删除流表项

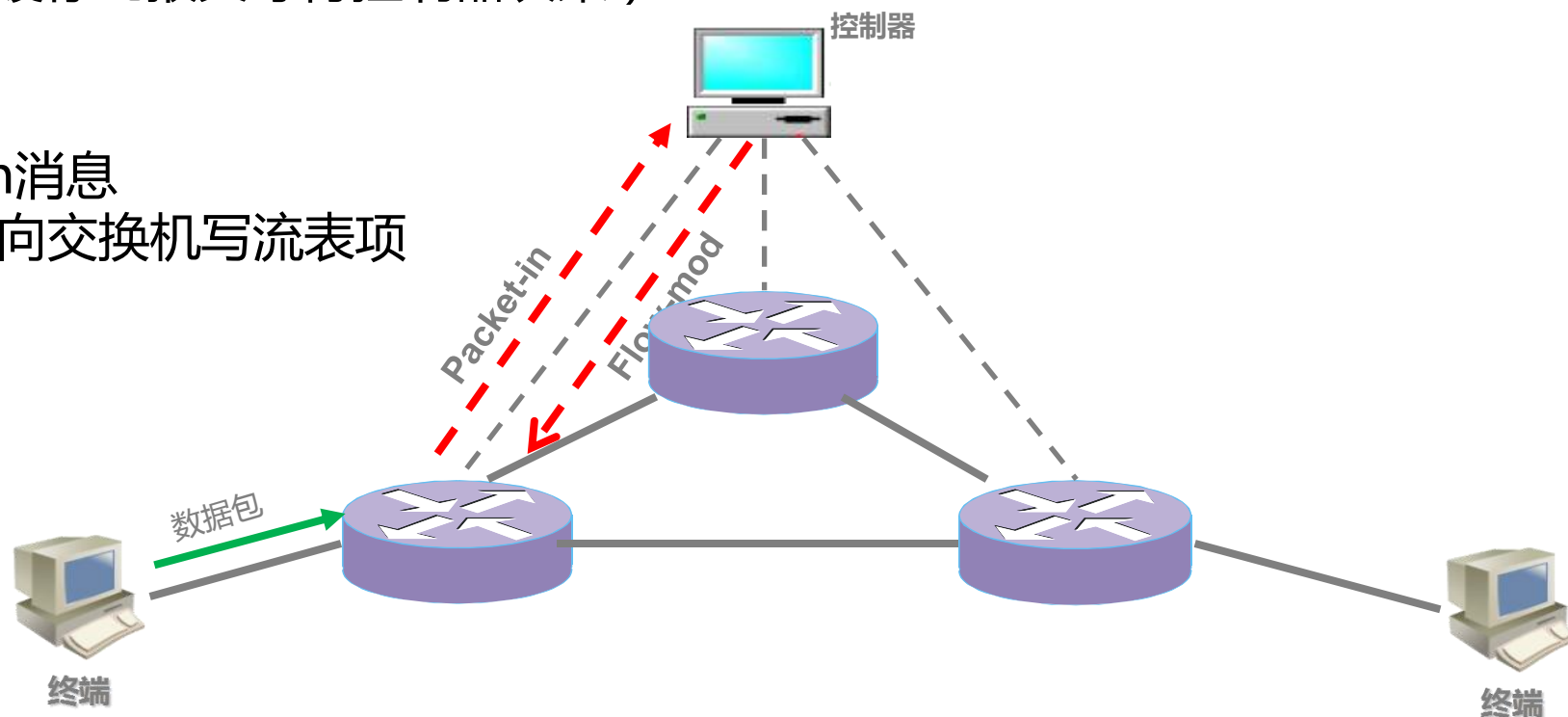
OpenFlow1.0 — Flow-Mod消息响应Packet-in

◆ OpenFlow交换机

- ◆ 接收报文
- ◆ 查找匹配流表项——没有流表项匹配
- ◆ packet-in报文根据SET_CONFIG消息中设置miss_send_len长度大小封装数据发送给控制器处理（交换机中缓存此报文等待控制器决策）

◆ 控制器

- ◆ 控制器收到Packet-in消息
- ◆ 发送flow-mod消息向交换机写流表项



从而控制器向交换机写入了一条与数据包相关的流表项，并且指定该数据包按照此流表项的action列表处理

OpenFlow1.0 — Packet-Out消息

◆Packet-Out消息

- ◆Packet-Out消息是控制器向OpenFlow交换机发送的包含**数据包命令**的消息
 - ◆替代Flow-Mod消息（网络中数量很少的ARP、IGMP等报文，没必要下发流表项去处理）
 - ◆将OpenFlow控制器创建的数据包发送至OpenFlow交换机。 **buffer_id=-1**

◆ Packet-Out消息结构

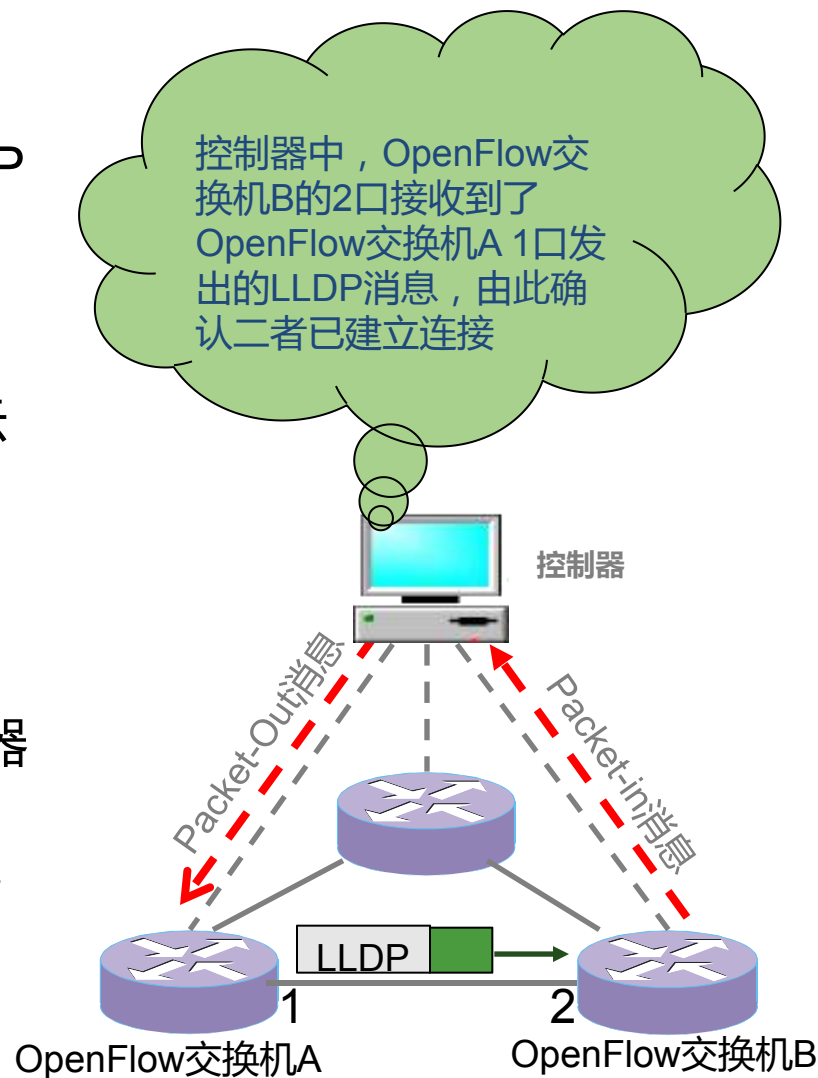
```
struct ofp_packet_out {
    struct ofp_header header;
    uint32_t buffer_id;           /* ID assigned by datapath (-1 if none). */
    uint16_t in_port;            /* Packet's input port (OFPP_NONE if none). */
    uint16_t actions_len;        /* Size of action array in bytes. */
    struct ofp_action_header actions[0]; /* Actions. */
    /* uint8_t data[0]; */        /* Packet data. The length is inferred
                                   from the length field in the header.
                                   (Only meaningful if buffer_id == -1.) */
};
```

字段	比特数	说明
buffer_id	32	数据包在交换机中的缓存区ID，当buffer_id为-1时，在packet-out消息的data段添加数据包
in_port	16	为packet-out消息提供额外的匹配信息，当packet-out的buffer_id为-1，且action列表中指定了output=TABLE的动作，in_port将作为data段数据包的额外匹配信息进行流表查询
Actions_len	16	指定action列表的长度，用来区分actions和data段
data	任意大小	为一个缓存区，可以存储一个以太网帧(buffer_id=-1)

OpenFlow1.0—Packet-Out消息的应用场景

◆ 典型应用：链路发现

- ◆ 控制器向一个交换机A发送Packet-Out消息（携带LLDP指示），buffer_id=-1，actions为从交换机A的1端口转发
- ◆ 交换机A接收到packet-out消息后，根据actions的指示从1端口发送包含在packet-out消息中的LLDP帧
- ◆ 发出这个数据包的端口连接一个OpenFlow交换机B，对端的交换机B会产生一个（添加了此LLDP帧的）Packet-In消息，然后将这个特殊的数据包上交给控制器
- ◆ 控制器根据OpenFlow交换机发送的Packet-in消息中包含的消息，构建拓扑视图。



OpenFlow1.0—协议消息类型



controller-to-switch

由控制器发起，用来管理或获取OpenFlow交换机的状态



asynchronous (异步)

由OpenFlow交换机发起，用来将网络事件或交换机状态变化更新到控制器



symmetric (对称)

由交换机或控制器发起，对端收到消息之后必须回应。

OpenFlow1.0—controller-to-switch消息列表

名称	说明	备注
Features	在建立TIS会话时，控制器发送features请求消息给交换机，交换机需要应答自身支持的功能	➤ 由控制器发起，对OpenFlow交换机进行状态查询和修改配置等操作 ➤ OpenFlow交换机接收并处理可能发送或不需要发送的应答消息
Configuration	控制器设置或查询交换机上的配置参数,交换机仅需要应答查询消息	
Modify-state	控制器管理交换机流表项和端口状态等	
Read-state	控制器向交换机请求诸如流表、端口、各个流表项等方面的统计信息	
Packet-Out	控制器通过交换机指定端口发出数据包	
Barrier	控制器通过barrier请求及相应报文，确认相关消息已经被满足或收到完成操作的通知	

OpenFlow1.0—asynchronous消息列表

名称	说明	备注
Packet-in	交换机收到一个数据包，在流表中没有匹配项，或者在流表中规定的行为是“发送到控制器”，则发送Packet-in消息给控制器；如果交换机缓存足够多，数据包被临时放在缓存中，数据包的部分内容（默认128字节）和在交换机缓存中的序号也一同发给控制器；如果交换机缓存不足以存储数据包，则将整个数据包作为消息的附带内容发给控制器	➤ 由OpenFlow交换机主动发起，用来通知交换机上发生的某些异步事件，消息是单向的，不需要控制器应答 ➤ 主要用于交换机向控制器通知收到报文、状态变化及出席错误等事件信息
Flow-removed	OpenFlow交换机中的流表项因为超时或收到修改/删除命令等原因被删除掉，会触发Flow-removed消息	
Port-status	OpenFlow交换机端口状态发生变化时，触发Port-status消息	
Error	OpenFlow交换机通过Error消息通知控制器发生的问题	

OpenFlow1.0——symmetric消息列表

名称	说明	备注
Hello	用于在OpenFlow交换机和控制器之间发起连接建立	➤ 不必通过请求建立，控制器和交换机都可以主动发起，并需要接受方应答 ➤ 消息为双向对称，主要用于建立连接、检测对方是否在线等
Echo	交换机和控制器均可以向对方发出Echo消息，接收者则需要回复Echo reply。该消息用来协商延迟、带宽、是否连接保持等控制器到OpenFlow交换机之间隧道的连接参数	
Vendor	用于OpenFlow交换机协商厂家自定义的附加功能，为未来版本预留	



- 数据包匹配+数据包处理
- 下一个问题：控制器如何写流表？

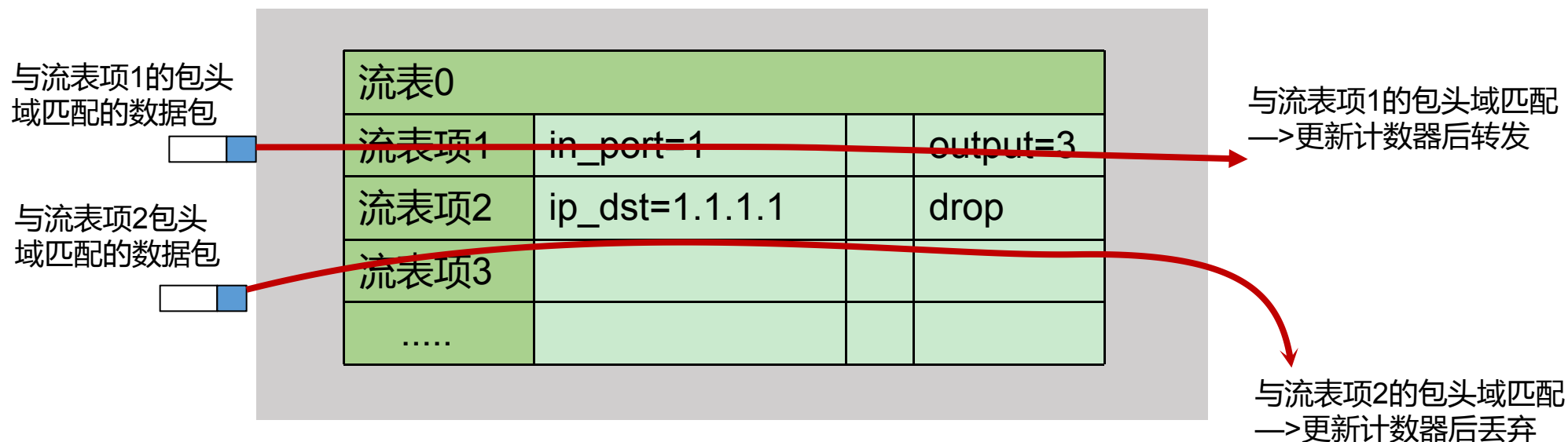
OpenFlow—OpenFlow1.0 流表

- ◆ 流表是交换机进行转发控制的核心数据
- ◆ OpenFlow交换机通过查找流表中表项来决定对进入交换机的网络流量采取何种行动

控制器下发命令到OpenFlow交换机

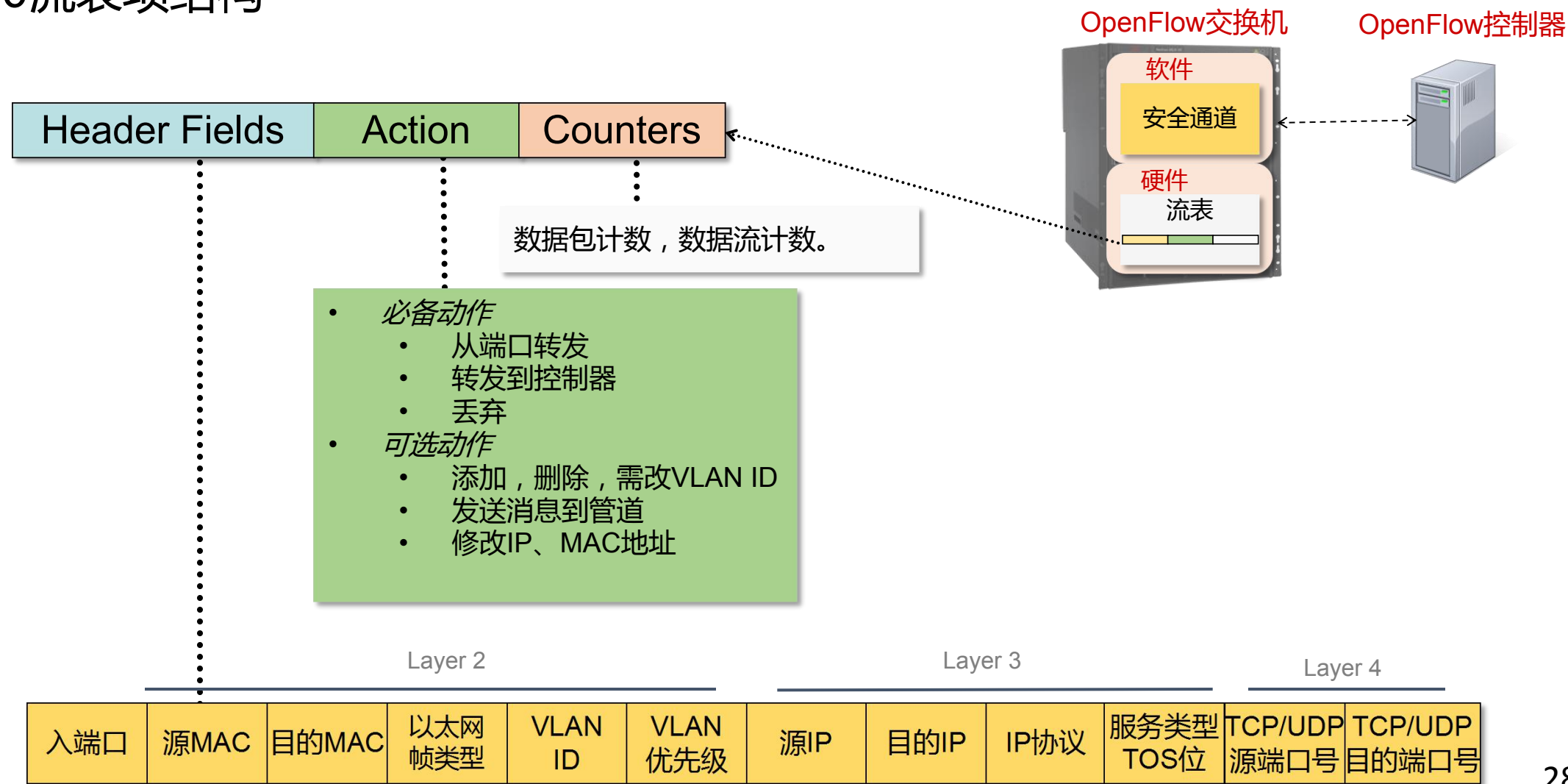
- 譬如从端口1进入的流量从端口3出
- 譬如目标IP为1.1.1.1的包进行Drop

OpenFlow交换机



南向接口协议：OpenFlow1.0流表

1.0流表项结构



OpenFlow—OpenFlow 1.0包头域

OpenFlow 1.0包头域包含12个元组 (tuple)

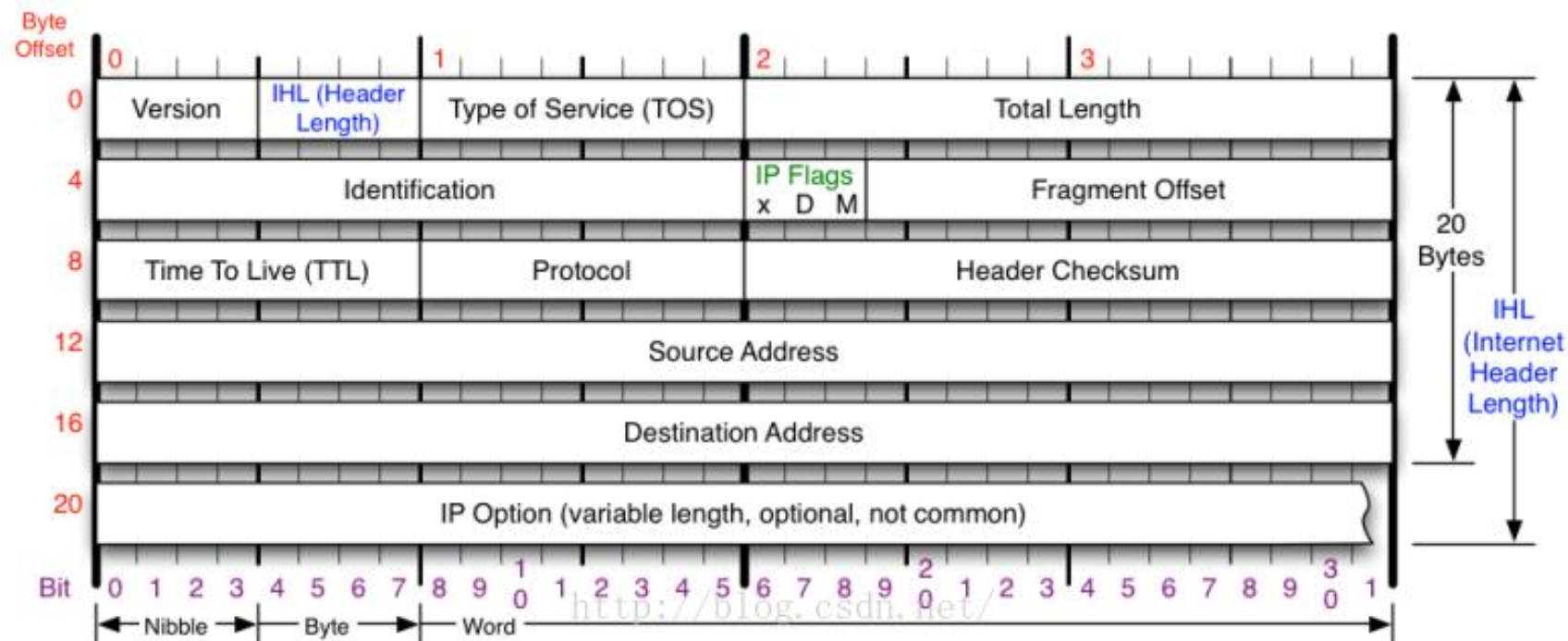
- ◆ 涵盖ISO网络模型中L1~L4层的网络配置信息

Layer1		Layer2				Layer3				Layer4	
入端口	源MAC地址	目的MAC地址	以太网类型	VLAN ID	VLAN 优先级	源IP地址	目的IP地址	IP协议	IP 服务类型	TCP/UDP源端口	TCP/UDP目的端口
Ingress Port	Ether Source	Ehter Des	Ether Type	VLAN ID	VLAN Priority	IP Src	IP Dst	IP Proto	IP TOS bits	TCP/UDP Src Port	TCP/UDP Dst Port

Header Fields	Action	Counters
---------------	--------	----------

每一个元组中的值可以是一个确定的值或者是 “ANY” 以支持对任意值的匹配

- ◆ 譬如从端口1进入的流量出端口3出
- ◆ 譬如源MAC地址2C:53:4A:02:1B:5B，目的MAC地址2C:53:4A:02:1E:4B
- ◆ 譬如目标IP为11.11.11.11的包进行Drop
- ◆ 譬如匹配目的IP为192.168.5.7且目的端口为5001的TCP流,转发到端口6



Version

Version of IP Protocol. 4 and 6 are valid. This diagram represents version 4 structure only.

Header Length

Number of 32-bit words in TCP header, minimum value of 5. Multiply by 4 to get byte count.

Protocol

IP Protocol ID. Including (but not limited to):

1 ICMP	17 UDP	57 SKIP
2 IGMP	47 GRE	88 EIGRP
6 TCP	50 ESP	89 OSPF
9 IGRP	51 AH	115 L2TP

Total Length

Total length of IP datagram, or IP fragment if fragmented. Measured in Bytes.

Fragment Offset

Fragment offset from start of IP datagram. Measured in 8 byte (2 words, 64 bits) increments. If IP datagram is fragmented, fragment size (Total Length) must be a multiple of 8 bytes.

Header Checksum

Checksum of entire IP header

IP Flags

x D M

x 0x80 reserved (evil bit)
D 0x40 Do Not Fragment
M 0x20 More Fragments follow

RFC 791

Please refer to RFC 791 for the complete Internet Protocol (IP) Specification.

OpenFlow—OpenFlow1.0 计数器

- ◆ 针对交换机中的每张流表、每个流、每个设备端口、每个转发队列进行维护，统计数据流量的相关信息
 - ◆ 针对每张流表：
统计当前活动的表项数、数据包查询次数、数据包匹配次数等
 - ◆ 针对每个数据流：
统计接收到的数据包数、字节数、数据流持续时间等
 - ◆ 针对每个设备端口：
除统计接收到的数据包数、发送数据包数、接收字节数，发送字节数等指标之外，还可以对各种错误发生的次数进行统计
 - ◆ 针对每个队列：
统计发送的数据包数和字节数，还有发送时的溢出（Overrun）错误次数等

Header Fields	Action	Counters
---------------	--------	----------

Counter	Bits
Per Table	
Active Entries	32
Packet Lookups	64
Packet Matches	64
Per Flow	
Received Packets	64
Received Bytes	64
Duration (seconds)	32
Duration (nanoseconds)	32
Per Port	
Received Packets	64
Transmitted Packets	64
Received Bytes	64
Transmitted Bytes	64
Receive Drops	64
Transmit Drops	64
Receive Errors	64
Transmit Errors	64
Receive Frame Alignment Errors	64
Receive Overrun Errors	64
Receive CRC Errors	64
Collisions	64
Per Queue	
Transmit Packets	64
Transmit Bytes	64
Transmit Overrun Errors	64

OpenFlow—OpenFlow1.0 行动 (action)

◆ 指示交换机在收到匹配的数据包后应该如何进行处理

- OpenFlow交换机缺少控制平面的能力，因此需要用action来详细说明交换机将要对数据包所做的处理
- 每个流表项可以对应零至多个action，如果没有定义转发动作，那么与流表项包头域匹配的数据包将被默认丢弃
- 同一流表项中的多个动作的执行可以具有优先级，但是在数据包的发送上并不保证其顺序
如果流表项中出现有OpenFlow交换机不支持的参数值，交换机将向控制器返回相应的出错信息

◆ 必备行动(Required Actions) vs. 可选行动(Optional Actions)

- 必备动作：需要所有OpenFlow交换机默认支持
- 可选动作：需要交换机告知控制器它所能支持的行动类型



OpenFlow—OpenFlow1.0 必备行动 (action)

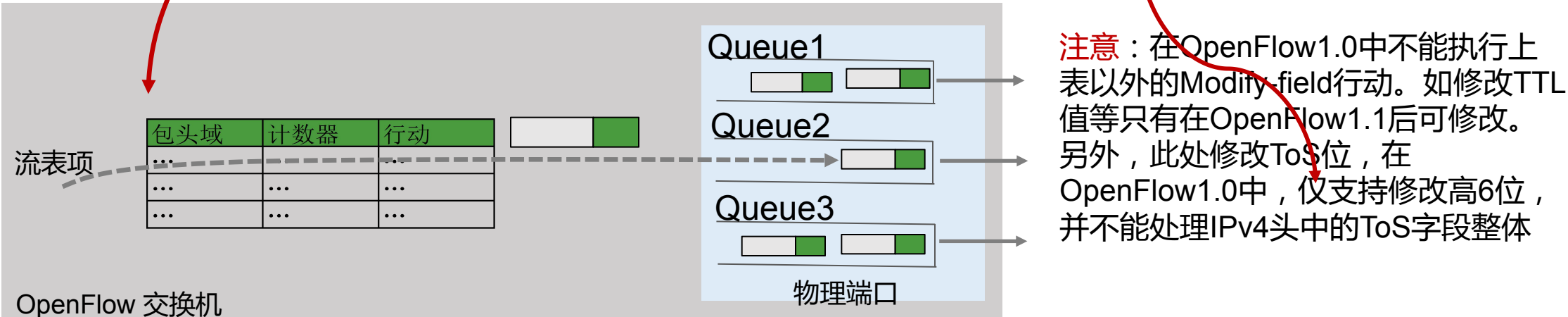
类型	名称	说明
必备行动	转发 (Forward)	交换机必须支持将数据包转发给设备的物理端口及如下的一个或多个虚拟端口 ➤ALL：转发给所有出端口，但不包括入端口 ➤CONTROLLER：OpenFlow数据包封装并转发给控制器 ➤LOCAL：转发给本地的网络栈 ➤TABLE：对packet_out消息执行流表的动作 ➤IN_PORT：从入端口发出
	丢弃 (Drop)	交换机对没有明确指明处理动作的流表项，将会对与其所匹配的所有数据包进行默认的丢弃处理

- ◆ 在OpenFlow1.0规范中，将Forward行动标记为OFPAT_OUTPUT。另外，将ALL、CONTROLLER、LOCAL、TABLE、NORMAL、IN_PORT、FLOOD作为表示输出目的地的虚拟端口，分配了16比特的数值。

虚拟端口	数值
OFPP_IN_PORT	0xfff8
OFPP_TABLE	0xfff9
OFPP_NORMAL	0xfffa
OFPP_FLOOD	0xfffb
OFPP_ALL	0xfffc
OFPP_CONTROLLER	0xfffd
OFPP_LOCAL	0xfffe
OFPP_NONE	0xffff

OpenFlow—OpenFlow1.0 可选行动 (action)

类型	名称	说明
可选动作	转发 (Forward)	交换机可选支持将数据包转发给如下的虚拟端口： ➢NORMAL：利用交换机所能支持的传统转发机制（例如二层的MAC、VLAN信息或者三层的IP信息）处理数据包 ➢FLOOD：遵照最小生成树从设备出端口洪泛发出，但不包括入端口
	排队 (Enqueue)	交换机将数据包转发到某个出端口对应的转发队列中，便于提供QOS支持
	修改域 (Modify-Field)	交换机修改数据包的包头内容，具体可以包括： ➢设置VLAN ID、VLAN优先级，剥离VLAN头 ➢修改源MAC地址、目的MAC地址 ➢修改源IPv4地址、目的IPv4地址、ToS位 ➢修改源TCP/IP端口、目的TCP/IP端口



OpenFlow—OpenFlow1.0 行动类型

◆ Action——OUTPUT类型

```
struct ofp_action_output {
    uint16_t type;           /* OFPAT_OUTPUT. */
    uint16_t len;            /* Length is 8. */
    uint16_t port;           /* Output port. */
    uint16_t max_len;        /* Max length to send to controller. */
};
```

Output类型Action的结构包含一个port参数和一个max_len参数
Port参数指定了数据包的输出端口，输出端口可以是交换机的一个实际物理端口，也可以是以下虚拟端口

ALL	将数据包从除入端口以外其他所有端口发出
CONTROLLER	将数据包发送给控制器
LOCAL	将数据包发送给交换机本地端口，(如管理)
TABLE	将数据包按照流表匹配条目处理
IN_PORT	将数据包从入端口发出
NORMAL	按照普通二层交换机流程处理数据包
FLOOD	将数据包从最小生成树使能端口转发（不包括入端口

OpenFlow—OpenFlow交换机类型

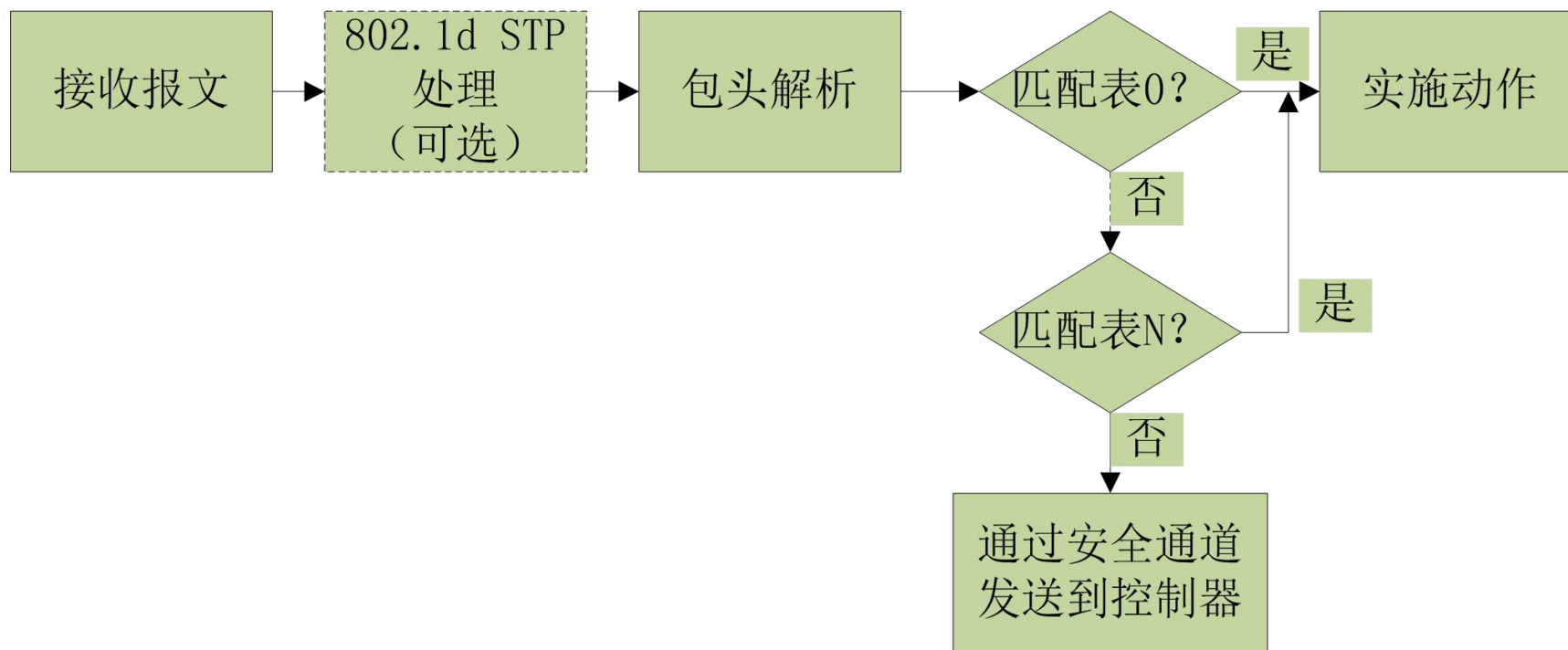
- ◆ 根据交换机的应用场景及其所能够支持的流表动作类型，OpenFlow交换机可以分为两类
 - ◆ OpenFlow专用交换机（OpenFlow-only）
 - ◆ 只支持OpenFlow协议
 - ◆ OpenFlow使能交换机（OpenFlow-enabled）

- ◆ 两类交换机对action的支持

ACTION	Of-only	Of-enable
Required Actions	YES	YES
NORMAL	NO	CAN
FLOOD	CAN	CAN

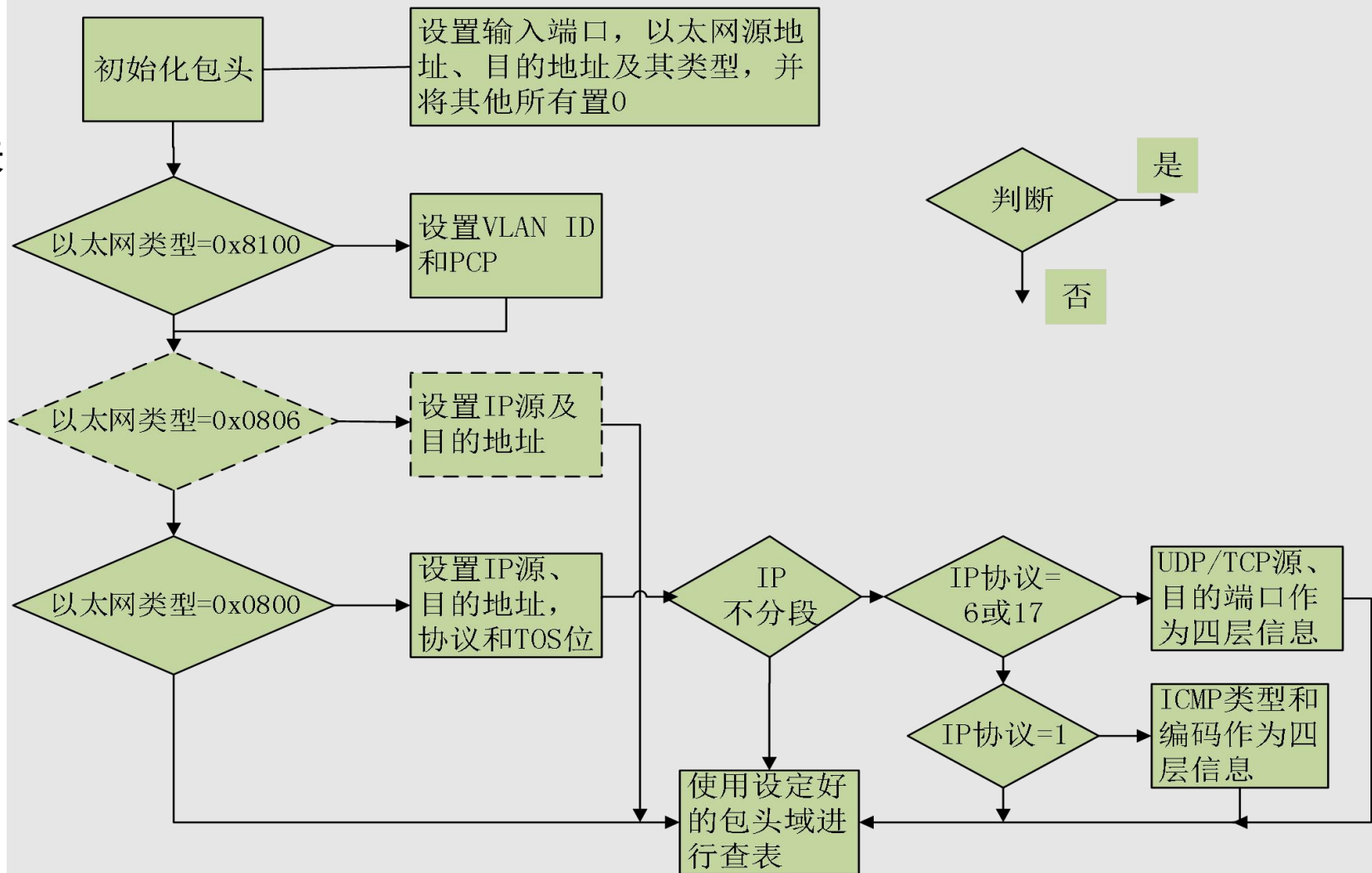
OpenFlow—OpenFlow1.0 匹配流程（实验演示）

- ◆ 进入交换机的数据包依次匹配交换机中的流表项，当数据包与多条流表项进行匹配时，优先级最高的流表项即为匹配结果。匹配成功，对应的计数器更新；如果没有匹配项，则转发给控制器



OpenFlow—OpenFlow1.0 包头域解析流程

- ◆ 0x8100:以太网自动保护开关
- ◆ 0x0806:ARP协议
- ◆ 0x0800:IP网际协议
- ◆ 6 : TCP协议
- ◆ 17 : UDP协议
- ◆ 1 : ICMP协议



入端口	源MAC	目的MAC	以太网类型	VLAN ID	VLAN 优先级	源IP	目的IP	IP协议	IP TOS 位	TCP/UDP 源端口	TCP/UDP 目的端口
-----	------	-------	-------	---------	-------------	-----	------	------	-------------	----------------	-----------------

目录

01

OpenFlow总述

02

OpenFlow1.0 规范

03

OpenFlow流表演进

04

OpenFlow1.3 规范

简书

1.0 版本：1.0版本中只有单流表

1.1版本：为了避免单流表膨胀，1.1版本采用多级流表，以流水线的形式提高数据包匹配效率。

1.2 版本：由于OpenFlow将所有的控制协议都集中到控制器中，导致控制器成为众矢之的，为了提高安全性和健壮性，1.2版本引进了多控制器的概念。

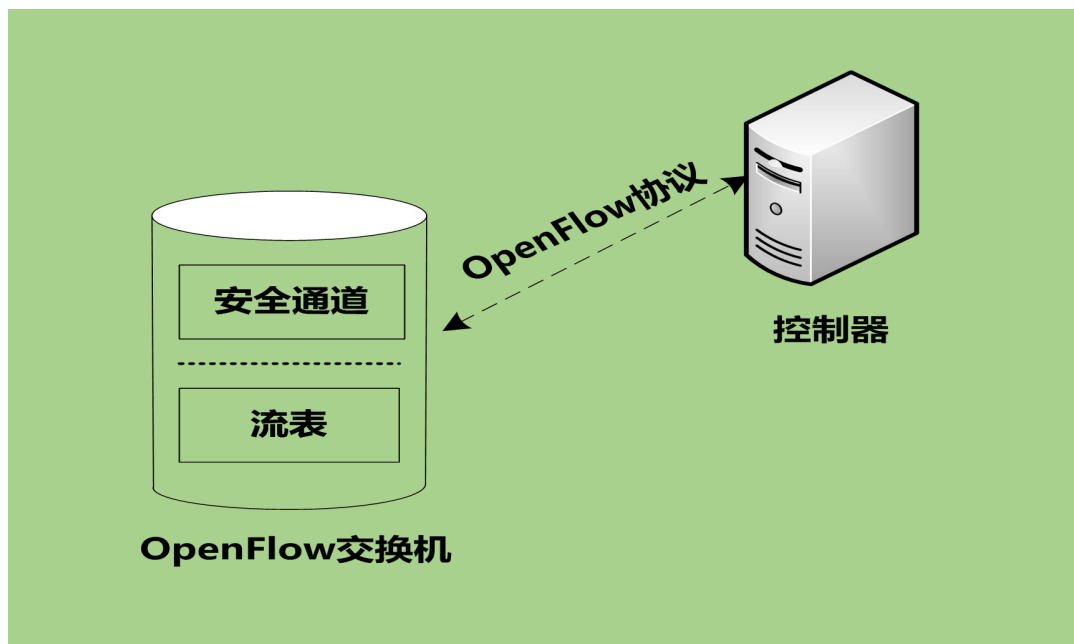
1.3 版本：增加了meter表，匹配项更加丰富。

1.5 版本：增加数据包类型识别流程，提升数据包的处理效率

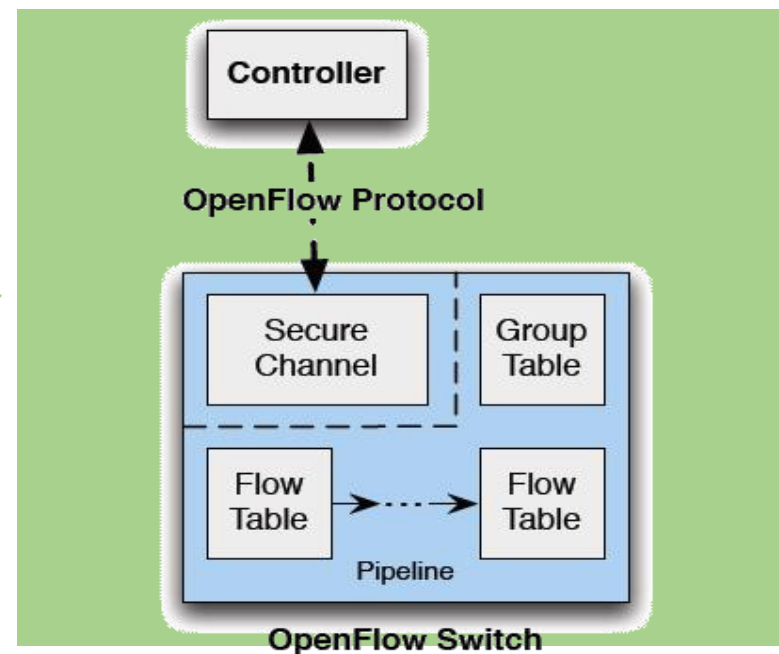
OpenFlow正在一点一点摸索一个平衡点，既能保证控制器充分的控制器主权又让交换机有一定的自主功能。或许，最初的OpenFlow十分理想化，但是在其不断演进的过程中我们可以看出OpenFlow在不断向实际应用场景靠拢。

OpenFlow—协议的演进

- ◆ 自OpenFlow 1.1开始，交换机架构发生了变化以实现性能提升
 - ◆ 流表由单一流表演变为由流水线串联而成的多流表
 - ◆ 增加了组表（Group Table）



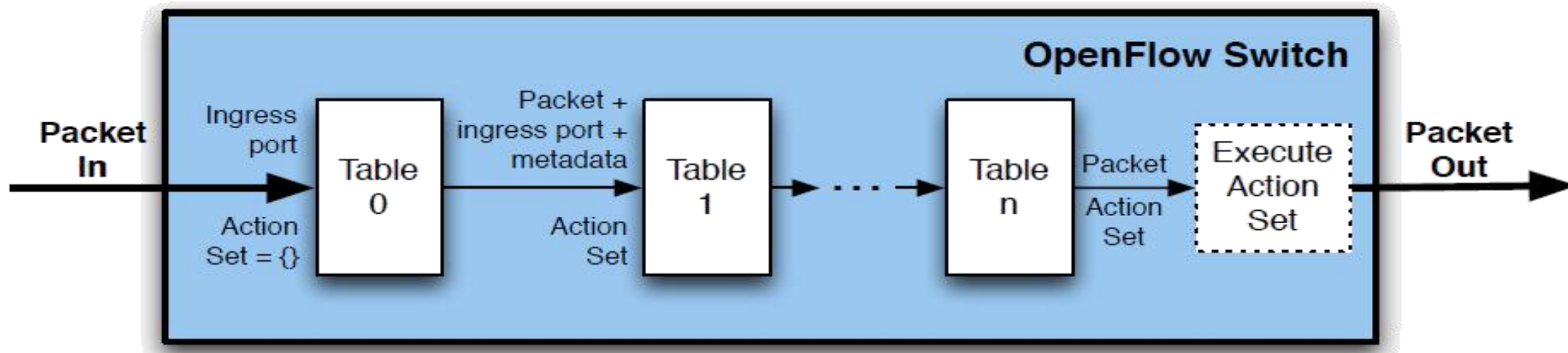
OpenFlow1.0



OpenFlow1.1及后续版本

OpenFlow-1.3版本流表匹配

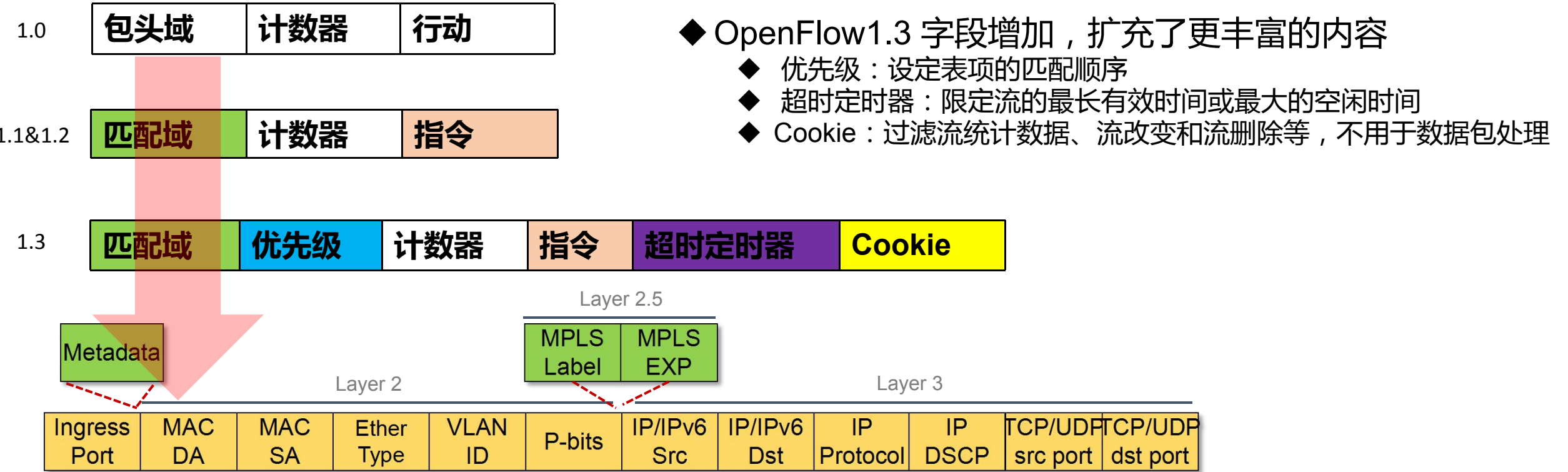
OpenFlow Pipeline:



pipeline:Goto

- OpenFlow可以存在多个流表，但必须从Table 0开始匹配；
- OpenFlow交换机可通过建立安全通道时的Feature消息确认自身所支持的流表个数；
- 数据包与任何一个流表都不匹配的情况称为Table-Miss，此时利用packet-in消息转发到控制器，或丢弃数据包；

OpenFlow—协议的演进之流表结构



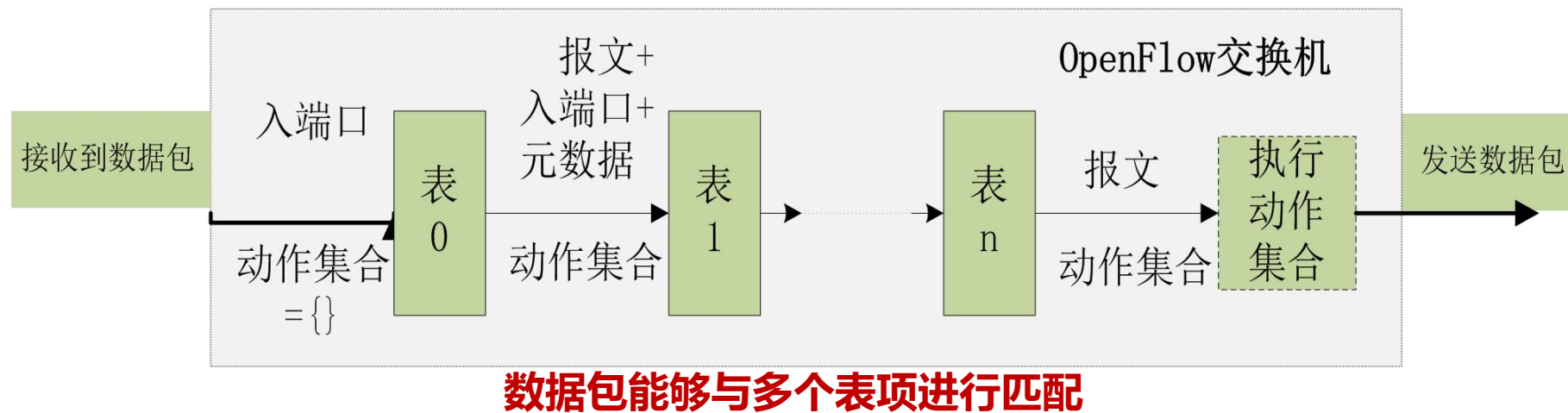
- ◆ OpenFlow1.3 字段增加，扩充了更丰富的内容
 - ◆ 优先级：设定表项的匹配顺序
 - ◆ 超时定时器：限定流的最长有效时间或最大的空闲时间
 - ◆ Cookie：过滤流统计数据、流改变和流删除等，不用于数据包处理

◆ OpenFlow1.1&1.2

- ◆ Header Fields ——> Match Fields：在匹配字段中新添加了MPLS标签，MPLS流量类型，元数据等3个字段，并添加了SCTP流控制传输协议处理。
- ◆ Actions ——> Instructions：指令是对与流表项匹配的数据包所执行的命令，提供执行行动，在之后批量执行的行动集中,添加及删除行动、写入元数据等功能，移动到已指定流表的动作也是指令，它是实现流水线处理的基础

OpenFlow—协议的演进之多级流表

- ◆ OpenFlow交换机对匹配内容长度不作区分，均以具有最大长度的表项为准（OpenFlow 1.0中，匹配字段为252位），造成TCAM成本的增加
- ◆ 为了减小流表规模，OpenFlow 1.1及后续版本引入多级流表，基于流表特征提取将匹配过程分解为多个步骤，形成流水线处理模式，降低流表的记录条数



- ◆ OpenFlow 1.3提出在多流表每个表的最都增加Table-miss项

OpenFlow—协议的演进之组表（ Group ）

◆ OpenFlow 1.1及后续版本引入组表（ Group Table ）



组表结构

字段	说明
组标识符（ 32 ）	用于表示组的识别符。依据该识别符使用各组
组类型	指定组的动作。分别为all、select、indirect、fast failover四种
计数器	记录通过该组表项处理的数据包数
行动桶（ action buckets ）	多个行动数据桶。各行动桶存储了多个执行行动和其对应的参数

- ◆ 适合于实现广播或组播，或者规定只执行某些特定的操作集
- ◆ 其中，组类型规定了是否所有的动作桶中的指令都会被执行
 - ◆ 所有（ all ）：执行所有动作桶中的动作，可用于组播或广播
 - ◆ 选择（ select ）：执行该组中的一个动作桶中的动作，可用于多路径
 - ◆ 间接（ indirect ）：执行该组的一个确定的动作桶中的动作
 - ◆ 快速故障恢复（ fast failover ）：执行第一个具有有效活动端口的动作桶中的指令

OpenFlow—协议的演进之Meter表

- ◆ OpenFlow 1.3增加了对单个数据流的计量功能，使得OpenFlow能够实现简单的QoS服务（例如流量限速），并且可以结合每个端口队列来实现更复杂的QoS框架（例如DiffServ）
- ◆ 计量表（Meter Table）：包含若干针对各个数据流的计量表项

Meter Identifier	Merter Bands	Counters
------------------	--------------	----------

Meter表结构

字段	说明
Meter Identifier (32)	32位的无符号整数，用来唯一识别该计量表项
Merter Bands	由计量带组成的无序列表，其中每个计量带都指明了其速率及处理数据包的方式
Counters	用于在报文被计量表项处理时更新相关计数

OpenFlow—协议的演进之Meter表续

- ◆ 每个计量表项可能具有有一个或多个计量带，每个计量带都指定了其所适用的速率和数据被处理的方式

Band Type	Rate	Counters	Type specific arguments
-----------	------	----------	-------------------------

Merter band结构

字段	说明
Band Type	定义了数据包怎样被处理（ drop,dscp remark ）
Rate	用于选择计量带，定义了带可以运行的最高速率
Counters	当数据报文被计量带处理时，更新计数器
Type specific arguments	带类型的可选参数

- ◆ 计量带Band Type，当前只有两种可选的计量带类型

字段	说明
Drop	通过丢弃数据包，定义带宽速率限制
Dscp remark	降低数据包的IP头中的DSCP字段丢弃的优先级，可用于定义简单的DiffServ策略

OpenFlow—协议的演进之匹配域

- ◆ 随着OpenFlow的演进，匹配域（ OpenFlow v1.0称作包头域 ）的覆盖范围越来越广，以满足更灵活的转发决策

规范版本	匹配域数量	匹配域主要变化	备注
1.0	12		
1.1	15	在OpenFlow1.0基础上增加了元数据（ Metadata ）、 MPLS 标签、 MPLS流量类型等3个匹配域 支持SCTP流控制传输协议，支持OpenFlow1.0中定义的IP协议TCP/UDP源和目的端口及ICMP源和目的端口共享	元数据不是数据包自带的，而是多流表处理时所需的附加信息，用于在同一交换机的不同流表间传递信息
1.2	36	将OpenFlow1.1定义为可复用的匹配域全部拆分（ 取消了TCP/UDP/SCTP/ICMP重载使用相同的字段） 增加对IPv6信息的匹配（ TLV结构的OXM ）	规定OpenFlow交换机必须支持13个匹配域
1.3	39	增加了PBB（ mac-in-mac ）、 tunnel-id及IPv6扩展头的匹配	OpenFlow交换机必须支持的匹配域与OpenFlow1.2同

- ◆ OpenFlow1.2/1.3中交换机必须支持输入端口、目标及发送源以太网地址、以太网类型、IP协议号、IPv4和IPv6的目标及源地址、TCP/UDP的目标及发送源端口号等13个字段

OpenFlow—协议的演进之计数器

- ◆ OpenFlow 1.1增加了组表的概念，因此计数器相应增加了针对每组、每个动作桶的相关统计
- ◆ OpenFlow 1.3增加了针对数据流的计量表，因此计数器相应增加了针对各个数据流计量表（meter）的统计

Counter	Bits	
Per Flow Table		
Reference Count (active entries)	32	<i>Required</i>
Packet Lookups	64	<i>Optional</i>
Packet Matches	64	<i>Optional</i>
Per Flow Entry		
Received Packets	64	<i>Optional</i>
Received Bytes	64	<i>Optional</i>
Duration (seconds)	32	<i>Required</i>
Duration (nanoseconds)	32	<i>Optional</i>
Per Port		
Received Packets	64	<i>Required</i>
Transmitted Packets	64	<i>Required</i>
Received Bytes	64	<i>Optional</i>
Transmitted Bytes	64	<i>Optional</i>
Receive Drops	64	<i>Optional</i>
Transmit Drops	64	<i>Optional</i>
Receive Errors	64	<i>Optional</i>
Transmit Errors	64	<i>Optional</i>
Receive Frame Alignment Errors	64	<i>Optional</i>
Receive Overrun Errors	64	<i>Optional</i>
Receive CRC Errors	64	<i>Optional</i>
Collisions	64	<i>Optional</i>
Duration (seconds)	32	<i>Required</i>
Duration (nanoseconds)	32	<i>Optional</i>

Per Queue		
Transmit Packets	64	<i>Required</i>
Transmit Bytes	64	<i>Optional</i>
Transmit Overrun Errors	64	<i>Optional</i>
Duration (seconds)	32	<i>Required</i>
Duration (nanoseconds)	32	<i>Optional</i>
Per Group		
Reference Count (flow entries)	32	<i>Optional</i>
Packet Count	64	<i>Optional</i>
Byte Count	64	<i>Optional</i>
Duration (seconds)	32	<i>Required</i>
Duration (nanoseconds)	32	<i>Optional</i>
Per Group Bucket		
Packet Count	64	<i>Optional</i>
Byte Count	64	<i>Optional</i>
Per Meter		
Flow Count	32	<i>Optional</i>
Input Packet Count	64	<i>Optional</i>
Input Byte Count	64	<i>Optional</i>
Duration (seconds)	32	<i>Required</i>
Duration (nanoseconds)	32	<i>Optional</i>
Per Meter Band		
In Band Packet Count	64	<i>Optional</i>
In Band Byte Count	64	<i>Optional</i>

OpenFlow—协议的演进之指令

◆ OpenFlow 1.0中将流表项与数据包匹配后对其进行的操作称为行动，而在OpenFlow 1.1及其后续版本中都将其改称为指令

包头域	计数器	行动
-----	-----	----

◆ OpenFlow 1.1和1.3先后增加了5条指令和1条指令

规范版本	指令类型	指令名称	说明
v1.1	可选指令	Apply-Actions	立即进行指定行动，而不改变行动集。经常在修改数据包，以及在两个表之间执行同类型的多个行动时使用
	可选指令	Clear-Actions	清除行动集中所有的行动
	可选指令	metadata/mask	在元数据区域记录元数据（由掩码指定修改哪一位元数据值）
	必备指令	Write-Actions	将指定的多个行动合并到当前的行动集中，行动集中已存在同一类型时将其覆盖，不存在同一类型时进行添加
	必备指令	Goto-Table next-table-id	指示在流水线中的下级流表，参数table-id必须大于当前流表的table-id而流水线最后的流表不能包含这个指令
v1.3	可选指令	Meter meter id	将数据包指向指定的计量表，通过计量表处理的数据大多都是被丢弃（实际处理结果根据计量表的配置和状态）

OpenFlow—协议的演进之指令

- ◆ OpenFlow指令的执行可能会导致数据包在多流表之间的转移，也可能会指示交换机对数据包采取真正的行动
- ◆ OpenFlow 1.0定义了必备的转发、丢弃，以及可选的转发、排队、修改域等动作，后续版本对其进行了完善，在OpenFlow1.3版本中定义了7种行动类型

指令名称	指令类型	说明
OutPut	必备行动	输入动作，将数据包从指定的OpenFlow端口转发，OpenFlow交换机必须支持转发到物理端口、交换机定义的逻辑端口和必须支持的保留端口
Set-Queue	可选行动	为数据包设置队列ID,已完成QoS功能在OpenFlow1.0中被称作Enqueue,OpenFlow1.1将其更名
DROP	必备行动	用于丢弃数据包
Group	必备行动	用指定组处理数据包，而具体的操作要根据组类型决定。OpenFlow1.1中增加
Push-Tag/Pop-Tag	可选行动	用于VLAN,MPLS Tag的入栈和出栈，在OpenFlow1.1中增加
Set-Field	可选行动	用于设置数据包头的类型和修改数据包头的值，该动作在OpenFlow 1.0中被称作Modify-field，OpenFlow1.1及后续版本将其更名为Set-Field
Change-TTL	可选行动	用于修改TTL值，在OpenFlow 1.2中增加

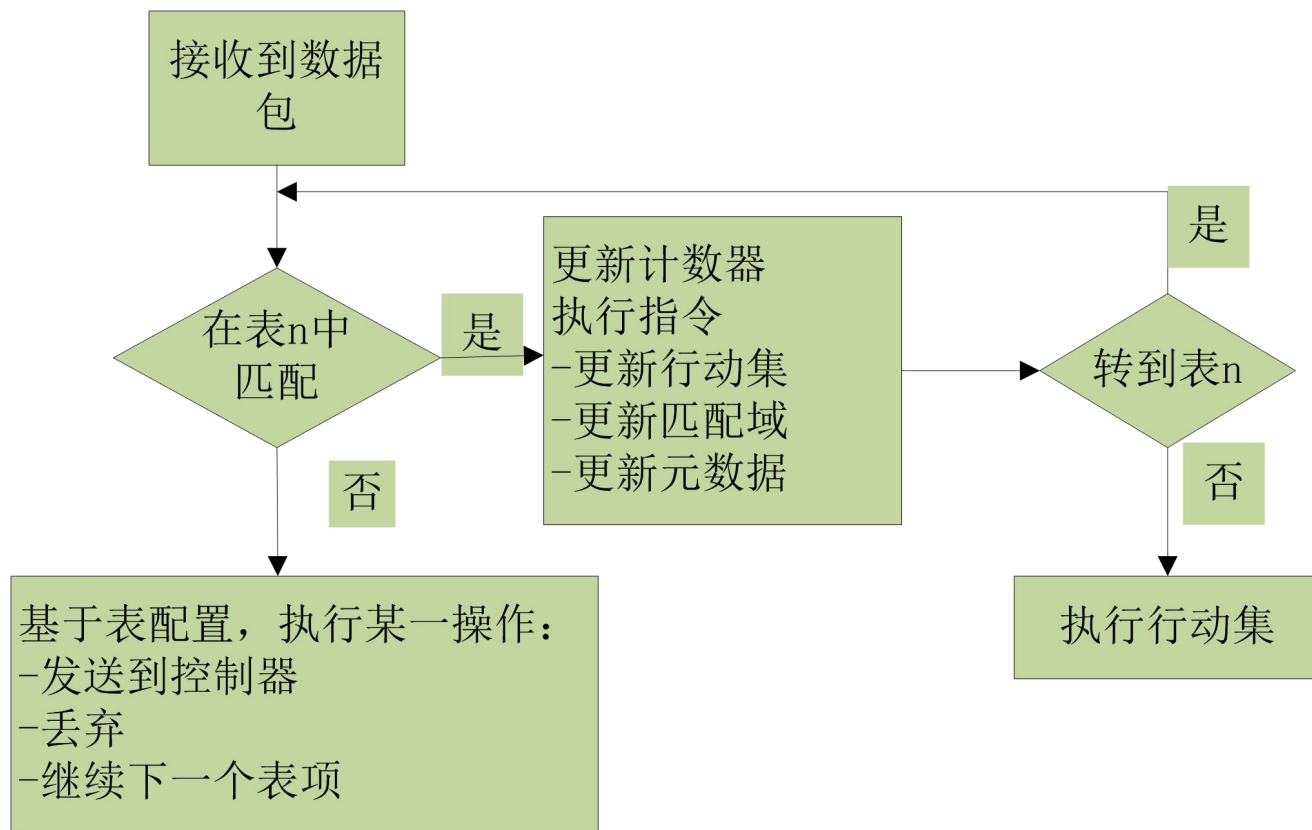
OpenFlow—协议的演进之行动集

- ◆ OpenFlow 1.1引入多级流表，提出行动集（ action set ）。action从1.0协议的1个能够增加到多个。
- ◆ 在各数据包到达OpenFlow交换机时，行动集会被初始化为空状态。在流水线处理的过程中，首先会匹配包头域，在流水线处理的最后执行所有的行动。
- ◆ 行动集中添加的行动并非按照添加顺序执行，而是按照下表所示的顺序执行

优先顺序	行动	说明
1	copy TTL inwards	将TTL复制到内侧
2	pop	应用所有的标签pop
3	push	应用所有的标签push
4	copy TTL outwards	将TTL复制到外侧
5	decrement TTL	将TTL减去1
6	Set	将set-Field行动运用到数据包中
7	qos	将queue中设置的所有QoS行动应用到数据包中
8	group	Group行动已指定时，应用相应列表中指定的适当的组行动
9	output	Group行动未指定时，将数据包转发至通过output行动指定的端口中

OpenFlow—协议的演进之流表匹配流程

- ◆ 流表的匹配在OpenFlow 1.1引入多流表后变得更加复杂，需要依次对多张流表中的流表项进行比对后，按照匹配成功与否执行相应的操作。

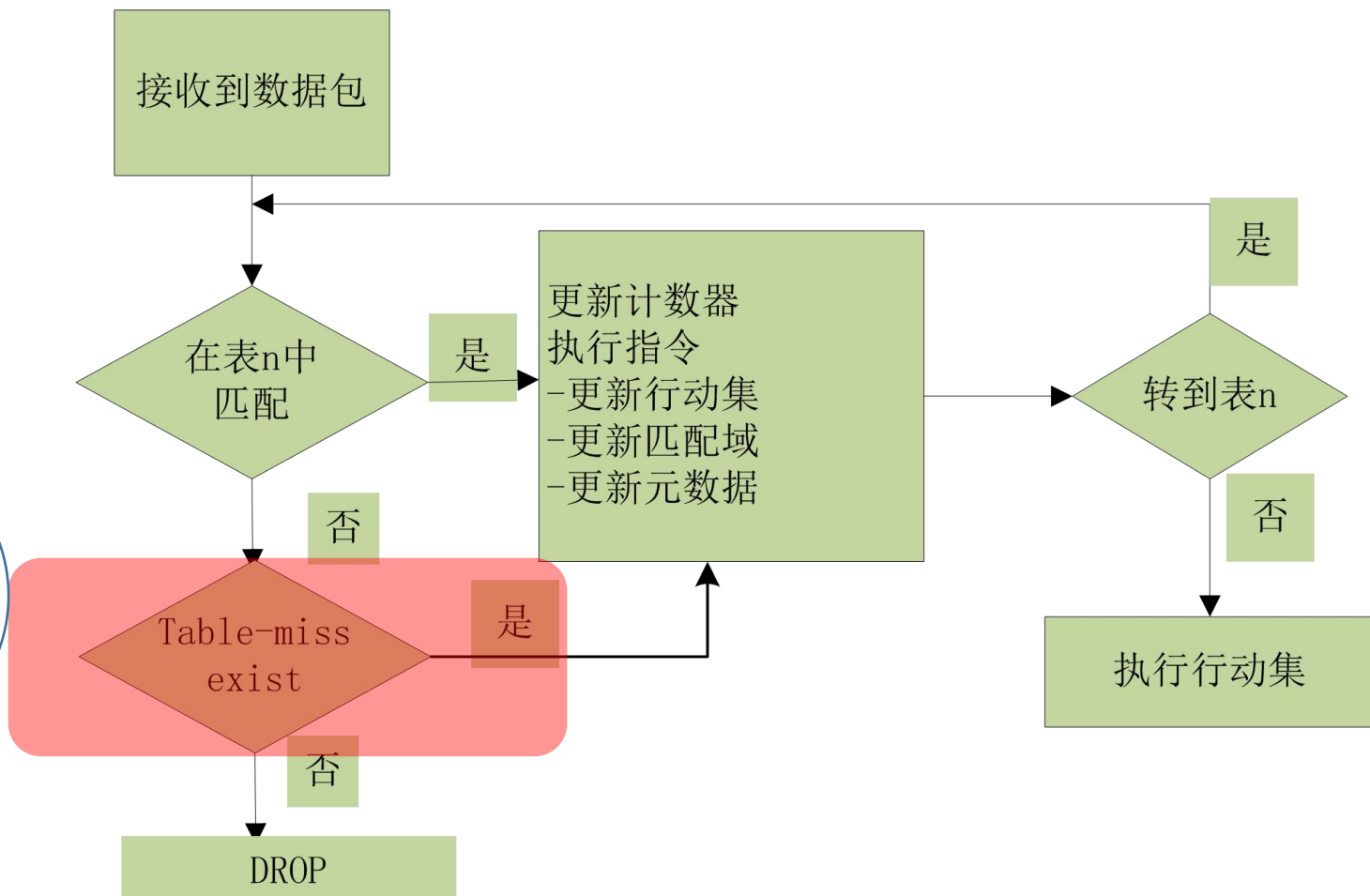


OpenFlow—协议的演进之流表匹配流程续

◆ OpenFlow1.3&OpenFlow1.4

当报文匹配失败了：

- ✓ 如果存table miss就按照该表项执行指令，一般是将报文转发给控制器、丢弃或转发给其他流表。
- ✓ 如果没有table miss表项则默认丢弃该报文。



OpenFlow—协议的演进之OpenFlow端口

- ◆ OpenFlow 1.2及后续版本提出OpenFlow端口的概念，它是OpenFlow进程和网络之间传递数据包的网络接口，OpenFlow交换机之间通过OpenFlow端口在逻辑上相互连接
 - ◆ **OpenFlow标准端口**：包括物理端口，逻辑端口，LOCAL 保留端口（假如支持）
 - ◆ 标准端口可以作为入端口匹配和出端口动作端口
 - ◆ 标准端口可以在group组中使用
 - ◆ 标准端口关联有计数器信息
 - ◆ **OpenFlow物理端口**：一般和物理硬件端口一一对应，交换机硬件接口对应的虚拟切片
 - ◆ **OpenFlow逻辑端口**：交换机定义的接口，不与硬件接口直接对应。譬如聚合端口，隧道，环回端口等
 - ◆ 尽管逻辑端口可能和实际的物理端口有各种映射，但是对于OpenFlow协议层面来说它和物理端口一样，区别在于逻辑端口数据包拥有元数据Tunnel-ID
 - ◆ **OpenFlow 保留端口**：ALL、CONTROLLER、LOCAL、TABLE、NORMAL、IN_PORT、FLOOD等端口，其在OpenFlow1.0中成为“虚拟端口”

OpenFlow—协议的演进之OpenFlow端口续

◆ OpenFlow 1.3 定义了8种端口

类型	名称	说明
必备	ALL	表示所有端口均可用于转发指定数据包。当其被用作输出端口时，数据包被复制后发送到所有的标准端口，但不包括数据包的入端口及被配置为OFPPC_NO_FWD的端口
	CONTROLLER	表示到OpenFlow控制器的控制通道，它可以用作一个入端口或作为一个出端口。当用作一个出端口时，封装数据包中为数据包消息，并使用OpenFlow协议发送。当用作一个入端口时，确认来自控制器的数据包
	TABLE	表示OpenFlow流水线的开始。这个端口仅在输出行为的时候有效，此时OpenFlow交换机提交报文给第一流表使数据包可以通过OpenFlow流水线处理（转发给某一个流表）
	IN PORT	代表数据包进入端口。只有一种情况用于输出端口，即从入端口发送数据包
	ANY	某些OpenFlow命令中的特定值，用在没有指定端口的（端口通配符）情形，不能用于入端口和出端口
可选	LOCAL	表示交换机的本地网络堆栈和管理堆栈。可以用作一个入端口或者一个出端口。该端口使得远程实体可以与交换机通过OpenFlow网络交互，而不再需要单独的控制网络。通过配置一组合适的默认流表项，该端口可用于实现一个带内控制器的连接
	NORMAL	代表传统的非OpenFlow流水线处理。仅可用作普通流水线的输出端口。如果交换机不能从OpenFlow流水线转发数据包到普通流水线，它必须表明它不支持这一动作
	FLOOD	表示使用普通流水线处理洪泛过程。可用于作为一个输出端口，除入端口或OFPPS_BLOCKED状态的端口外，可以将数据包发往其他所有标准端口。交换机也可以通过数据包的VLAN ID选择哪些端口洪泛

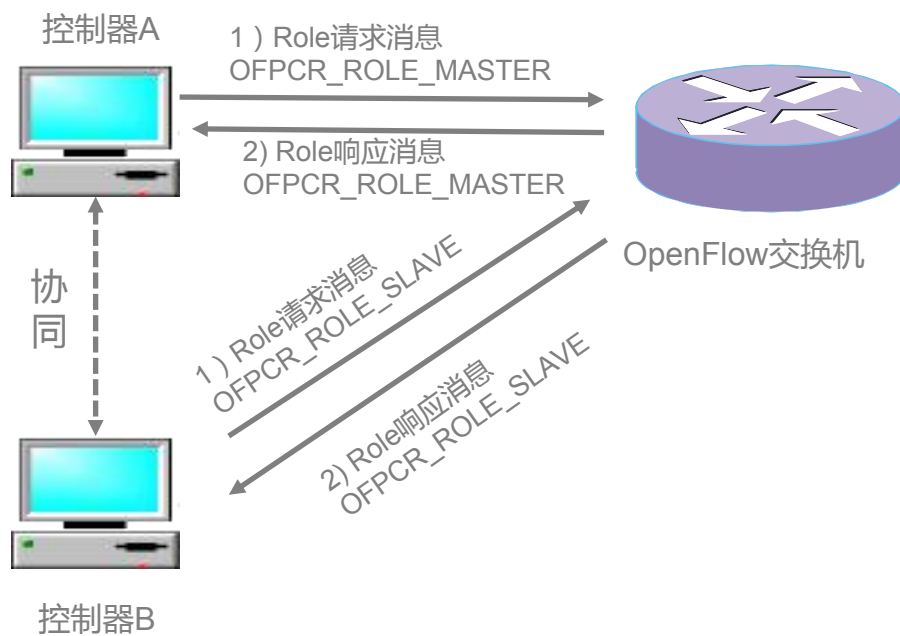
OpenFlow—协议的演进之多控制器支持

- ◆ OpenFlow采用集中化的控制方式，一旦控制器出现故障或者其与交换机之间的连接中断，将会对整个网络造成影响
- ◆ OpenFlow 1.2引入多控制器的理念，希望通过多个控制器的协同工作提高全网的可靠性
 - ◆ 多控制器支持
 - ◆ 多个控制器之间可以提供负载均衡能力和快速故障倒换，同时增加角色类消息用于控制器之间协商
 - ◆ 控制器的角色
 - ◆ EQUAL（缺省），在此状态下的控制器可以响应来自OpenFlow交换机发来的请求
 - ◆ SLAVE，在此状态下控制器只负责监听，只响应交换机发送port-status消息
 - ◆ MASTER，这种状态下的控制器行为与EQUAL类似，唯一的差异在于系统中只能有一台MASTER

OpenFlow—协议的演进之多控制器支持续

◆ OpenFlow控制器之间的协作

- ◆ 当MASTER角色的控制器出现故障时，其它OpenFlow控制器将自身的角色变成MASTER，并将自己的角色通知交换机



OpenFlow—协议的演进之总结

◆ 两个方面

◆ 增强控制面，使系统更可靠稳定，功能更丰富，更灵活

◆ 1.2版本多控制器支持

◆ 增强转发面，丰富匹配域，提升数据包匹配效率，提升流表转发性能

◆ 1.0 版本中只有单流表，为了避免单流表膨胀，

◆ 1.1版本增加组表，采用多级流表，以流水线的形式提高数据包匹配效率

◆ 1.2版本采用TLV OXM扩展匹配扩展，丰富匹配域

◆ 1.3版本增加Meter表，提升了流转发的性能

◆ 1.5版本增加数据包类型识别流程，提升数据包的处理效率

目录

01

OpenFlow总述

02

OpenFlow1.0 规范

03

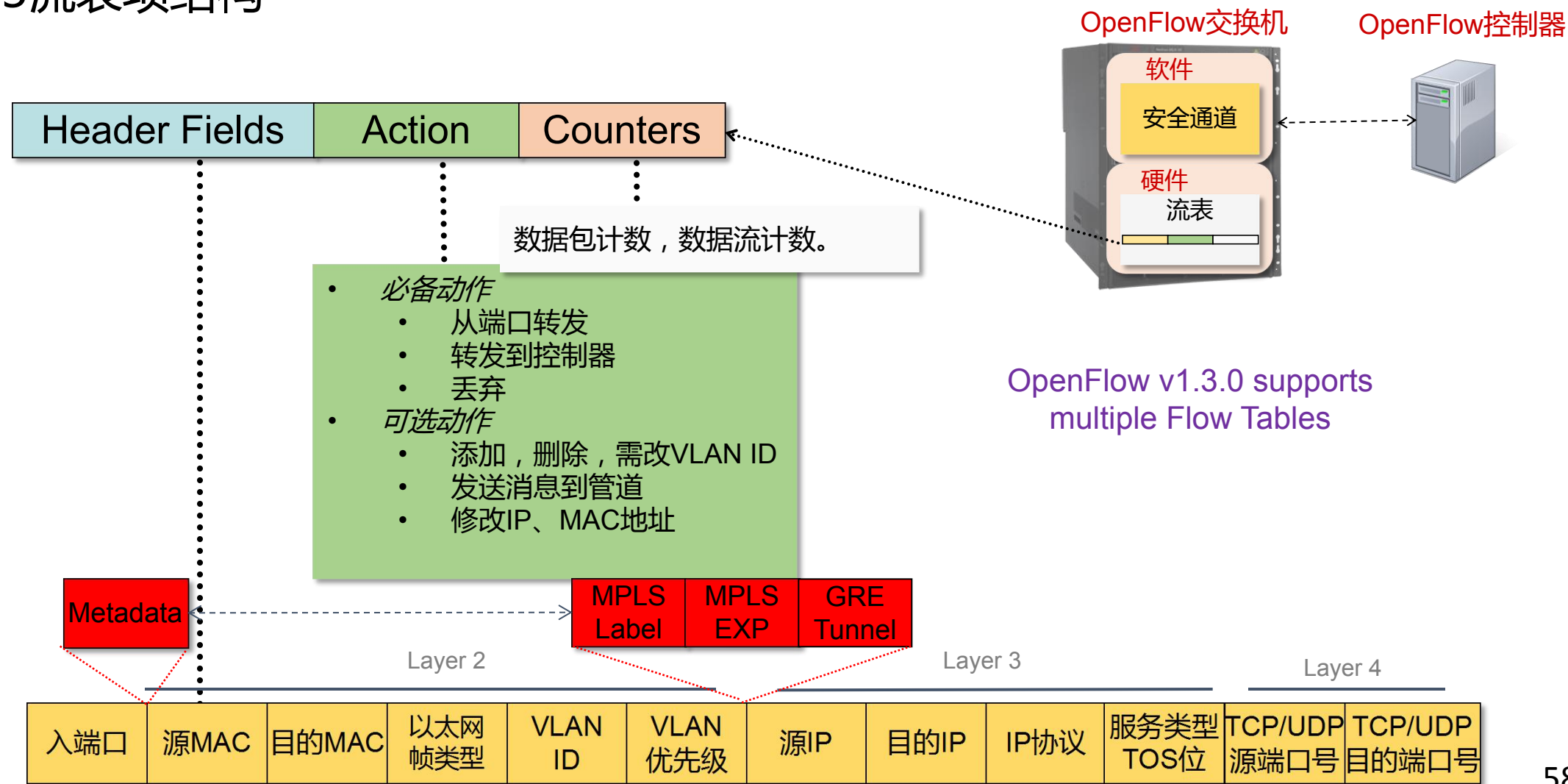
OpenFlow流表演进

04

OpenFlow1.3 规范

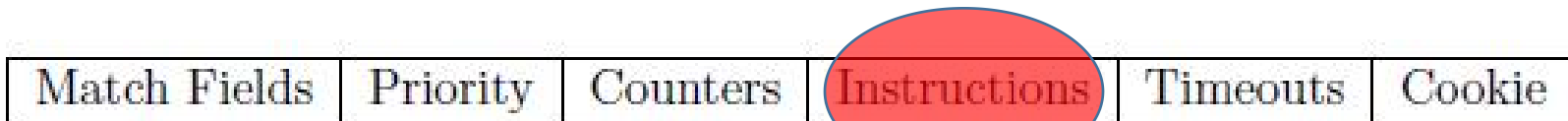
南向接口协议：OpenFlow1.3流表

1.3流表项结构



OpenFlow 1.3 — 指令、行动、行动集

- ◆ **指令**：是编辑行动集和元数据、执行行动列表、转移到下一个流表而准备的命令



指令	参数	是否必须支持
Meter	meter_id	optional
Apply-Actions	action(s) [action list]	optional
Clear-Actions	N/A	optional
Write-Actions	action(s) [action set]	required
Write-Metadata	metadata/mask	optional
Goto-Table	next-table-id	required

行动：

- ◆ Output
- ◆ Set-Queue
- ◆ Drop
- ◆ Group
- ◆ Push-Tag/Pop_tag
- ◆ Set-Field
- ◆ Change-TTL

行动集：

- ◆ 暂时存储多个行动的集合
- ◆ 默认为空。在流水线处理时，根据指令在行动集中添加、删除、更新行动
- ◆ 行动集在最后执行时，按照一定的顺序执行

OpenFlow 1.3 组表 —节约Action资源 (SRAM)

组表结构

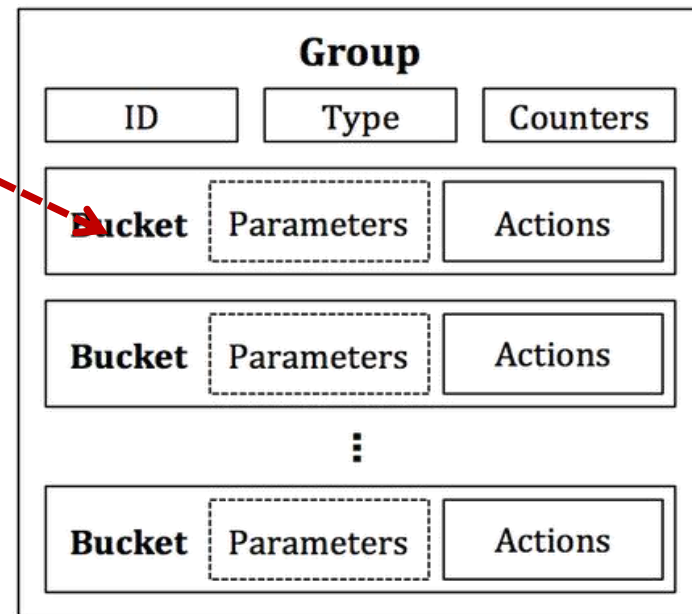


all : 执行action buckets中的所有动作，可以用于组播

select : 随机执行action buckets中的一个动作，可以用于多径

indirect : 只包含一个Action列表的组表，效率更高，可以用于路由聚合

fast failover : 执行action buckets中的第一条生存条目，可以用于快速故障恢复



```
ovs-ofctl add-group s2 group_id=1,type=all,bucket=output:2,bucket=output:3 -O  
OpenFlow13
```

```
ovs-ofctl dump-groups s2 -O OpenFlow13
```

```
OFPST_GROUP_DESC reply (OF1.3) (xid=0x2):
```

```
group_id=1,type=all,bucket=actions=output:2,bucket=a
```

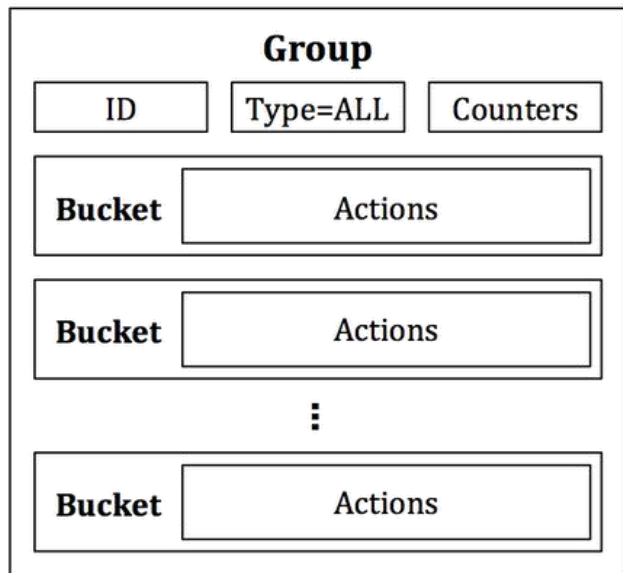
```
ctions=output:3
```

```
OVS-OFCTL ADD-FLOW BR-SW
```

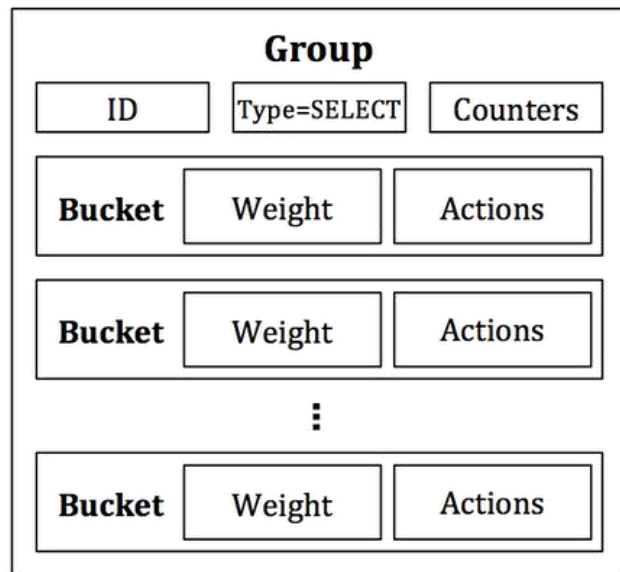
```
IN_PORT=2,ACTIONS=group:s2,outpu:3
```

OpenFlow 1.3 组表 —节约Action资源续

IN_port =1 action=group:1

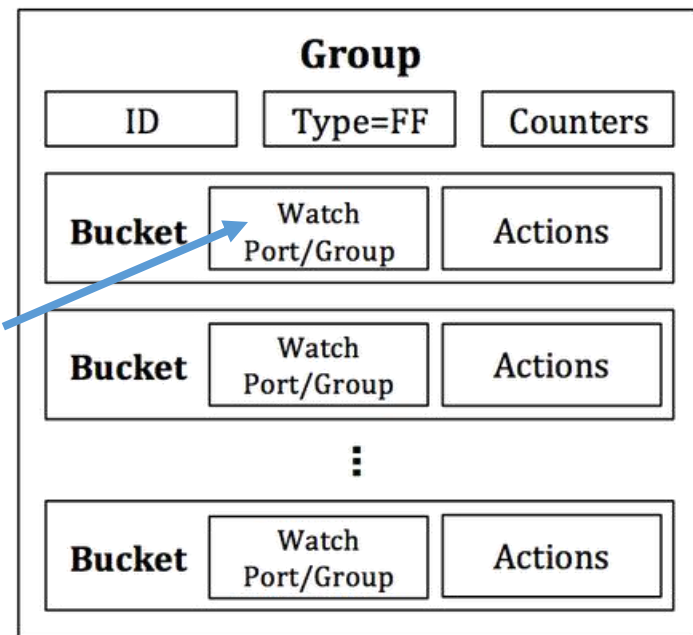
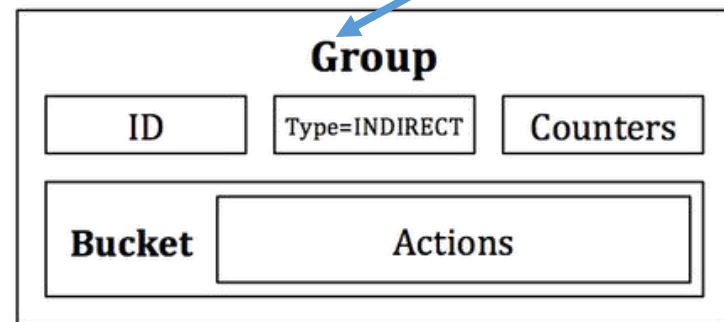


包复制到每个桶，
所有都执行
桶没有参数



包选择到特定的桶，
只有一个桶执行
有加权参数
选择的算法依赖实现

为share相同fate的包/流提供统一的执行桶
只有一个执行桶
可以降低流表条目的占用量



检测端口或者组
的存活信息
有序选择第一个
活着的bucket

OpenFlow 1.3 Meter表 — 性能管理

Meter 表结构

Meter Identifier	Meter Bands	Counters
------------------	-------------	----------

Band Type	Rate	Counters	Type specific arguments
-----------	------	----------	-------------------------

- drop: 丢弃数据包(用于流量限速)
- dscp remark: 增加IP报文头部的dscp字段. 可用于简单的DiffServ 策略

流限速:

```
ovs-ofctl add-meter br0 meter=2,kbps,band=type=drop,rate=30000
```

```
ovs-ofctl add-flow br0 in_port=1,actions=meter:2,output:2
```

```
ovs-ofctl add-meter br0 meter=3,kbps,band=type=dscp_remark,rate=30000,prec_level=14
```

```
ovs-ofctl add-flow br0 in_port=2,actions=meter:3,output:1
```

OpenFlow1.3— meter限流 demo

- ◆ 在和控制连接的情况下用ovs-ofctl dump-flows 显示当前流表
- ◆ 清除交换机上流表

ovs-ofctl del-flows br0

```
admin@PicOS-OVS$ ovs-ofctl dump-flows br0
OFPST_FLOW reply (OF1.3) (xid=0x2):
  cookie=0x2b00000000000007, duration=156.736s, table=0, n_packets=n/a, n_bytes=0, priority=100, dl_type=0x88cc actions=CONTROLLER:65535
  cookie=0x2b00000000000007, duration=156.736s, table=0, n_packets=n/a, n_bytes=0, priority=0 actions=drop
admin@PicOS-OVS$
```

在host上使用

iperf的server端

iperf -s -p 12000 -u -P -i 1 -f -k

客户端

iperf -c 192.168.1.2 -P 1 -i 1 -p 12000 -f -k -t 10 -u -b 10000k

添加meter表：

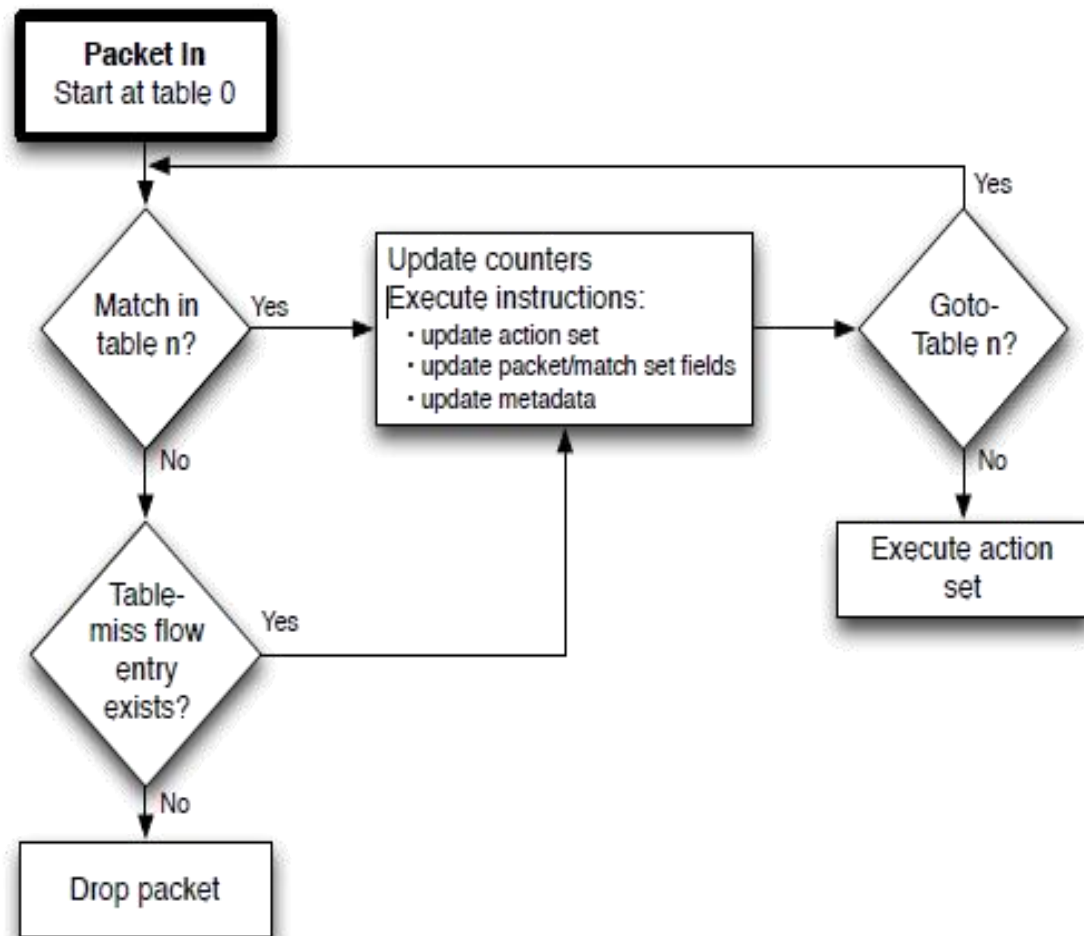
ovs-ofctl add-meter br0 meter=2,kbps,burst,band=type=drop,rate=7000,burst_size=7000

ovs-ofctl add-flow br0 in_port=3,actions=meter:2,output=8

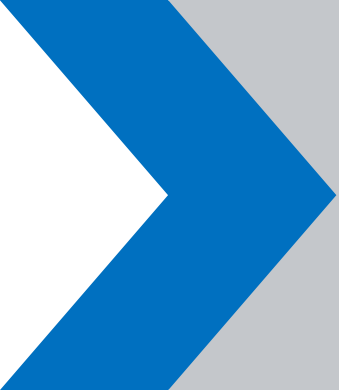
ovs-ofctl add-flow br0 in_port=8,actions=output=3

再次使用上面的命令验证限速情况

OpenFlow 1.3—流水线处理



- 首先从table 0开始查找匹配
- 假如查找到匹配流表条目，执行与之相关定义的指令
- 指令中如果包括apply-actions，需要立即执行
- 指令中如果包括write-actions，写入动作集 (action set)
- 指令中如果包括clear-actions，清除动作集
- 指令中如果包括goto-table，跳到下一张流表继续处理
- 直到流水线操作完成，执行动作集



谢谢！